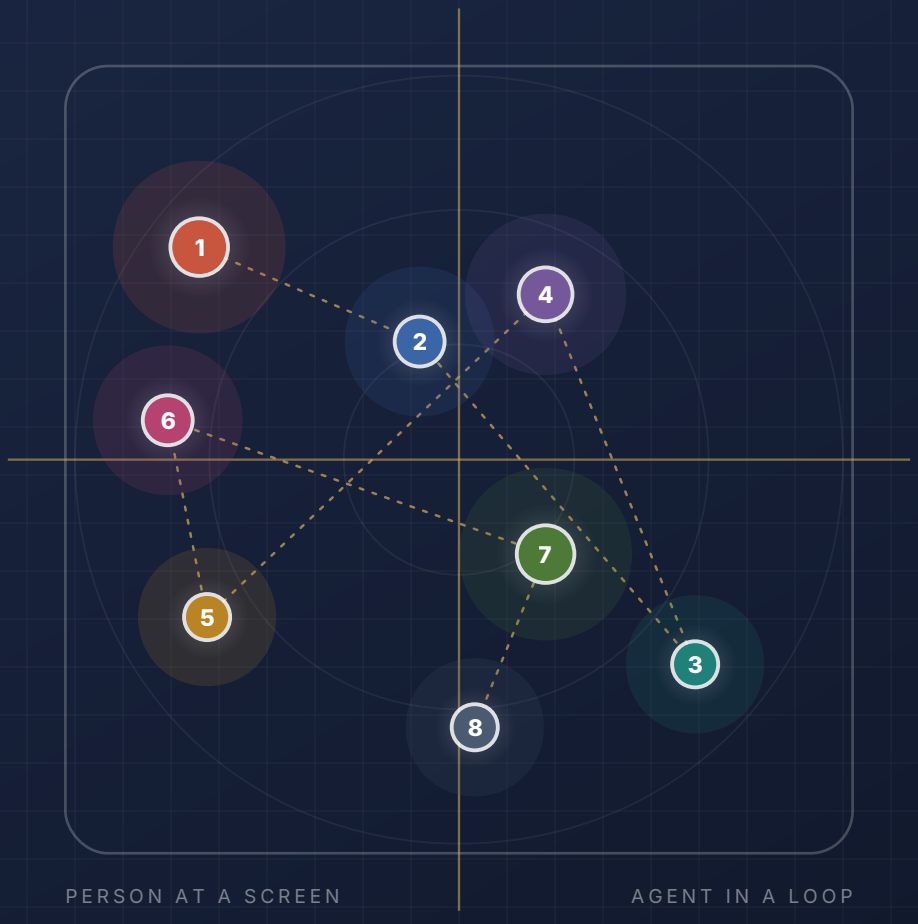
A PLAIN-ENGLISH STUDY GUIDE



The AI Business *Blueprint*

Eight Models for Building a Company in the Age of AI Agents

WORKSHEETS · REVIEW QUESTIONS · 90-DAY PLANS

ANSWER KEY INCLUDED

The AI Business Blueprint

A PLAIN-ENGLISH STUDY GUIDE

The AI Business Blueprint

Eight Models for Building a Company in the Age of AI Agents

The AI Business Blueprint: Eight Models for Building a Company in the Age of AI Agents

Copyright © 2026 the Author. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise) without the prior written permission of the copyright holder, except for brief quotations in reviews and the reproduction of the worksheets by the original purchaser for personal use.

Disclaimer. This book is for educational and informational purposes only. It is not legal, tax, accounting, investment, or financial advice. Business outcomes depend on many factors, and no result is guaranteed. Figures labeled “illustrative” or “estimate” are teaching examples, not forecasts or survey data. Laws on privacy, data, licensing, and professional services vary by location and change over time; consult a qualified professional before acting on any matter with legal or financial consequences.

A note on names. To keep this guide neutral and durable, it describes products, platforms, and companies by category (for example, “a payment processor” or “a no-code app builder”) rather than by brand. Technical standards, laws, and licenses are referred to by their common names.

Sources. The eight-model framework explored here was inspired by a public talk on AI business models. All text, analysis, diagrams, charts, examples, and exercises in this edition are original.

First edition, 2026

For everyone who knows something
better than almost anyone,
and is ready to build on it.

Contents

The New Map of AI Business 3

PART ONE — SELLING EXPERTISE & OUTCOMES

1 Software-Enabled Services 11

2 Vertical Agent Harnesses 27

PART TWO — SERVING AGENTS & THEIR OWNERS

3 Headless AI Software 45

4 Agent-Native Systems 61

PART THREE — SERVING COMMUNITIES & GROUPS

5 Taste-Led Niche Products 77

6 Ambient Consumer Agents 91

PART FOUR — BUILDING ASSETS THAT COMPOUND

7 Proprietary Data Products 107

8 Open-Source-Core Products 123

PART FIVE — CHOOSING & LAUNCHING YOURS

9 How to Pick Yours 139

10 Your First Ninety Days 149

STUDY TOOLS & REFERENCE

Afterword 157

The Workbook 159

Answer Key 167

Glossary 171

Tool Categories Directory 175

Index 181

Figures & Charts

Figure I.1 The positioning map 7
Figure 1.1 Software-enabled services at a glance 12
Figure 1.2 Monthly economics of a forty-client bookkeeping practice, traditional versus AI-native (illustrative)
Figure 1.3 Clients one person can serve each month (illustrative) 16
Figure 2.1 Vertical agent harnesses at a glance 28
Figure 2.2 How model improvements affect value over time (illustrative) 32
Figure 2.3 Where an inspector's report time goes, with and without a harness (illustrative) 34
Figure 3.1 Headless AI software at a glance 46
Figure 3.2 The three doors agents use 48
Figure 3.3 Daily requests from a person versus an agent (illustrative, log scale) 48
Figure 3.4 A customer's monthly bill under pay-as-you-go pricing, with and without a spending cap (illustrative)
Figure 4.1 Agent-native systems at a glance 62
Figure 4.2 Monthly revenue from about twenty hours of the expert's time each week, under three delivery models (illustrative)
Figure 5.1 Taste-led niche products at a glance 78
Figure 5.2 Annual revenue by number of paying fans and average yearly spend 80
Figure 5.3 A healthy revenue mix for a niche product (illustrative) 86
Figure 6.1 Ambient consumer agents at a glance 92
Figure 6.2 The novelty trap as a retention curve (illustrative) 95
Figure 7.1 Proprietary data products at a glance 108
Figure 7.2 Why more data is worth more (illustrative) 112
Figure 8.1 Open-source-core products at a glance 124
Figure 8.2 An open-source funnel over a first year (illustrative) 127
Figure 8.3 Where an open-source-core company's revenue typically comes from (illustrative) 128
Figure 8.4 The license spectrum 129
Figure 9.1 The decision guide 140
Figure 9.2 Technical lift versus typical months to first revenue for each model (estimates) 142
Figure 9.3 Estimated monthly starter budgets before revenue 146
Figure 10.1 The first ninety days at a glance 150
Figure 10.2 One common stacking path 153

How to Use This Study Guide

This book is meant to be read, marked up and worked through. Each chapter explains one business model in plain English and then turns it into a plan you can follow, with tools described by category so the advice stays useful as products come and go.

What you'll find in every model chapter

Section	What it does for you
Learning objectives	Tells you what you should be able to explain or do by the end of the chapter.
The model at a glance	A one-page visual summary: the core diagram plus the eight facts that define the model (why now, an example, who builds it, how it earns, the trap, and more).
Key terms	Definitions of the vocabulary used in the chapter. Every term also appears in the Glossary.
The model explained	A careful walk-through of the idea, the diagram, and the economics, with charts.
Case study	A worked example, written out from problem to price.
Step-by-step plan	Phases you can follow, with the categories of tools you'll need at each step.
Risks and budget	What usually goes wrong, how to avoid it, and what it costs to start.
Seven-day checklist	Concrete actions for your first week.
Review questions	Multiple-choice and short-answer questions to check your understanding. Answers are in the Answer Key.
Apply it worksheet	A page to write your own version of the model.

Three ways to read it

If you want the big picture, read the Introduction and Chapter 9, "How to Pick Yours," first. Together they give you the whole map in under an hour. Then read the chapter for the model that fits you best.

If you're studying, read the chapters in order. Do the review questions before you check the Answer Key, and fill in each worksheet before moving on. The models build on one another, and later chapters refer back to earlier ones.

If you're ready to start, go straight to the Workbook at the back. Complete the Advantage Inventory and the Model Scorecard, choose a model, and use Chapter 10's ninety-day plan alongside your model's chapter.

A few conventions

- **"Illustrative"** on a chart or table means the numbers are realistic teaching examples, not measured data. Use them to understand the shape of an idea, then replace them with your own numbers.
- **Prices and budgets** are given in U.S. dollars and reflect typical costs for small, early-stage businesses. Expect them to change.
- **Tools are named by category**, such as "an AI assistant subscription," "a no-code app builder," or "a payment processor." The Tool Categories Directory in the back explains what each category does and what to look for when choosing one.
- **Boxes** marked *Plain English* restate a tricky idea simply. *Watch Out* boxes flag the trap for each model, and *Try This* boxes give you a small action to take right away.

TRY THIS BEFORE CHAPTER 1

On a blank page, write three answers: (1) something you know better than almost anyone, (2) who already pays for help with that today, and (3) whether you'd rather build something people use or something that works on its own. You'll come back to these three answers throughout the book.



The New Map of AI Business

When anyone can build, the advantage shifts to what you know, who you serve, and what you own.

IN THIS INTRODUCTION

- Why the rules for starting a business changed when AI got cheap and capable
- The eight business models this book covers, in one table
- How to read the positioning map that appears on every chapter opener
- The single question that decides which model fits you

For most of the last twenty years, building a software business meant hiring engineers, raising money, designing an app, and then spending a fortune convincing strangers to download it. Running a service business meant the opposite problem: every new client required another person on payroll. Both paths were slow, expensive, and closed to most people.

That has changed, and quickly. AI models can now read, write, summarize, analyze, plan, and take actions in other software. AI coding assistants let people with no programming background build working applications. AI *agents*, meaning programs that pursue a goal by deciding on steps and using tools without a person clicking every button, are becoming everyday users of software. The cost of producing code, content, and routine professional work has fallen sharply.

When something becomes cheap, value moves to whatever is still scarce. This book is about where it moved, and about the eight distinct ways ordinary people, not just engineers or venture-backed founders, can build a business on that shift.

What Became Cheap, and What Stayed Scarce

Every model in this book rests on one observation: the things that used to be the hard part of building a business are now easy, and the things that used to be taken for granted are now the advantage.

TABLE I.1 • THE GREAT RE-PRICING

Became cheap and abundant	Stayed scarce and valuable
Writing code and building apps	Knowing exactly what a niche needs
Drafting documents, reports, and content	Judgment about what's correct, safe, and good
General knowledge (what any AI model knows)	Fresh, specific, private, or niche data
Routine, repetitive professional tasks	Client relationships and trust
Features (they can be copied in days)	Communities, distribution, and owned audiences
Generic tools anyone can access	Taste, obsession, and insider understanding

Read the right-hand column again. None of it requires a computer science degree. It requires experience, relationships, a community, a method, or a passion. That's why this moment favors people who know a field deeply over people who only know technology.

The model is the engine. Your knowledge, data, community and method are what turn it into a business.

The Eight Models in One Table

Each chapter in Parts One through Four covers one of these models in depth. Read the table now for orientation. You don't need to memorize it, because Chapter 9 returns to it as a decision tool.

TABLE I.2 • THE EIGHT MODELS AT A GLANCE

Model	The core idea	Best if you have	How it earns
1. Software-enabled services	Sell finished work, delivered by a tiny team running on AI	Service experience and clients	Retainers, per project, per outcome
2. Vertical agent harnesses	Wrap an AI model in the tools, knowledge, and workflow of one niche	Deep niche expertise	Premium subscription, per job
3. Headless AI software	Build tools for AI agents to use, with no screen at all	A data source or one hard, narrow skill	Usage-based, credits, open-source core
4. Agent-native systems	Package your method so it installs into a customer's AI assistant	A repeatable method	Plugin access, membership, dashboard
5. Taste-led niche products	Build for the thousand true fans of a passion you share	A passion and a community	Subscription, purchases, sponsors, affiliates
6. Ambient consumer agents	An AI that lives in a group chat and gets things done	A group whose rituals you know	Per-group subscription, booking fees
7. Proprietary data products	Collect data no one else has, then sell the answers	Access to data nobody captures	Insights, reports, API access
8. Open-source-core products	Give the code away; sell hosting, features, and support	Comfort building in public	Hosted cloud, paid tiers, support

How to Read the Map

Every chapter opener shows a small square with eight dots, one of them highlighted. That's *the map*, and it places each model according to two questions.

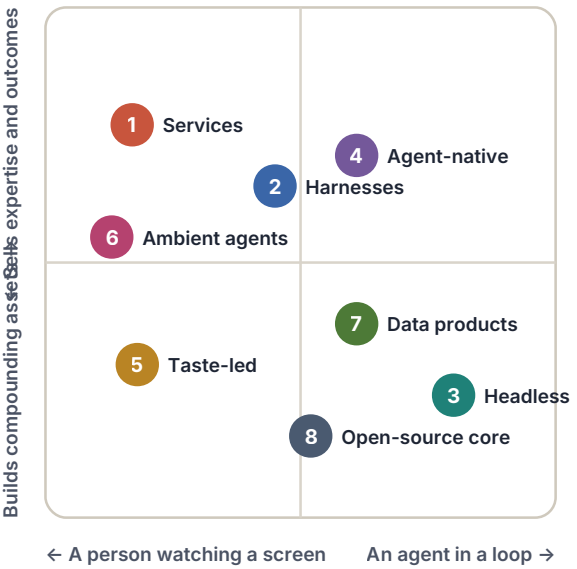


Figure I.1 The positioning map. The horizontal axis asks who does the work: a person watching a screen (left) or an agent working in a loop (right). The vertical axis, as this guide reads it, separates models that sell expertise and outcomes (top) from models that build compounding assets such as code, data, and community (bottom).

The horizontal axis: who does the work? On the left, a person does the work, with AI as a helper, and the product is something people look at and interact with. On the right, an AI agent does the work on its own, running in a loop: it acts, checks the result, and acts again while people supervise or aren't involved at all. Left-side businesses succeed through user experience and relationships. Right-side businesses succeed through reliability, documentation, and scale.

The vertical axis: what are you really selling? Models near the top sell expertise and outcomes directly: finished work, a professional's workflow, a coach's method. Models near the bottom build assets that grow in value over time, such as a community of fans, a body of unique data, or widely adopted code. Neither half is better. Top-half models usually earn sooner, and bottom-half models are usually harder to copy once established.

PLAIN ENGLISH

Left means people use it; right means agents do it. Top means you sell what you know; bottom means you build something that keeps growing in value.

The One Question That Matters Most

The rest of this book can be boiled down to a single question, which Chapter 9 turns into a full decision process: *What do you know better than almost anyone, and who already pays for it?*

Your answer points to your model. If people pay firms or freelancers for what you know, start with Model 1. If professionals pay for tools, look at Models 2 through 4. If fans and groups spend money on your passion, look at Models 5 and 6. If businesses buy the kind of information you can gather, look at Model 7. And if you enjoy building openly with a community of developers, look at Model 8.

You won't pick wrong by reading all eight. Many successful businesses combine two or three of them, starting with one that earns money quickly and growing into one that's harder to copy. Chapter 10 shows how to stack them.

KEY TAKEAWAYS

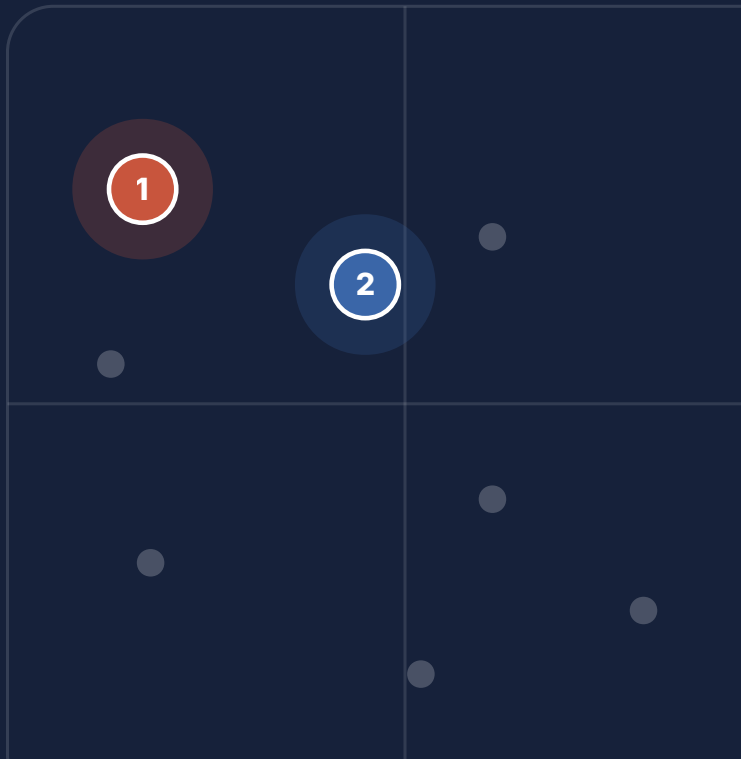
- AI made code, content, and routine work cheap. Value moved to knowledge, judgment, data, trust, community, and taste.
- There are eight distinct ways to build on that shift. Each favors a different kind of existing advantage.
- The map sorts the models by who does the work (person or agent) and what is sold (expertise or compounding assets).
- Your best starting model is the one that matches what you already know and who already pays for it.

PART ONE

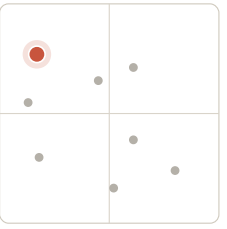
Selling Expertise & Outcomes

Two models for people who already know how professional work gets done. The first sells the finished work itself, delivered by a tiny team running on AI. The second packages a niche's knowledge and workflow into software that professionals pay for.

01 Software-Enabled Services
02 Vertical Agent Harnesses



CHAPTER 1 • MODEL 1 OF 8



1

Software-Enabled Services

Don't sell professionals software. Become the firm that's ten times faster.

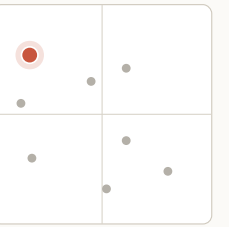
LEARNING OBJECTIVES

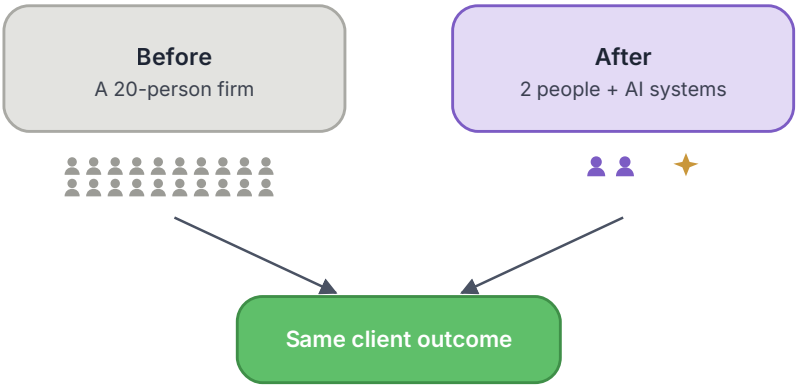
- Explain why selling finished work can beat selling software
- Calculate how AI changes the economics of a service firm
- Choose a niche using five screening questions
- Map a deliverable, automate 80% of it, and keep quality high
- Price, launch, and scale without turning back into an agency

TYPE 1 OF 8

Software-enabled services

Don’t sell professionals software. Be the firm that’s ten times faster.





The client sees the result, not the headcount.

Why now
Clients want the job done. AI lets two people deliver what used to take twenty.

Real-world example
AI-native firms already operate in contract review and in bookkeeping for startups.

Proof it works
Product studios now run client work on internal AI “operating systems” instead of large teams.

Who builds it
Anyone who has run, or worked in, a service business.

How it earns
Retainers, per-project fees, or per-outcome pricing.

The trap
Quietly turning back into an agency with headcount.

This week
Time one deliverable by hand. Build the system that does 80% of it.

In one line
Same outcome, a fraction of the people.

Figure 1.1 Software-enabled services at a glance. The client gets the same outcome whether a twenty-person firm or a two-person team with AI systems produces it. What changes is the cost structure behind it.

Many people who see what AI can do decide to build software and sell it to accountants, lawyers, marketers, or recruiters. It's a hard road. Professionals are slow to adopt new tools, sales cycles are long, and hundreds of funded startups are already competing for the same customers. This chapter describes the opposite approach: skip the software sale and sell the finished work.

The client doesn't want a bookkeeping tool. They want their books closed. They don't want contract-review software. They want to know whether the contract is safe to sign. A *software-enabled service* is a business that sells that result, like any traditional firm, but produces it with a very small team running on AI systems. From the outside it looks like a normal professional firm. Inside, it runs like a technology company.

KEY TERMS

Software-enabled service
A business that sells finished professional work (not software) and uses AI systems internally to produce it with far fewer people.

Deliverable
The specific thing a client receives: closed books, a reviewed contract, a monthly report, a set of articles.

Retainer
A recurring monthly fee for an agreed scope of ongoing work.

Per-outcome pricing
Charging for a result, such as a hire made or a claim resolved, rather than for time spent.

Human-in-the-loop review
A checkpoint where a qualified person checks AI-produced work before it reaches the client.

Revenue per person
Total revenue divided by the number of people on the team. It's the key health metric for this model.

The Model Explained

Reading the diagram

The hero diagram in Figure 1.1 has three parts. On the left, **Before**: a traditional firm needs roughly twenty people to serve its clients. On the right, **After**: an AI-native firm delivers the same work with two people supported by AI systems. Both arrows point to the same box, **Same client outcome**. That's the whole idea. The client experience stays the same, or gets better because work arrives faster, while the cost of producing it collapses.

The eight facts, unpacked

Why now. Clients have always wanted the job done rather than a tool to learn. Until recently, doing the job meant hiring people, because only people could read documents, make judgment calls, draft, and check. AI can now do most of the repetitive portion of that work: categorizing, extracting, summarizing, drafting, formatting, and checking against a list. A small team can now deliver at a scale that once required a floor of staff.

Real-world examples. Firms built this way already exist. One kind of AI-native law practice focuses narrowly on reviewing contracts quickly. Another kind of firm handles bookkeeping for startups using its own software plus a small accounting team. Both look like service firms to their customers and run like software companies inside.

Who builds it. Anyone who has run or worked in a service business. Your advantage isn't technical skill. It's knowing the work well enough to see which 80% is repetitive and which 20% needs judgment.

How it earns. Like a service business: monthly retainers, per-project fees, or per-outcome pricing. You aren't charging software subscriptions, which means you can charge what the outcome is worth to the client.

The trap. When demand grows, the instinct is to hire people instead of improving systems. Do that, and you've rebuilt a normal agency, with normal thin margins and normal management headaches. The advantage disappears.

This week. Take one real deliverable, do it by hand, and time every step. Then build a system (AI plus simple automations) that does 80% of it, while you review and finish the last 20%.

Why the economics are so attractive

Consider a bookkeeping practice serving small businesses. A traditional bookkeeper can handle roughly ten to fifteen clients a month. Forty clients at $800 a month brings in $32,000 a month, but it takes three or four staff to serve them. After salaries, software, and overhead, the owner might keep 15 to 25 percent.

Now give one experienced bookkeeper an AI system that categorizes transactions, reconciles accounts, drafts the monthly report, and writes the client update email. The person reviews and handles exceptions. The same forty clients, the same $32,000, but the owner might keep 60 to 75 percent. Figure 1.2 shows the difference.

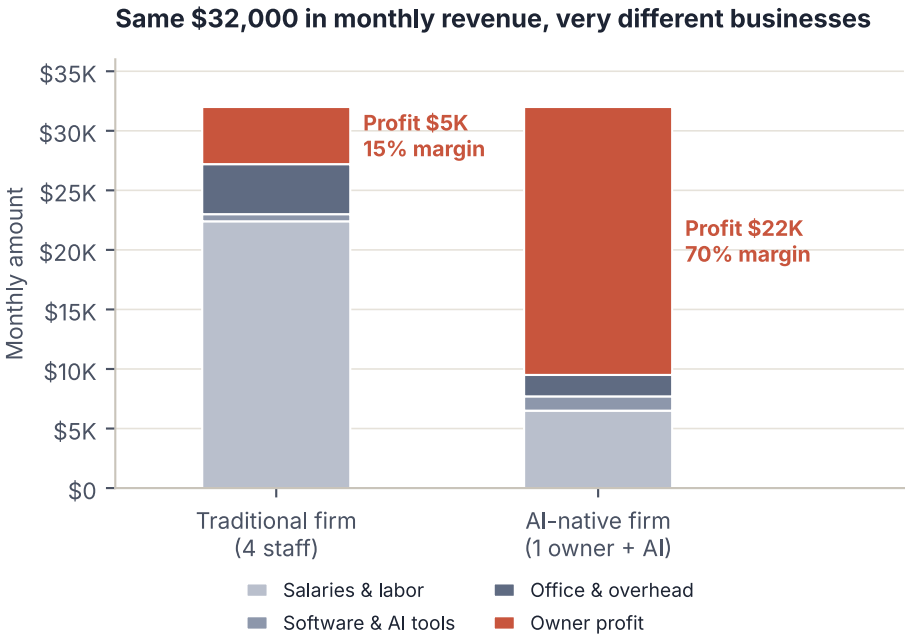


Figure 1.2 Monthly economics of a forty-client bookkeeping practice, traditional versus AI-native (illustrative). Revenue is identical; the AI-native firm converts most of it into profit because labor is its smallest cost instead of its largest.

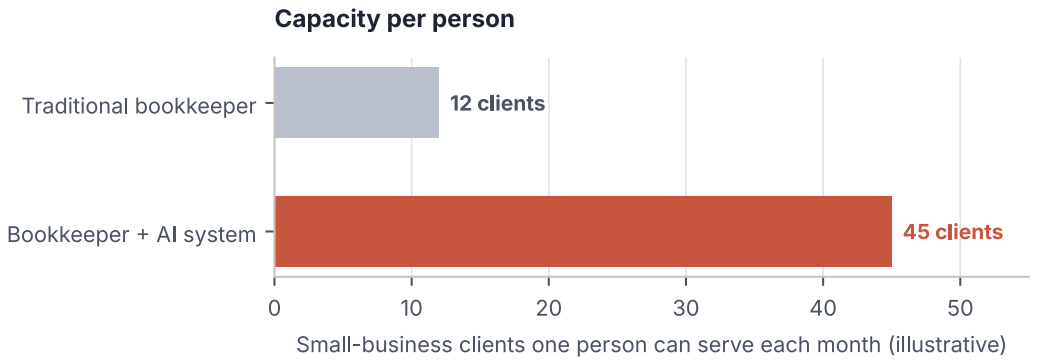


Figure 1.3 Clients one person can serve each month (illustrative). The capacity jump is what makes it possible to charge less than competitors, deliver faster, and still earn more.

This creates three advantages at once. You can **undercut competitors on price** and still earn more per client. You can **deliver faster**, in twenty-four hours instead of two weeks, which becomes your marketing message. And you can **grow without hiring**, so every new client adds mostly profit.

> PLAIN ENGLISH
>
> A traditional firm sells hours, so growing means hiring. An AI-native firm sells results, so growing means improving its systems. Same customers, same price, very different profit.

Choosing Your Niche

The best niche is one you already understand. Run each candidate through five screening questions:

1. **Do I know this work?** Have I done it, managed it, or bought it?
2. **Is it repetitive?** Is the same type of deliverable produced over and over?
3. **Is it mostly text, documents, or data?** That's where AI is strongest.
4. **Do clients already pay for it?** Look for an existing budget line, so you aren't creating a new habit.
5. **Is it unregulated, or can I partner around the regulation?** Law, tax filing, medicine, and financial advice require licenses. You can still work in these fields, but you'll need a licensed professional who supervises and signs off.

TABLE 1.1 • NICHES THAT SUIT NON-TECHNICAL FOUNDERS

Niche	Typical deliverable	Regulation note
Bookkeeping & monthly reporting	Reconciled books, one-page financial summary	Low (tax filing and audit work need credentials)
Industry-specific marketing content	Blog posts, newsletters, social posts for one industry	Low (watch advertising rules in health and finance)
Local search and AI-visibility services	Optimized listings, reviews strategy, monthly report	Low
Recruiting & candidate screening	Shortlists of screened candidates	Medium (anti-discrimination and privacy rules)
Proposal, grant & bid writing	Submitted proposals and applications	Low
Insurance, permit & paperwork processing	Completed, filed forms	Medium (some filings need licensed agents)
Real-estate transaction coordination	Deadline tracking, document packets	Medium (varies by state)
AI-first customer support outsourcing	Resolved tickets, escalation reports	Low to medium (data privacy)
Medical billing & coding	Coded, submitted claims	High (partner with a certified coder; health-privacy law applies)
Market & competitor research	Research reports and briefings	Low

Narrow it down to one service for one type of client. “Monthly bookkeeping for restaurants” is far easier to sell, systematize, and price than “accounting services.” Narrow niches also produce more consistent work, and consistent work is easier to automate.

Step-by-Step Plan

WEEK 1 — **Phase 0 — Pick your niche.** Score candidates with the five screening questions. Choose one service for one client type.

WEEKS 1–2 — **Phase 1 — Do it by hand and time it.** Deliver one real job manually and log every step.

WEEKS 2–5 — **Phase 2 — Build the system that does 80%.** Chain AI and automation tools; add a review checkpoint.

WEEK 5 — **Phase 3 — Package and price it.** One clear offer: fixed scope, fixed price, fixed turnaround.

WEEKS 5–10 — **Phase 4 — Win your first 5–10 clients.** Warm network, targeted outreach, before-and-after content.

MONTHS 3–12 — **Phase 5 — Scale with systems, not headcount.** Improve the playbook before every hire.

Phase 1: Do it by hand and time it

Get one real or practice client: a friend's business, a discounted pilot, or a side project. Do the deliverable start to finish by hand. As you work, fill in a step log like the one below. Record your screen while you work so you can review it later. You'll be surprised by how many small steps you do without noticing.

TABLE 1.2 • SAMPLE STEP LOG (MONTHLY BOOKKEEPING FOR ONE RESTAURANT)

Step	Minutes	Inputs → output	Could AI do it?
Download bank & card statements	15	Bank portals → files	Partly (automation)
Categorize ~400 transactions	110	Statements → categorized ledger	Yes, with review of low-confidence items
Match receipts to expenses	45	Receipt photos → matched records	Yes (document reading)
Reconcile accounts	40	Ledger → reconciled balances	Partly (flag differences)
Investigate oddities	30	Flags → resolved items	No (judgment and client contact)
Draft monthly summary	40	Reports → one-page summary	Yes (draft), human edits
Email client with questions	15	Open items → email	Yes (draft)

Usually 70 to 85 percent of the steps are marked “Yes” or “Partly.” Those are what you automate. The “No” steps (judgment, relationship, and exceptions) are what the client is really paying a professional for, and they’re where your time should go.

Phase 2: Build the system that does 80%

You don’t need to code. You’re connecting existing tools. For each “Yes” step, write a reusable instruction set, a *playbook*, in your AI assistant’s project or workspace feature, and include examples of excellent past output. Then connect the steps with an automation tool so that, for example, a client’s upload triggers processing, the draft lands in your review queue, and approved work is sent automatically.

TABLE 1.3 • THE TOOL STACK, BY CATEGORY

Job to be done	Tool category and what to look for
The “brain” (drafting, analysis, checking)	**AI assistant subscription** with a projects or workspace feature where you can save instructions, examples, and reference files. Choose a business plan that doesn’t train on your data.
Automation (moving data between apps)	**No-code automation platform** that connects your apps with triggers and actions. Simpler platforms are easiest; open-source, self-hostable ones are most flexible and cheapest at scale.
Client data and job tracking	**Spreadsheet-database or all-in-one workspace**: one row per client, per job, per deliverable, with a status column for your review queue.
Intake forms	**Online form builder** with file uploads and automation triggers.
Documents and file collection	**Cloud storage and office suite** with shared folders per client.
Client portal (looks professional)	**No-code portal or app builder** on top of your database, or an AI app builder.
Meeting notes	**AI meeting recorder** that turns client calls into notes and action items.
Proposals and contracts	**Proposal and e-signature software**.
Invoicing and payments	**Payment processor** with recurring billing, plus your **accounting software**.
Scheduling	**Online scheduling link** tool connected to your calendar.
Niche-specific	Bookkeeping: accounting software plus receipt-capture and AI categorization tools. Content and SEO: keyword research and content-optimization tools. Recruiting: applicant tracking systems. Support: help-desk software with AI agents.

Add a *human review checkpoint* before anything reaches a client. That review is your quality control, and it's what the client is really paying for. A simple review checklist (numbers tie out, names correct, tone right, open questions listed) catches most AI errors. When you're done, time the deliverable again. Aim for a five- to tenfold reduction: a six-hour job should take under an hour.

QUALITY CONTROL THAT SCALES

- **Confidence flags:** ask the AI to mark anything it's unsure about, and review those items first.
- **Spot checks:** review 100% of work for new clients, then a sample once the system proves reliable.
- **Error log:** every mistake becomes a new rule in the playbook, so the system gets better every month.
- **Golden examples:** keep five excellent past deliverables in the AI's reference files. Examples teach style better than instructions.

Phase 3: Package and price it

Productize the service: one clear offer with a fixed scope, fixed price, and fixed turnaround, for example, *"Monthly books closed by the 5th, with a one-page plain-English report, $750 a month."* A productized offer is easier to sell, easier to automate, and easier to deliver consistently.

TABLE 1.4 • PRICING MODELS COMPARED

Model	Best for	Example	Watch out for
Retainer	Recurring work: bookkeeping, content, support	$750/month, books closed by the 5th	Scope creep: define what's included
Per project	Proposals, audits, research reports	$1,500 per grant application	Lumpy revenue; build a pipeline
Per outcome	Results you can measure	15% of first-year salary per hire	Hardest to price; outcomes you don't control

Price against the old way, not against your cost. If a traditional firm charges $1,200 and takes three weeks, charge $800 to $1,000 and deliver in three days. Don't charge $100 just because the work costs you little. Your low cost is your margin and your growth budget, not a discount you owe anyone.

Phase 4: Win your first five to ten clients

1. **Warm network first:** former colleagues, past employers, and people in your industry.
2. **Lead lists:** use a B2B contact database to find businesses in your niche and their decision-makers.
3. **Outreach:** a short, personal email or professional-network message offering a free or discounted first deliverable. Cold-email sending tools can scale this once your message works.
4. **Content:** post short before-and-after examples: "Here's how we closed a restaurant's books in two hours instead of two weeks." Simple design and video-editing tools make this easy.
5. **Website:** a one-page site with one offer, one price, and one "book a call" button.
6. **Referral partners:** businesses that serve the same clients but don't compete with you. For bookkeeping, that could be tax preparers, business bankers, or point-of-sale vendors.

Phase 5: Scale with systems, not headcount

This is where you avoid the trap. Before every hire, ask: *Can I improve the system so I don't need this person?* Track one number above all others, *revenue per person*. Traditional service firms typically generate roughly $100,000 to $150,000 per employee. Aim for $300,000 to $1,000,000 or more. When you do hire, hire reviewers and operators who manage and improve the AI system, not people to do the work by hand.

5–10×
Target reduction in time per deliverable

60–75%
Achievable margin vs. 15–25% traditional

$300K+
Revenue-per-person goal (vs. ~$100–150K)

Over time, the internal system you've built becomes an asset in its own right. You can license it to other firms (Model 2), package your method as a plugin (Model 4), or use it to buy traditional firms and transform their economics, a strategy sometimes called an AI *roll-up*.

WATCH OUT: THE AGENCY TRAP

Signs you're sliding back into a traditional agency: revenue per person is flat or falling, you're hiring to handle volume instead of exceptions, the playbook hasn't been updated in a month, and turnaround times are getting longer. The fix is always the same. Stop, find the step that's eating time, and improve the system before adding people.

CASE STUDY

A two-person grant-writing studio

The background. A former nonprofit development director knows that small nonprofits pay freelance grant writers $2,000 to $5,000 per application and often wait weeks.

The system. An intake form collects the organization's programs, budgets, and past proposals. An AI workspace holds a playbook for each major funder type, twenty winning past proposals as examples, and a compliance checklist. The AI drafts the needs statement, program description, and budget narrative; the founder edits for strategy and voice; a part-time reviewer checks every requirement against the funder's guidelines.

The offer. "A complete, submission-ready grant application in five business days, $1,500." Monthly retainers for organizations that apply frequently.

The numbers. Time per application falls from about 30 hours to about 6. Two people deliver 25 applications a month (about $37,500) instead of the 6 or 7 a traditional freelancer could manage.

Risks and How to Handle Them

Risk	Mitigation
AI makes errors	Always keep a human review step. Use checklists and confidence flags. Never send unreviewed AI output to clients.
Licensing and regulation (law, tax, medical, financial advice)	Partner with a licensed professional who supervises and signs off, or stay in the unregulated part of the work.
Client data privacy	Use business-tier AI plans that don't train on your data. Have a confidentiality agreement and data-processing terms ready. Consider a security certification once larger clients ask.
Liability	Form a limited liability company, buy professional liability (errors and omissions) insurance, and use contracts that define scope clearly.
Turning into an agency (the trap)	Review revenue per person monthly. Fix systems before hiring.
Competitors copying you	Your moat is niche expertise, client relationships, and refined playbooks, not the AI tools, which everyone can access.

Starter Budget

TABLE 1.5 • TYPICAL MONTHLY COSTS TO START

Item	Approximate cost
AI assistant subscription (business tier)	$20–$30 per user
No-code automation platform	$20–$70
Spreadsheet-database or workspace	$0–$25
Email, office suite, and cloud storage	~$7–$14 per user
Website builder	$0–$20
Scheduling, forms, screen recording	Mostly free tiers
Business formation (one time)	~$50–$500, depending on state
Professional liability insurance	~$40–$100
Total to start	**Roughly $100–$300 a month**, far less than one employee

Your Seven-Day Checklist

- ☐ **DAY 1** List three service businesses you know from experience.
- ☐ **DAY 2** Score them with the five screening questions and pick one.
- ☐ **DAY 3** Find one pilot client, free or discounted.
- ☐ **DAY 4** Deliver one job by hand, logging every step and its time.
- ☐ **DAY 5** Mark which steps AI could do (Yes / Partly / No).
- ☐ **DAY 6** Build an AI workspace with instructions and examples for the biggest step.
- ☐ **DAY 7** Redo the deliverable with AI and compare the time.

CHAPTER SUMMARY

- Sell the outcome, not the software. Clients pay for results, not for how you produce them.
- AI turns a service firm's biggest cost, labor, into its smallest, which can lift margins from about 20% to about 70%.
- Pick one service for one client type that you know, that's repetitive and document-heavy, and that clients already pay for.
- Do the work by hand first, then automate the 80% and keep a human on the 20% that needs judgment.
- Price against the old way, deliver faster, and measure revenue per person to avoid becoming an agency.

Review Questions

1. What does a software-enabled service sell?

   A. Software licenses to professional firms
   B. Finished professional work, produced with AI systems and a small team
   C. Training courses on AI tools
   D. Hourly consulting

2. In the chapter's bookkeeping example, what mainly explains the jump in profit margin?
   A. Higher prices
   B. More clients
   C. Labor costs fall because one person can serve far more clients
   D. Cheaper office space
3. Which is the "trap" for this model?
   A. Charging too much
   B. Quietly turning back into an agency with headcount
   C. Using too many AI tools
   D. Choosing a niche that's too narrow
4. Why should you price against the old way rather than against your cost?
5. Name the five screening questions for choosing a niche.
6. What is a human review checkpoint, and why is it essential?
7. What metric best shows whether you're scaling with systems or with headcount? What target range does the chapter suggest?
8. Give two ways to handle a niche that requires a professional license.

APPLY IT • WORKSHEET 1

Design your AI-native service

1. The service, and the one client type I'll serve:

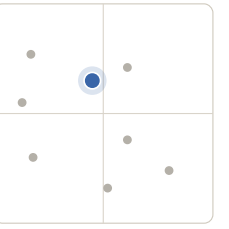
2. The deliverable, described as a productized offer (scope, price, turnaround):

3. The steps AI can do (Yes / Partly) and the steps that need my judgment (No):

4. What the old way costs the client, and what I'll charge:

5. My first three potential clients or referral partners:

CHAPTER 2 • MODEL 2 OF 8



2 Vertical Agent Harnesses

The model is the engine. The harness is the car built for one road.

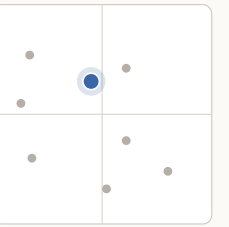
LEARNING OBJECTIVES

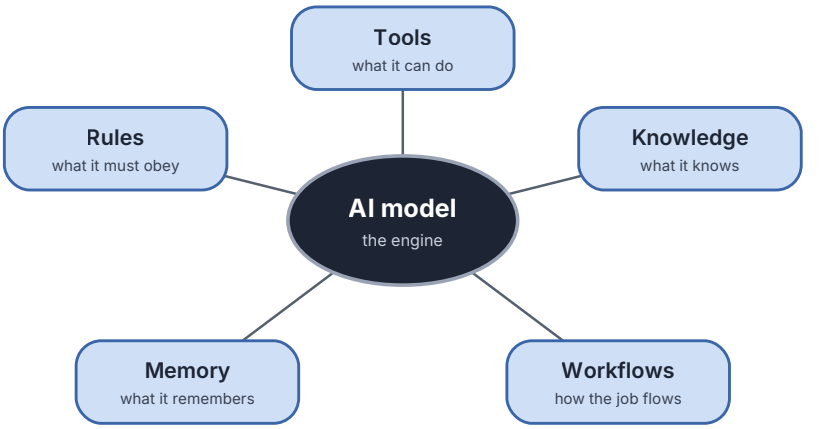
- Define a vertical agent harness and its five components
- Tell the difference between a true harness and a thin wrapper
- Find the most painful job in a niche with the “ten weekly jobs” exercise
- Prototype a harness without code, then turn it into software
- Build a moat that grows stronger as AI models improve

TYPE 2 OF 8

Vertical agent harnesses

The model is the engine. The harness is the car built for one road.





The harness: built around the model for one niche

Why now
Models are brilliant generalists. The gap is niche context and workflow.

Real-world example
AI products built for legal work, for writing clinical notes from doctor visits, and for drafting injury-claim demand letters.

Proof it works
A harness built only for people who run tabletop role-playing games, a small but passionate niche.

Who builds it
Deep experts in a niche the big players will ignore.

How it earns
Premium subscription or per-job pricing.

The trap
A thin wrapper. If a better model kills it, it wasn't a harness.

This week
List ten weekly jobs in your niche. Build a harness for the most painful one.

In one line
Whoever encodes the niche wins it.

Figure 2.1 Vertical agent harnesses at a glance. A general AI model sits at the center; five components built for one niche (tools, knowledge, rules, memory, and workflows) turn it into a product professionals pay for.

Today's AI models, such as the ones behind popular chat assistants, are engines: enormously powerful and completely general. But an engine on its own doesn't get anyone to work. You need a car around it, with steering, seats, brakes, and a map. This chapter is about building that car for one specific road.

A *vertical agent harness* is software that wraps an AI model in everything it needs to do one niche's job extremely well, so well that a professional in that niche will pay for it instead of using a general chat assistant. "Vertical" means focused on a single industry or type of user. Its opposite, "horizontal," means built for everyone, like a general-purpose assistant.

KEY TERMS

Vertical vs. horizontal
Vertical products serve one industry deeply; horizontal products serve everyone broadly.

Harness
The tools, knowledge, rules, memory, and workflows built around an AI model for a specific job.

Thin wrapper
A product that is little more than a prompt and a nice screen on top of a general model, and is easily replaced by the next model release.

Moat
Anything that makes a business hard for competitors to copy.

Evaluation set (evals)
A library of real test cases with known-good answers, used to measure whether the product's output is correct.

Retrieval
Searching a knowledge base for the most relevant documents and giving them to the model as context.

The Model Explained

The five parts of a harness

The hero diagram places the model in the middle and surrounds it with five components. Each one supplies something a general model lacks.

TABLE 2.1 • THE FIVE COMPONENTS, WITH A LEGAL EXAMPLE

Part	Plain English	Example for a law practice
Tools	Things the AI can *do*: search a database, fill in a form, send an email, connect to the software the niche already uses.	Pull case files from the firm's document system; search court records.
Knowledge	Specialized information the general model doesn't have or doesn't reliably know.	State-specific statutes, the firm's past contracts, a clause library.
Rules	The niche's standards, regulations, formats, and "never do this" guardrails.	Always cite the statute. Never advise outside the jurisdiction. Follow court formatting rules.
Memory	It remembers each user, their clients, their preferences, and past work.	Remembers this attorney's writing style and every past matter for the client.
Workflows	The step-by-step processes the niche actually follows, built in so users don't have to prompt from scratch.	Intake → research → draft → review checklist → final document.

The caption under the diagram, "the harness, built for one niche," is the key phrase. A general model knows a little about everything. A harness knows *everything* about one thing, and it plugs into how that niche already works.

The eight facts, unpacked

Why now. Models are now smart enough for almost any reasoning task. What they lack is *context* (niche knowledge and the user's own data) and *workflow* (knowing the exact steps of a job). That gap is the opportunity.

Real-world examples. Some of the fastest-growing AI companies are harnesses: an AI platform for law firms, an AI scribe that listens to doctor–patient conversations and writes the clinical note, and a service that drafts demand letters for personal-injury attorneys and charges per letter. All three use models that anyone can access. Their value is the harness.

Proof it works. One builder created a harness just for people who run tabletop fantasy role-playing games. It shows that the niche can be a hobby, and can be small, as long as users are passionate and the workflow is specific.

Who builds it. Deep experts in a niche the big players will ignore. Large technology companies won't build a tool for funeral directors, HVAC estimators, or game masters. You can, if you know that world from the inside.

How it earns. A premium subscription, for example $99 to $500 per user per month, which is far more than a general assistant costs because it saves hours. Or per-job pricing: a fixed fee per report, letter, or claim.

The trap. A *thin wrapper* is a product that's just a pleasant screen over a general model with a clever prompt. When the next, better model comes out, users realize they can get the same result from a general assistant, and the product dies. A real harness gets *better* when models improve, because its tools, knowledge, memory, and workflows are where the value lives.

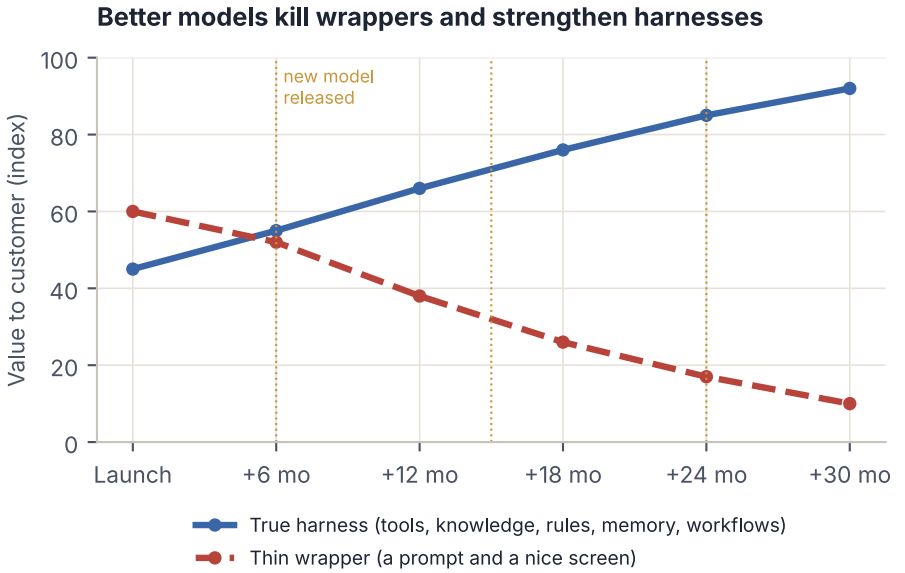


Figure 2.2 How model improvements affect value over time (illustrative). Each new model release makes a thin wrapper easier to replace, while a true harness turns the smarter engine into a better product.

How a harness differs from Models 1 and 4

In Model 1 you *do the work* for clients and sell results. In Model 2 you *sell the tool* so professionals do the work faster themselves. In Model 4 (Chapter 4) you

sell your *method* as a package that installs into the customer's own AI assistant. The three combine well. Many founders start with Model 1, refine their internal system on real clients, then sell that system as a harness.

Why This Is a Real Opportunity

- **Software margins.** Once built, each new subscriber costs only AI usage and hosting, so gross margins of 70 to 85 percent are common.
- **Niche means little competition.** A niche with 50,000 professionals paying $150 a month is a $90 million-a-year market: too small for the giants, plenty for you.
- **Experts have the edge.** AI coding assistants and no-code app builders let a domain expert build working software. The hard part, knowing exactly what the niche needs, is what you already have.
- **Professionals pay for time saved.** If a tool saves a $100-an-hour professional five hours a week, $200 a month is an easy yes.

70–85%
Typical gross margin for niche software

$90M
Yearly market: 50,000 users × $150/month

5 hrs
Weekly time saved that justifies $200/month

CASE STUDY

A harness for home inspectors

The most painful weekly job: writing the inspection report. It takes two to four hours per house, usually at night after the inspection.

Part	What you'd build
Tools	Upload photos and voice notes from a phone; export into the inspection-report software inspectors already use; email the report to the agent and buyer.
Knowledge	A library of 1,000+ common defects with standard descriptions, repair cost ranges, and reference photos; building-code references.
Rules	The state's standards of practice; liability-safe phrasing ("recommend evaluation by a licensed electrician"); never state anything that wasn't observed.
Memory	Each inspector's writing style, favorite phrases, company template, past reports, and regular real-estate agents.
Workflows	Walk the house → photograph and dictate → AI identifies each defect and drafts its description → inspector approves or edits → report generated in their template → sent.

The result: report time falls from about three hours to about thirty minutes. At $149 a month, it pays for itself on the first house. A general assistant can't do this, because it doesn't know the state rules, the defect library, the inspector's template, or how to connect to their software.

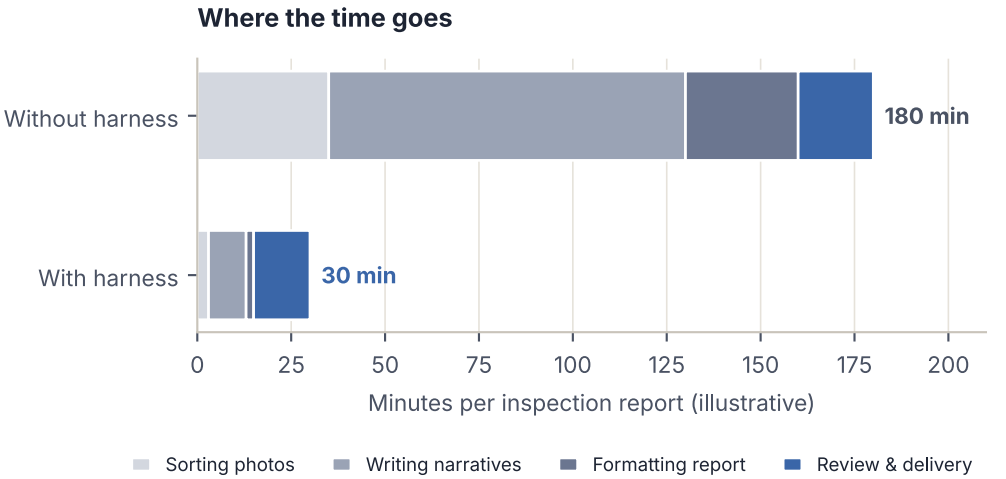


Figure 2.3 Where an inspector's report time goes, with and without a harness (illustrative). The biggest saving is in writing narratives; review time barely changes, because the professional still signs off.

Step-by-Step Plan

WEEK 1	**Phase 0 — Choose your niche.** Apply the six-point checklist.
WEEK 1	**Phase 1 — The ten weekly jobs.** Score the jobs, pick the most painful, and validate it in 10–15 interviews.
WEEKS 2–3	**Phase 2 — Build the harness by hand.** Use an AI workspace with knowledge, rules, and workflow, with no code yet.
WEEKS 3–8	**Phase 3 — Turn it into software.** Use a no-code route or a developer route.
ONGOING	**Phase 4 — Build your moat.** Run the "better model" test.
WEEKS 8–12	**Phase 5 — Price and launch.** Founding-member pricing; niche channels.
MONTHS 3–12	**Phase 6 — Grow.** Add job #2 and #3, and track churn and usage.

Phase 0: Choose your niche

1. **You know it deeply:** you've worked in it, run it, or obsessed over it.
2. **Its people are reachable:** associations, online groups, forums, trade shows, newsletters.
3. **They already pay for tools,** even small ones.
4. **The work is heavy on text, documents, images, speech, or data.**
5. **Big players ignore it.** If a giant is already building it, move on.
6. **There's niche-specific knowledge or rules** that general AI gets wrong. That's your moat.

Niche ideas to spark thinking: home inspectors, insurance adjusters, property managers, homeowners-association managers, funeral homes, veterinary clinics, dental office managers, appraisers, nonprofit grant writers, construction estimators, HVAC and plumbing contractors (quotes and permits), customs brokers, court reporters, church administrators, youth sports league organizers, wedding planners, tabletop gamers, online craft sellers, immigration paralegals, and freight brokers.

Phase 1: The "ten weekly jobs" exercise

List ten jobs your niche does every week. Score each from 1 to 5 on five dimensions, then add up the scores.

TABLE 2.2 • SCORING THE TEN WEEKLY JOBS (HOME INSPECTOR EXAMPLE, PARTIAL)

Weekly job	Time	Hated	Cost of mistakes	Frequency	AI fit	Total
Writing the report	5	5	4	5	5	**24**
Scheduling inspections	3	3	2	5	4	17
Answering agent questions	2	3	3	4	4	16
Invoicing and collections	2	4	2	4	3	15
Continuing-education tracking	1	3	3	1	3	11

Pick the highest score. That single job is your first product. Don't build for all ten. Then **validate it**: talk to ten to fifteen people in the niche. Ask how they do it today, how long it takes, what they've tried, and what they'd pay to cut it by 80 percent. Don't pitch. Listen, and record and summarize each call with an AI meeting recorder.

Phase 2: Build the harness by hand first

Before writing any software, prove the AI can do the job:

1. Gather ten to twenty real examples of the finished work, with permission and anonymized.
2. In an AI assistant's project or workspace feature, load the **knowledge** (reference documents, examples, standards), the **rules** (a written list of requirements and prohibitions), and the **workflow** (step-by-step instructions).
3. Run real jobs through it. Compare the output to expert-quality work and adjust the instructions until at least 80 percent of the output is usable with light edits.
4. Do the work for a few people for free. They send inputs, you run them through your setup and send results back. You're temporarily running Model 1, and it teaches you what really matters.

> **PLAIN ENGLISH**
>
> Your first harness is a folder of documents and instructions inside an AI assistant. If you can't make it work there, software won't fix it. If you can, software just makes it faster and easier to use.

Phase 3: Turn it into software

Route A, no-code (non-technical founders): describe the app to an **AI app builder** that generates working web apps from plain-language descriptions; use an **AI coding assistant** to make changes; use a **hosted database-and-login service** for user accounts and data (often built into app builders); and use a **workflow automation tool** for background steps.

Route B, developer stack (more control, scales further):

TABLE 2.3 • THE DEVELOPER STACK, BY HARNESS COMPONENT

Harness part	Tool category
Model (the engine)	A **frontier AI model API**. Test two or three providers on *your* task and choose on quality, not hype.
Agent framework (the chassis)	An **agent SDK or framework** that handles tool calls, multi-step plans, and retries.
Tools	**MCP servers** (Model Context Protocol, an open standard; see Chapter 3) to connect the AI to the niche's existing software, plus integration platforms for quick connections.
Knowledge	A **vector database** (or vector search inside your main database) for retrieval, plus a **document-parsing service** that reads PDFs, forms, and tables reliably.
Memory	Per-user tables in your database, plus a **long-term memory layer** for preferences and history.
Rules and quality	Written system instructions, checklists, a second "reviewer" AI pass, and an **evaluation and observability tool** to test quality against real cases.
Speech and audio	**Speech-to-text and text-to-speech APIs**, if your niche talks more than it types.
App, hosting, login, payments	A modern web framework on a **front-end hosting platform**; an **authentication service**; a **payment processor** with subscriptions and usage billing.
Monitoring	**AI call logging**, **product analytics**, and **error tracking**.

Build only the one workflow. It should take raw input, run your tools, knowledge, rules, memory, and workflow, and produce output that's 80 percent done. Give users an easy edit screen, and export into the format or software they already use. The export step matters more than most founders expect.

Phase 4: Build your moat

Test your product with one question: *if a model twice as smart came out tomorrow, would customers still need us?* You want the answer to be yes, for these reasons:

- ☐ **Proprietary knowledge:** a defect library, clause bank, rate table, or rules database you curated.
- ☐ **Integrations** with the niche's specific software.
- ☐ **Customer memory:** months of each user's data, style, and history. Switching means losing it.
- ☐ **Encoded workflow:** the niche's exact steps, checklists, and formats.
- ☐ **Rules and compliance:** regulations, liability-safe language, and audit trails.
- ☐ ***Evaluation sets*:** a growing library of real test cases proving your output is right.

A smarter model makes every item on this list *more* valuable. That's the difference between a harness and a wrapper.

Phase 5: Price and launch

Pricing. For a subscription, price against the time saved; common niche ranges are $49 to $500 per user per month, with a founding-member discount for the first 20 to 50 users. Use per-job pricing when jobs are large and irregular, or combine a base subscription with usage. Avoid $9 to $20 a month. That's the price of a general assistant, and it signals "wrapper."

Launch channels. Start with your interview list. Then go to niche communities and forums, industry associations and conferences (a booth or speaking slot can fill a pipeline), and sixty-second demo videos showing a three-hour job done in twenty minutes. Integration marketplaces of software the niche already uses are another channel, along with targeted outbound email built from B2B contact databases. Your landing page should have one problem, one demo, one price, and a free trial.

Phase 6: Grow

Add job #2 from your list, then #3. Each one deepens the harness and supports a higher price. Track weekly active usage, time saved per job, and monthly churn (aim for under 3 percent). Save every user edit, because users' corrections show you exactly how to improve your knowledge and rules. Consider a done-for-you tier (Model 1) for customers who want you to run it. A few hundred loyal customers make a sellable, fundable business.

Risks and How to Handle Them

Risk	Mitigation
Becoming a thin wrapper	Run the "better model" test every quarter. Invest in knowledge, integrations, and memory.
AI errors in high-stakes fields	Human approval before anything is final, citations and sources shown, clear terms of service. Pick lower-stakes niches for a first product.
Data privacy and compliance	Use model APIs that don't train on customer data. Healthcare requires health-privacy compliance and a business associate agreement with every vendor that touches patient data. Pursue a security certification when larger customers ask.
AI usage costs eating margin	Price with usage in mind, cache repeated content, and route simple steps to smaller, cheaper models.
Niche too small or unreachable	Validate with 10–15 interviews and confirm communities exist *before* building.
An incumbent copies you	Move faster, go deeper into the niche, and own the community relationships.

Starter Budget

TABLE 2.4 • TYPICAL MONTHLY COSTS TO LAUNCH

Item	Approximate cost
AI assistant / coding assistant subscription	$20–$200
AI app builder	$20–$50
Hosted database and login	$0–$25
Front-end hosting	$0–$20
AI model API usage	$20–$200 early, then scales with revenue
Domain and website	~$20
Payment processing	~3% per transaction, no monthly fee
Total	**About $100–$500 a month** to reach a launched product

Your Seven-Day Checklist

- ☐ **DAY 1** Pick two or three niches you know deeply.
- ☐ **DAY 2** For the best one, list ten weekly jobs and score them.
- ☐ **DAY 3** Choose the most painful job.
- ☐ **DAY 4** Message fifteen people in the niche and book five or more calls.
- ☐ **DAY 5** Collect ten real examples of that job's finished output.
- ☐ **DAY 6** Build an AI workspace with the knowledge, rules, and workflow, and test it on the examples.
- ☐ **DAY 7** Write down your moat: which tools, knowledge, rules, and memory make this impossible to copy with a general assistant?

CHAPTER SUMMARY

- A harness wraps a general model in tools, knowledge, rules, memory, and workflows for one niche.
- The test of a real harness: a better model makes it stronger, not obsolete.
- Start from the niche's most painful weekly job, and prove it works in an AI workspace before writing software.
- Price against time saved; avoid general-assistant price points that signal "wrapper."
- Deep niche expertise, not programming skill, is the scarce ingredient.

Review Questions

1. Which is *not* one of the five harness components?

   A. Memory
   B. Rules
   C. Advertising
   D. Workflows

2. What happens to a thin wrapper when a much better general model is released?

   A. It improves automatically
   B. Users realize they can get the same result elsewhere, and it loses value
   C. Nothing changes
   D. Its price can go up

3. In the "ten weekly jobs" exercise, which job should you build for first?

   A. The easiest one
   B. The one with the highest combined score for time, hatred, cost of mistakes, frequency, and AI fit
   C. All ten at once
   D. The one competitors already serve

4. Explain the difference between a vertical and a horizontal product, with one example of each.

5. Why does the chapter recommend building the harness inside an AI workspace before building software?

6. List four moat components that become more valuable as models improve.
7. Why does the chapter warn against a $9–$20 monthly price?
8. How do Models 1, 2, and 4 differ in what they sell?

APPLY IT • WORKSHEET 2

Map your harness

My niche, and the most painful weekly job I'll start with:

Tools (what it must connect to and do):

Knowledge (what it must know that general AI doesn't):

Rules (standards, regulations, never-do-this):

Memory (what it should remember about each user):

Workflow (the steps, start to finish):

My answer to the "better model" test:

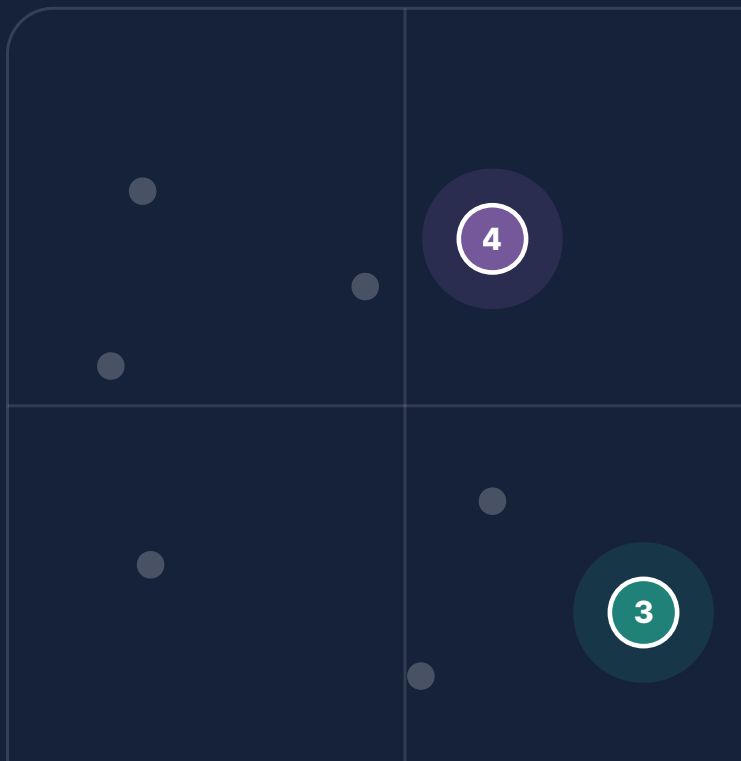
PART TWO

Serving Agents & Their Owners

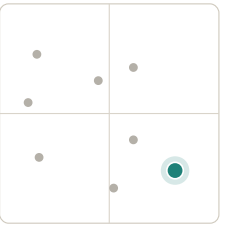
Two models built for a world where AI agents do more and more of the work. The first builds tools that agents use directly, with no screen at all. The second packages your method so it installs into the agent a customer already uses every day.

03 Headless AI Software

04 Agent-Native Systems



CHAPTER 3 • MODEL 3 OF 8



3

Headless AI Software

Your next million users won't have eyes.

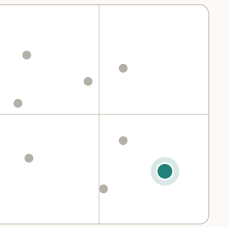
LEARNING OBJECTIVES

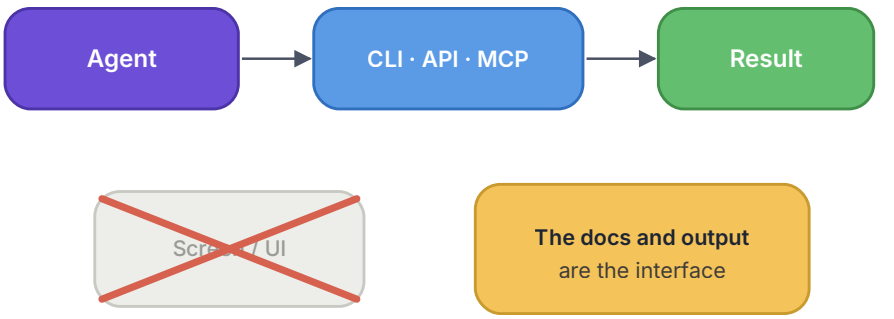
- Explain "headless" software and why agents are its customers
- Describe the three "doors" agents use: CLI, API, and MCP
- Turn an AI weakness into a product idea, and test it against the trap
- Design an agent-friendly tool, its documentation, and its pricing
- Distribute a tool where developers and agents will find it

TYPE 3 OF 8

Headless AI software

Your next million users won't have eyes.





No screen to design. The agent reads your documentation and parses your output.

Why now
Agents are becoming software's heaviest users. They need tools, not interfaces.

Real-world example
Web-data services that turn pages into clean text, search engines built for AI, cloud browsers agents can drive, and developer email-sending services.

Proof it works
A one-command tool that checks a software build against the project's written design rules.

Who builds it
Builders with a data source or one hard, narrow skill.

How it earns
Usage-based pricing, credits, or an open-source core.

The trap
Building what the big AI labs will bundle for free next quarter.

This week
Find a task AI does badly. Wrap the fix as a one-command tool.

In one line
No screen. The docs are the interface.

Figure 3.1 Headless AI software at a glance. An agent reaches your tool through a command line, an API, or an MCP connection and gets a result back. There's no screen; your documentation and output format are the user experience.

Until now, almost all software was built for people: screens, buttons, menus, dashboards. That's changing fast. AI agents, such as coding assistants and automated research and operations bots, now use software constantly, and they don't need a screen. They don't click. They read instructions and send commands.

Headless means software with no visual interface: all brain, no face. In this model you build tools that *AI agents use* to get work done, rather than apps that humans look at. Your customers are developers and businesses, but the day-to-day user is their agent.

KEY TERMS

Headless
Software without a graphical user interface, used by other programs through commands or requests.

CLI (command-line interface)
A program you run by typing a command in a terminal.

API (application programming interface)
A web address that accepts requests in a defined format and returns data.

MCP (Model Context Protocol)
An open standard that lets any compatible AI agent discover and use a tool automatically.

JSON
A simple, structured text format for data that programs (and agents) read easily.

Usage-based pricing
Customers pay per call, page, search, or other unit they consume.

The Model Explained

The three doors

The top row of the diagram reads **Agent → CLI · API · MCP → Result**. An agent needs to do something it can't do well on its own, such as search the web, send an email, or check a file, so it calls your tool through one of three "doors" and gets a result back.

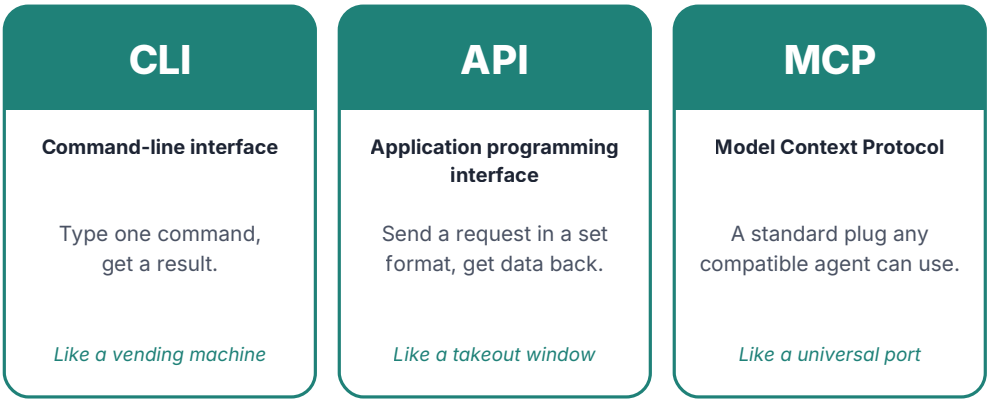


Figure 3.2 The three doors agents use. Most successful headless products offer all three: a command-line tool for local use, an API for software integration, and an *MCP server* so any compatible agent can plug in.

The bottom row crosses out **Screen / UI** and replaces it with **"The docs and output are the UI."** An agent learns how to use your tool by reading your documentation, and it understands the result by reading your output. So your docs and your output format *are* the product experience. They must be clear, structured, and predictable, because a confused agent simply picks a different tool.

The eight facts, unpacked

Why now. Agents are becoming software's heaviest users. One developer working with an AI coding agent may trigger hundreds of tool calls a day, and an automated pipeline can trigger thousands.

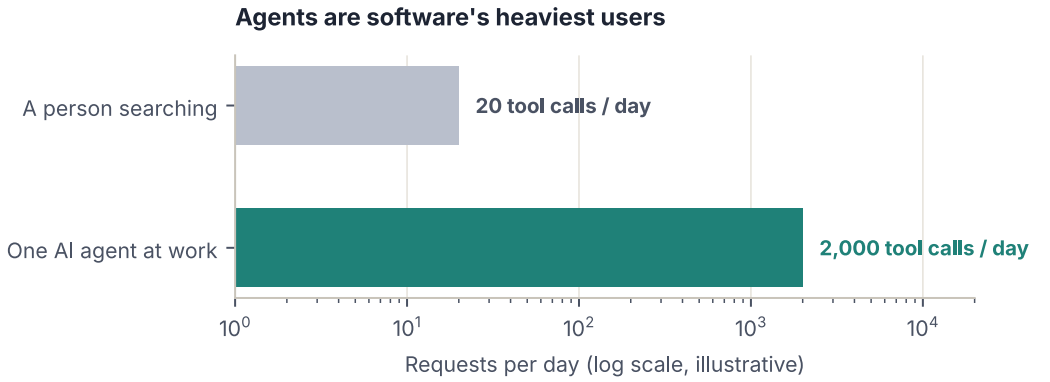


Figure 3.3 Daily requests from a person versus an agent (illustrative, log scale). Volume is why pay-per-use pricing on agent traffic can grow quickly.

Real-world examples. Some of the fastest-growing developer companies are headless: a service that turns any website into clean text an AI can read, a search engine designed for AI rather than people, a platform that runs cloud web browsers agents can control, and a simple email-sending service developers call with one command. None of them is mainly known for a dashboard.

Proof it works. One builder created a command-line tool that checks whether a software build matches the rules in the project's design-guidelines file (its colors, fonts, and spacing). AI coding agents often drift from a design system. The agent runs the tool, receives a list of violations, and fixes them.

Who builds it. Builders with **a data source** (unique data others can't easily get) or **one hard, narrow skill** (one thing done extremely well).

How it earns. *Usage-based pricing* (pay per call, page, or search), **credits** (buy a pack and spend it down), or an **open-source core** (a free version that spreads widely, with a paid hosted or pro version; see Chapter 8).

The trap. Building something the big AI labs will give away next quarter. When major assistants added built-in web search, many simple search wrappers lost their reason to exist. Don't build generic features. Build things that need special data or deep niche expertise.

If a lab could add it as a checkbox, don't build it. If it needs your data or your expertise, it's yours to own.

Why This Is a Real Opportunity

- **Volume.** A person might search twenty times a day; an agent might search two thousand.
- **No design or sales burden.** No beautiful app, onboarding flow, or sales team. Developers find you, try one command, and wire you in.
- **Agents choose tools by reading docs,** so a tiny tool with excellent documentation can compete with big companies.

- **Distribution is new and open.** Agent tool directories, plugin marketplaces, and skill catalogs are young. Early, good tools get discovered.
- **Small, sharp products win.** One command that does one thing perfectly can be a viable business.

How the models compare so far

	Model 1: Services	Model 2: Harness	Model 3: Headless
Customer	Businesses	Niche professionals	Agents and the developers who run them
You sell	Finished work	An app with an interface	A tool with no interface
Interface	Email, calls, deliverables	Screens and workflows	Docs, commands, output
Pricing	Retainers	Subscriptions	Pay per use
Technical skill	Low	Medium	Higher

Turning AI Weaknesses into Products

Agents are bad at certain things, and each weakness is a product opportunity.

TABLE 3.1 • WHERE AGENTS STRUGGLE, AND WHAT TO BUILD

What agents do badly	Headless tool idea
Getting fresh or hidden data	An API for niche data: county property records, building permits, court filings, product recalls, government contracts, local business licenses, state regulations.
Following a standard precisely	Checkers: accessibility checker, brand-guideline checker, legal-citation validator, structured-data validator, regulated-advertising language checker.
Exact calculations with complex rules	Sales-tax rate by address, payroll-tax calculator, shipping-rate calculator, insurance premium rules, medical billing-code validation.
Reading messy files	Converters: scanned PDFs to clean tables, engineering drawings to data, supply-chain data formats to JSON, bank statements to transactions.
Acting in the real world	Send faxes (still common in healthcare and law), mail physical letters, place phone calls, file forms on government portals.
Verifying output	Fact-checkers, link checkers, "does this code match the spec" checkers, visual-difference tools.
Remembering and organizing	Memory stores and knowledge bases scoped to a project or company.

The best ideas combine **your niche expertise** with **an AI weakness**. If you know insurance, construction, healthcare billing, real estate, or accounting rules deeply, you can build a checker or data tool for that field that the labs will never bother to make.

CASE STUDY

"PermitCheck" for construction agents

The weakness. Ask a general assistant whether a backyard deck needs a permit in a particular city and what the setback rules are, and it will guess, often wrongly or out of date.

The tool. A command-line tool, API, and MCP server. One command such as `permitcheck lookup --city "Austin, TX" --project deck --height`

`30in` returns structured JSON: whether a permit is required, the specific rules, source links, and a last-updated date.

The moat. You collected, and keep updating, the rules for the 500 largest U.S. cities. That's a data source, and the labs won't build it.

The customers. Contractor software companies, construction-estimating agents, and real-estate AI tools, paying perhaps $0.05 to $0.50 per lookup.

Step-by-Step Plan

WEEK 1	**Phase 0 — Find your idea.** Keep a week-long failure log of agent mistakes.
WEEKS 1–3	**Phase 1 — Build the one-command version.** Start with a CLI.
WEEKS 3–5	**Phase 2 — Add the API and MCP doors.**
WEEK 5	**Phase 3 — Write docs for robots and humans.**
WEEKS 5–6	**Phase 4 — Pricing and billing.**
WEEKS 6–10	**Phase 5 — Distribution.**
MONTHS 3–12	**Phase 6 — Grow and defend.**

Phase 0: Find your idea

1. **Use AI agents heavily for a week** on real work in a field you know: a coding agent, an assistant with tools, an agent mode.
2. **Keep a failure log** with columns for the task, what went wrong, how you fixed it manually, and how often it happens.
3. **Look for patterns:** wrong data, outdated information, sloppy adherence to rules, unreadable file types, actions it can't take.
4. **Apply the trap test:** "Will a major AI lab build this into its model or app within six months?" If yes (generic search, basic summarization, simple formatting), skip it. If it needs unique data or niche rules, keep it.
5. **Check demand:** search public code repositories, developer forums, and agent-tool directories for people asking for this, and for tools you'd compete with.

Phase 1: Build the one-command version

Start with a *CLI*. It's the simplest door, and coding agents use command-line tools natively. Write it with help from an AI coding assistant, in a popular language with a public package registry so users can install it with one command. Use a CLI framework for argument parsing. If your moat is data, use web-data extraction services or browser-automation platforms to collect public information (respecting each site's terms), a document-parsing service for PDFs, and a hosted database to store it.

> DESIGN RULES FOR AN AGENT-FRIENDLY TOOL
>
> - **One obvious command** for the main job: `toolname check`, `toolname lookup`.
> - **Complete help text** with examples. Agents read `--help`.
> - **A `--json` output option.** Agents parse structured data far better than pretty text.
> - **Actionable error messages:** "Missing --city. Example: --city 'Austin, TX'." An agent can then fix its own mistake.
> - **Exit codes** (0 = pass, 1 = fail) so agents and pipelines can react.
> - **No interactive prompts.** Agents can't answer "Are you sure? (y/n)."
> - **Deterministic results:** the same input gives the same output.

A good response looks like this:

```
$ permitcheck lookup --city "Austin, TX" --project deck --height 30in --json
{
  "permit_required": true,
  "rules": ["Decks over 30 in. above grade require a building permit",
            "Minimum 5 ft side setback"],
  "sources": ["https://example.gov/codes/decks"],
  "last_updated": "2026-10-01"
}
```

Phase 2: Add the API and MCP doors

Host a simple web *API* on a serverless or app-hosting platform using a lightweight web framework. Build an MCP server with the official open-source MCP SDKs, offering both a local version (runs on the user's machine) and a remote,

hosted version with login. Add an **API-key management service**, **rate limiting**, and **monitoring** (error tracking, logs, usage analytics).

> **PLAIN ENGLISH**
>
> An agent decides whether to use your MCP tool by reading its name and description. A tool named `check_design_compliance`, described as "Checks HTML/CSS against the rules in DESIGN.md and returns each violation with file, line, and fix," gets used. A tool named `run`, described as "runs the tool," doesn't.

Phase 3: Write docs for robots (and humans)

Your docs are your storefront. Use a **documentation-site generator**. Publish an `llms.txt` file, a plain-text summary of your docs written for AI agents, along with Markdown versions of every page. Publish an **OpenAPI specification** so tools can generate reference docs and client libraries automatically. Write a short **agent skill file** that teaches assistants when and how to use your tool, and an instructions snippet users can paste into their projects.

Test your docs on agents. Give a coding agent *only* your docs and ask it to complete a task with your tool. If it struggles, rewrite the docs, not the agent prompt. This is your usability testing.

Phase 4: Pricing and billing

TABLE 3.2 • PRICING MODELS FOR HEADLESS TOOLS

Model	When to use	Example
Usage-based	Every call costs you something (data, compute)	$0.01 per check; $1 per 1,000 lookups
Credits	Operations vary in cost	$20 buys 2,000 credits; simple lookup = 1, deep scan = 10
Open-source core	Adoption matters most	Free CLI; paid hosted API, team features, premium data
Free tier	Always, to remove friction	100–1,000 free calls a month

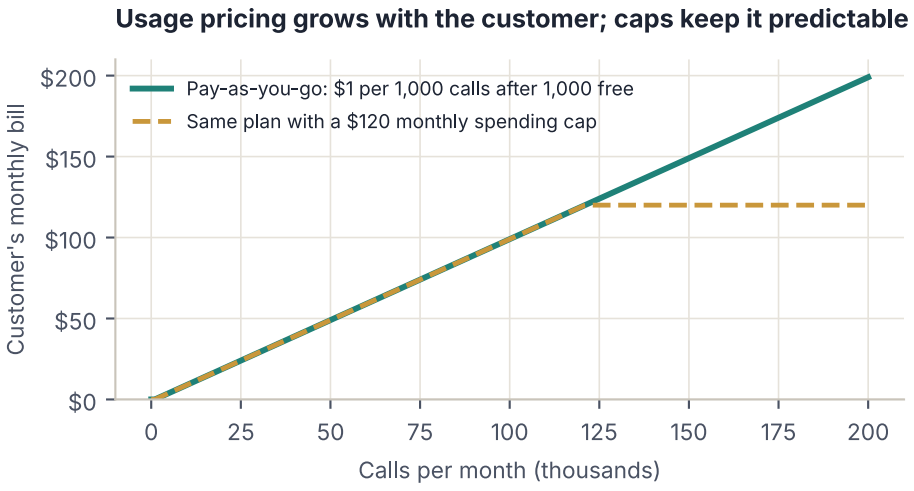


Figure 3.4 A customer's monthly bill under pay-as-you-go pricing, with and without a spending cap (illustrative). Keep per-call prices low and predictable, and offer caps, because developers hate surprise bills.

For billing, use a **payment processor with metered (usage-based) billing** or a dedicated **usage-billing platform**; some billing platforms are designed specifically for developer tools and open-source projects.

Phase 5: Distribution

1. **Package registries** for your language, plus the main operating-system package managers, so installation takes one command.
2. **MCP and agent-tool directories,** including the official MCP registry and community directories.
3. **Plugin and skill marketplaces** for AI assistants and coding tools, and integration lists of popular agent frameworks.
4. **A public code repository** with a strong README, a short terminal demo recording, and a "quick start in thirty seconds" section. Repository stars drive discovery.
5. **Launch posts** on developer news sites, product-launch sites, and relevant forums and social networks.
6. **Demos:** a thirty- to sixty-second recording showing an agent getting it wrong, then calling your tool and getting it right.
7. **Partnerships** with agent platforms and with vertical harness builders (Model 2) in your niche.

Phase 6: Grow and defend

Watch usage logs to see which commands agents use most and where they fail, and improve those first. Deepen your moat with more data coverage, fresher updates, and more niche rules. Add adjacent commands around the same core skill without sprawling. Offer an enterprise tier with higher limits, private deployment, an uptime guarantee, and a security certification. Every quarter, ask: *Did a lab just ship something that replaces me?* If one is getting close, move deeper into what they can't do.

Risks and How to Handle Them

Risk	Mitigation
Labs bundle it free (the trap)	Build on unique data or deep niche rules, not generic capability.
Too technical for you	Use an AI coding assistant, start with a CLI, or partner with a developer while you supply the niche knowledge and data.
Data licensing and legality	Check terms of service and robots.txt; prefer public records and open data; license data properly.
Security (agents running your code)	Least-privilege access, input validation, secret API keys, a published security policy.
Runaway infrastructure costs	Rate limits, caching, spending caps, and pricing above your per-call cost.
Being invisible	Invest in docs, directory listings, and demo content. Distribution matters as much as code.

Starter Budget

Item	Approximate monthly cost
AI coding assistant subscription	$20–$200
Hosting (serverless or app platform)	$0–$25
Hosted database	$0–$25
Documentation site	$0 (free tiers or open-source generators)
Data collection services, if needed	$0–$100
Billing	A percentage of revenue
Domain	~$1–$2
Total	**About $25–$350 a month**, the cheapest model to start because there's no interface to design

Your Seven-Day Checklist

- ☐ **DAY 1** Use an AI agent for real work in your field.
- ☐ **DAY 2** Log every failure: what went wrong and how you fixed it.
- ☐ **DAY 3** Pick the most frequent failure that needs unique data or niche rules.
- ☐ **DAY 4** Run the trap test: would a lab ship this free soon?
- ☐ **DAY 5** Write the one command you wish existed, with its help text and JSON output.
- ☐ **DAY 6** Build a rough version with an AI coding assistant.
- ☐ **DAY 7** Give an agent only your README and see whether it can use the tool correctly. Then fix the docs.

CHAPTER SUMMARY

- Agents are becoming software's heaviest users, and they need tools, not screens.
- Offer three doors (CLI, API, MCP). Your documentation and output format are the interface.
- Find ideas in AI's weaknesses: fresh data, precise rules, exact calculations, messy files, real-world actions.
- Avoid anything a lab could bundle as a free feature; build on data or niche expertise.
- Price per use with a free tier and spending caps, and distribute through registries, directories, and demos.

Review Questions

1. In headless software, what replaces the screen as the user experience?
   A. A mobile app
   B. The documentation and the output format
   C. A sales team
   D. Video tutorials
2. Which product idea best passes the trap test?
   A. A general web-search wrapper
   B. A basic text summarizer
   C. A continuously updated database of building-permit rules for 500 cities
   D. A grammar checker
3. Why is a `--json` option important?
   A. It looks professional
   B. Agents parse structured data more reliably than formatted text
   C. It is required by law
   D. It makes the tool faster
4. Describe the CLI, API, and MCP "doors" in your own words, with an analogy for each.
5. Name four agent weaknesses that can become headless products.
6. How should you test whether your documentation works?

7. Why should usage-based pricing include spending caps?
8. What two kinds of builders does this model favor?

APPLY IT • WORKSHEET 3

Spec your headless tool

The agent failure I observed (task, what went wrong, how often):

The data source or narrow skill that makes my fix hard to copy:

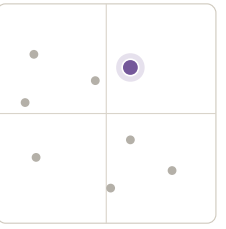
My one command, exactly as an agent would type it:

The JSON fields it returns:

Trap test: why a major lab won't bundle this soon:

Pricing (free tier, unit price, cap):

CHAPTER 4 • MODEL 4 OF 8



4

Agent-Native Systems

Teach their agent your method. Give them a window to watch it work.

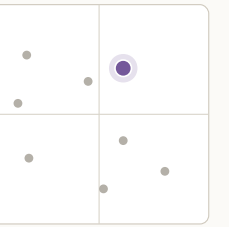
LEARNING OBJECTIVES

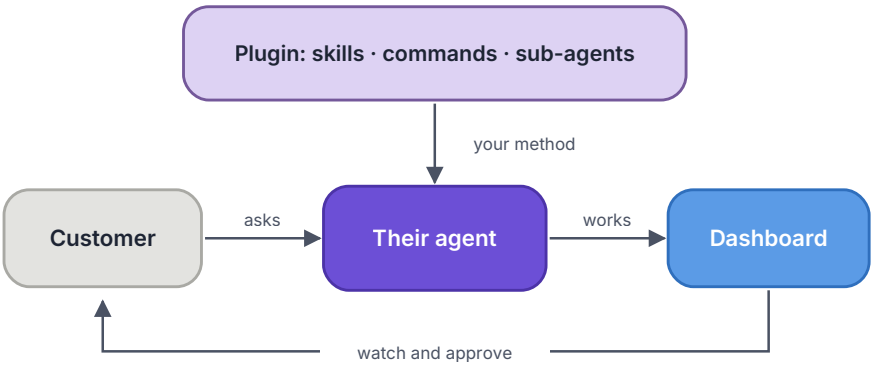
- Explain how a plugin carries a method into a customer's AI assistant
- Name the parts of a plugin: skills, commands, sub-agents, and connectors
- Write a method as three skills: diagnose, plan, do
- Design a dashboard for watching, not clicking
- Monetize while protecting yourself from platform risk

TYPE 4 OF 8

Agent-native systems

Teach their agent your method. Give them a window to watch it work.





The customer supervises; the agent does the work your way.

Why now
People already spend their day inside AI assistants and AI coding tools. Plugins are the new app store.

Real-world example
AI-assistant plugins that bundle skills, commands, and sub-agents into one install.

Proof it works
A product studio sells its product-building method as a plugin, plus a dashboard to watch it run.

Who builds it
Experts with a repeatable method: coaches, consultants, educators.

How it earns
Paid plugin access, membership, or a dashboard subscription.

The trap
Platform risk. Own the email list and the community.

This week
Write your method as three skills: diagnose, plan, do.

In one line
The dashboard is for watching, not clicking.

Figure 4.1 Agent-native systems at a glance. Your plugin installs your method into the customer's own AI agent. The customer asks, the agent works your way and reports to a dashboard, and the customer watches and approves.

Millions of people now spend their workday inside an AI assistant. Each of them effectively has an agent: an AI that can read their files, use their apps, and do tasks for them. This model asks a simple question: if you're an expert, why sell a course, a coaching call, or a new app, when you could install your know-how straight into the agent your customer already uses?

An *agent-native system* packages your method so it runs inside the customer's AI. The agent follows your method, and the customer watches progress and approves important steps on a **dashboard**. It's like hiring a consultant whose expertise works inside your own assistant, around the clock.

KEY TERMS

Plugin
An installable package that adds capabilities to an AI assistant: skills, commands, sub-agents, and connectors.

Skill
A written set of instructions, often with templates and examples, that teaches an AI how to do a task your way.

Command
A shortcut the user types to start a workflow, such as `/pipeline-review`.

Sub-agent
A specialized assistant with a narrow role that the main AI can delegate to.

Connector
A link (often via MCP) that lets the agent use another app, such as email, a calendar, or a CRM.

Platform risk
The danger of depending on a platform you don't control, whose rules, fees, or features can change.

The Model Explained

The diagram, step by step

1. **Plugin: skills, commands, sub-agents (top box).** This is your product: a downloadable package containing your method.
2. **"Your method" arrow.** The plugin installs into the customer's agent and teaches it how *you* do the work.

3. **Customer → asks → their agent.** The customer asks in plain language: "Help me fix my sales pipeline."
4. **Agent → works → dashboard.** The agent does the work using your method and reports progress, results, and decisions to a dashboard.
5. **Dashboard → watch and approve → customer.** The customer supervises and approves key decisions without doing the work themselves.

What's inside a plugin

TABLE 4.1 • PLUGIN COMPONENTS, WITH A SALES COACH'S EXAMPLE

Part	Plain English	Sales coach's plugin
Skills	Instructions that teach the AI how to do something your way: frameworks, checklists, templates, examples.	"How to qualify a lead with my five-question method"
Commands	Shortcuts the user types to start a workflow.	`/pipeline-review`, `/write-followup`
Sub-agents	Mini-assistants with a narrow role the main AI delegates to.	A "Deal Reviewer" that critiques each opportunity like the coach would
Connectors (optional)	Links to apps the method needs.	The customer's CRM, email, and calendar

The eight facts, unpacked

Why now. People already work inside AI assistants and AI coding tools all day. Instead of persuading someone to download a new app, you add capability to the tool they already have open. **Plugins are the new app store.**

Real-world example. Leading AI assistants now support plugins that bundle skills, commands, and sub-agents into one install, and other assistants offer similar ecosystems of custom assistants and app integrations.

Proof it works. The framework's originator sells their own product-building method as a plugin, paired with a dashboard for watching it run. It's the same internal "operating system" that powers their service work in Model 1, which shows how the models feed each other.

Who builds it. Experts with a repeatable method: coaches, consultants, and educators. If you've taught a framework, run a workshop, or have a "here's how I do it" process, this is for you. You don't need to code, because skills are mostly written instructions.

How it earns. Paid plugin access (one-time or yearly), a membership (plugin plus community plus updates), or a dashboard subscription (free plugin, paid dashboard).

The trap. *Platform risk*. An AI company could change its rules, its marketplace, or its plugin format, or build your feature itself. If your customers exist only inside someone else's platform, you could lose everything overnight. So own your email list and your community. Those are your real assets.

This week. Write your method as three skills: **diagnose** (figure out the situation), **plan** (decide what to do), and **do** (do it). That's the shape of almost every consulting method.

A course teaches people what to do. A plugin does it, using what the course would have taught.

How it compares

	1: Services	2: Harness	3: Headless	4: Agent-native
You sell	Finished work	An app	A tool for agents	Your method, inside their agent
Who works	You + your AI	Your app	Their agent + your tool	Their agent, your way
Tech skill	Low	Medium	High	Low to medium
Best for	Service operators	Niche experts	Developers, data owners	Coaches, consultants, educators

Why This Is a Big Opportunity

- **Courses are fading, and this replaces them.** People buy courses and never finish them. A plugin doesn't have to be studied, because it does the work.
- **Coaching doesn't scale, and this does.** You can coach maybe twenty clients. A plugin can serve twenty thousand.
- **Almost no code.** Skills are plain-language text files. If you can write a clear process document, you can write a skill.

- **The market is early.** Plugin marketplaces are new, and experts who package their method now can own their category.
- **Recurring revenue.** Methods evolve, so updates, community, and the dashboard justify a membership.

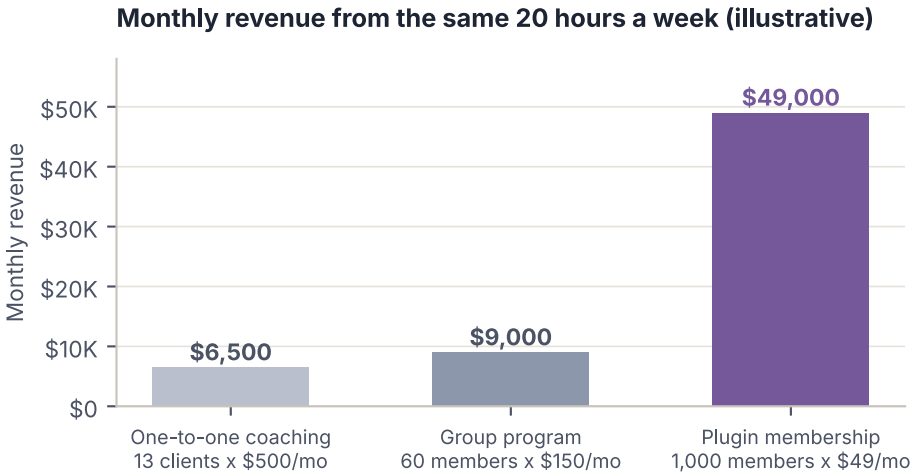


Figure 4.2 Monthly revenue from about twenty hours of the expert's time each week, under three delivery models (illustrative). The plugin doesn't consume the expert's hours per customer, so revenue is no longer capped by time.

CASE STUDY

"Local Growth OS" for a small-business marketing coach

Skill or part	What it teaches the AI
1. Diagnose (`/growth-audit`)	Review the business's website, local search listing, reviews, and social accounts. Score them against the coach's 20-point checklist and identify the top three gaps.
2. Plan (`/growth-plan`)	Turn the gaps into a 30-day plan using the coach's playbook: what to fix first, which channels, and a weekly content calendar.
3. Do (`/growth-week`)	Each week, draft posts, review responses, emails, and listing updates in the coach's templates and voice, and queue them for approval.
Sub-agent	A "Review Responder" that answers customer reviews exactly as the coach would teach.
Dashboard	Audit score, plan progress, drafts waiting for approval, and results (reviews gained, calls, traffic).

Business model: a $49–$99 monthly membership (plugin, updates, dashboard, private community), plus a higher tier with group coaching calls.

Step-by-Step Plan

- WEEK 1 — **Phase 0 — Define your method** and split it into diagnose, plan, do.
- WEEKS 1–2 — **Phase 1 — Write your three skills** and test them on real cases.
- WEEK 3 — **Phase 2 — Package it as a plugin** in a public or private repository.
- WEEKS 3–5 — **Phase 3 — Build the dashboard**, starting simple.
- WEEKS 5–6 — **Phase 4 — Monetize.** Use a free diagnose skill as the lead magnet.
- DAY 1 ONWARD — **Phase 5 — Own your audience** with an email list and community.
- WEEKS 6–12+ — **Phase 6 — Launch and grow.**

Phase 0: Define your method

1. **Write it down** as if training a new employee: your process start to finish, your frameworks and checklists, and what separates a good result from a bad one.

2. **Collect your examples:** best past deliverables, templates, scripts, and before-and-after cases. Examples are what make an AI follow your style instead of giving generic advice.
3. **Split it** into diagnose, plan, and do.
4. **Pick a narrow audience.** "Real-estate agents growing their listings" beats "anyone who wants to grow."

> **TRY THIS**
>
> Record yourself explaining your method, or coaching a real client, using an AI meeting recorder. Then ask an AI assistant to turn the transcript into numbered steps, checklists, and rules. It's much faster than writing from a blank page.

Phase 1: Write your three skills

A skill is usually a folder containing a `SKILL.md` file: a short description, so the AI knows *when* to use it, followed by your instructions. A simplified example:

```
---
name: growth-audit
description: Audit a local business's online presence using the
  Local Growth 20-point checklist. Use when the user asks for a
  marketing audit, a diagnosis, or "what should I fix first."
---

# Local Growth Audit

## Step 1: Gather
Ask for: business name, website, local listing link, top 2 competitors.

## Step 2: Score against the checklist
Score each item 0-2 (see checklist.md). Never guess; mark "unknown"
if you can't verify.

## Step 3: Report
Output: overall score out of 40, the top 3 gaps ranked by impact,
and one quick win doable today. Use the tone in voice.md: direct,
encouraging, no jargon.
```

Put supporting files in the same folder, such as `checklist.md`, `voice.md`, example reports, and templates, and the AI reads them when needed. Write and test skills in a paid AI assistant (some include a built-in "skill creator" to help you draft), then install and test the plugin in the assistant's desktop or coding

app. Run each skill on five to ten real scenarios and compare the output with what *you* would produce. Edit until it reaches 80 to 90 percent of your quality.

Phase 2: Package it as a plugin

A typical plugin is a folder you host in a code repository:

```
local-growth-os/
├── .plugin/plugin.json        ← name, version, description, author
├── skills/
│   ├── growth-audit/SKILL.md
│   ├── growth-plan/SKILL.md
│   └── growth-week/SKILL.md
├── commands/                  ← slash commands like /growth-audit
├── agents/
│   └── review-responder.md    ← sub-agent definitions
└── connectors.json            ← optional: links to email, CRM, dashboard
```

Exact folder and file names vary by AI platform, so follow your platform's plugin documentation. To distribute, many platforms let you publish a **marketplace**, which is simply a repository listing your plugins. Customers add your marketplace and install with one command. Use a public repository for a free version and a private one for paid access.

Phase 3: Build the dashboard

TABLE 4.2 • THREE LEVELS OF DASHBOARD

Level	How	Tool categories
Simple (week 1)	The agent writes progress into a shared document or table.	All-in-one workspace, spreadsheet-database, or online spreadsheet (agents can update these through existing connectors)
Medium	A web page the agent updates, showing scores, tasks, and drafts awaiting approval.	Interactive AI-generated pages with shared storage; no-code portal builders on a spreadsheet-database
Full product	Your own branded app with login, history, approvals, and charts.	AI app builder + hosted database + authentication service + an MCP server so agents can post updates

Design rule: the dashboard shows *status, results, and decisions waiting for approval*. It shouldn't be a complicated tool full of buttons. The agent does the work; the human watches and says yes or no.

Phase 4: Monetize

TABLE 4.3 • REVENUE MODELS

Model	How it works	Tool categories
Paid plugin access	One-time or annual payment unlocks a private plugin repository.	Digital-sales platforms that can grant repository access automatically; payment processors
Membership	Monthly fee for plugin, updates, community, and live calls.	Community and course platforms
Dashboard subscription	Plugin free for reach; dashboard with tracking and history is paid.	Payment processor subscriptions plus your dashboard app
Hybrid (recommended)	Free diagnose skill as lead magnet → paid plan and do + dashboard + community.	Any of the above

Pricing ideas: free for the diagnose skill alone; $29–$99 a month for the full plugin, dashboard, updates, and community; $300–$2,000 a month for a done-with-you tier with live coaching (a blend with Model 1); and team licenses for companies rolling it out to staff.

Phase 5: Own your audience from day one

This is the antidote to platform risk. Every customer should be on *your* list, not merely installed on someone else's platform. Use an **email newsletter platform**, a **community platform**, a **simple website**, and a **free CRM**. Require an email address for the free skill. Announce updates by email ("New in v1.3: /holiday-campaign"). Build versions for more than one AI platform, so no single company controls your business. Keep your method documents in your own files, because the plugin is only one way of packaging them.

Phase 6: Launch and grow

1. Your existing audience first: past clients, students, subscribers, followers.
2. Demo videos: the customer types one command, the agent works, the dashboard fills in. Show "watching, not clicking."
3. Plugin directories and marketplaces, plus a clear README in your repository.
4. Partnerships with associations, software vendors, and other coaches serving the same audience.

5. Webinars and workshops: teach the method live, then offer the plugin that does it.
6. Monthly iteration: dashboard data shows where people get stuck and which drafts get rejected. Improve the skills and add new ones members request.

Risks and How to Handle Them

Risk	Mitigation
Platform risk (the trap)	Own the email list and community, support multiple AI platforms, and keep your method independent of any one format.
People copying your skills	Installed skill files can be read. The real value is ongoing updates, the dashboard, the community, and you. Keep premium assets (data, templates, the dashboard) on your own servers.
Bad AI output harming your reputation	Approval steps, testing on real cases, "never do" rules, and showing sources and reasoning on the dashboard.
Customer data privacy	Keep data in the customer's own tools where possible, publish a clear privacy policy, and collect only what the dashboard needs.
Customers not using it	An onboarding email sequence, a five-minute first win (the diagnose skill), and weekly nudges.
Method too vague	If you can't write it as clear steps and checklists, refine it with real clients first.

Starter Budget

Item	Approximate monthly cost
AI assistant subscription	$20–$200
Code repository hosting	$0 (private repositories are commonly free)
Dashboard	$0–$25 to start; ~$25–$75 for a custom app later
Email newsletter platform	$0–$40
Community platform	$0–$99
Payments	A percentage of sales
Website	$0–$20
Total	**About $50–$450 a month**

Your Seven-Day Checklist

- ☐ **DAY 1** Write your method on one page, as if training a new hire.
- ☐ **DAY 2** Split it into diagnose, plan, and do.
- ☐ **DAY 3** Gather five to ten of your best examples and templates.
- ☐ **DAY 4** Write the diagnose skill and test it on five real cases.
- ☐ **DAY 5** Set up an email list (free tier) and a simple landing page.
- ☐ **DAY 6** Write the plan and do skills.
- ☐ **DAY 7** Show it to three to five past clients. Ask: “Would you pay $X a month for this plus a dashboard?”

CHAPTER SUMMARY

- Package your method as a plugin that installs into the customer's AI. Their agent does the work your way.
- A plugin combines skills, commands, sub-agents, and optional connectors, mostly written in plain language.
- Structure almost any method as diagnose, plan, do.
- The dashboard exists so customers can watch and approve, not operate software.
- Defend against platform risk by owning your email list and community and supporting multiple platforms.

Review Questions

1. In an agent-native system, who does the day-to-day work?
   A. The expert, by hand
   B. The customer's own AI agent, following the expert's method
   C. A call center
   D. The customer, using a course
2. What is the main purpose of the dashboard?
   A. To let customers do the work themselves
   B. To display ads
   C. To let customers watch progress and approve decisions
   D. To replace the plugin
3. Which best protects against platform risk?
   A. Lower prices
   B. Owning an email list and community, and supporting multiple platforms
   C. Adding more skills
   D. Making the plugin free
4. Name the four components that can go inside a plugin and describe each in one sentence.
5. Why does the chapter say a plugin can replace a course?
6. Describe the diagnose, plan, do structure, using a method you know.

7. What should a skill file's description tell the AI?
8. Explain the "hybrid" monetization approach.

APPLY IT • WORKSHEET 4

Turn your method into three skills

My method, in one sentence, and the narrow audience it serves:

Skill 1: Diagnose (inputs, checklist, output):

Skill 2: Plan (how a diagnosis becomes a plan):

Skill 3: Do (the recurring work the agent will produce):

What my dashboard shows (status, results, approvals):

How I'll own my audience (list, community, second platform):

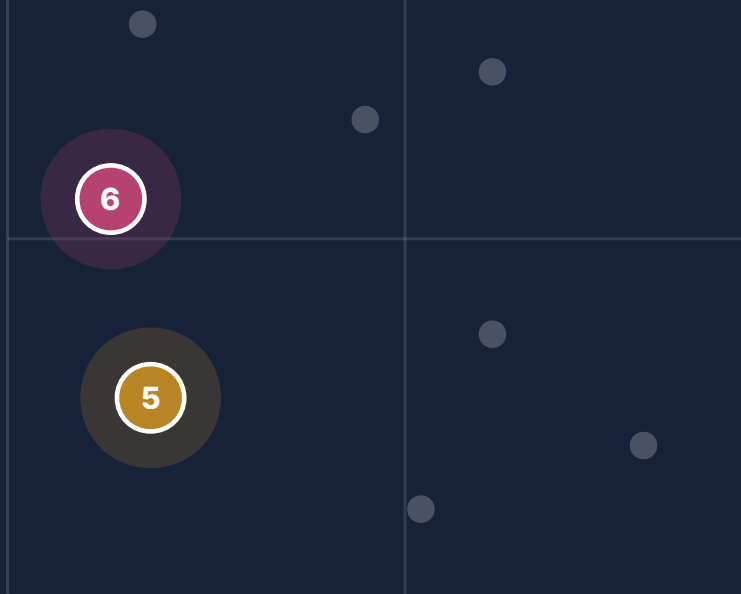
PART THREE

Serving Communities & Groups

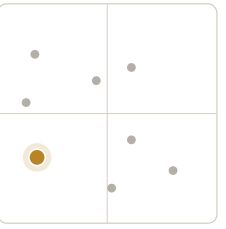
Two consumer models where belonging and everyday rituals matter more than technology. The first builds a product for the obsessive inner circle of a passion you share. The second puts a helpful AI inside the group chats where families, teams, and clubs already coordinate.

05 Taste-Led Niche Products

06 Ambient Consumer Agents



CHAPTER 5 • MODEL 5 OF 8



5

Taste-Led Niche Products

Everyone has the same tools. Nobody has your obsession.

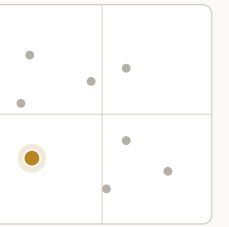
LEARNING OBJECTIVES

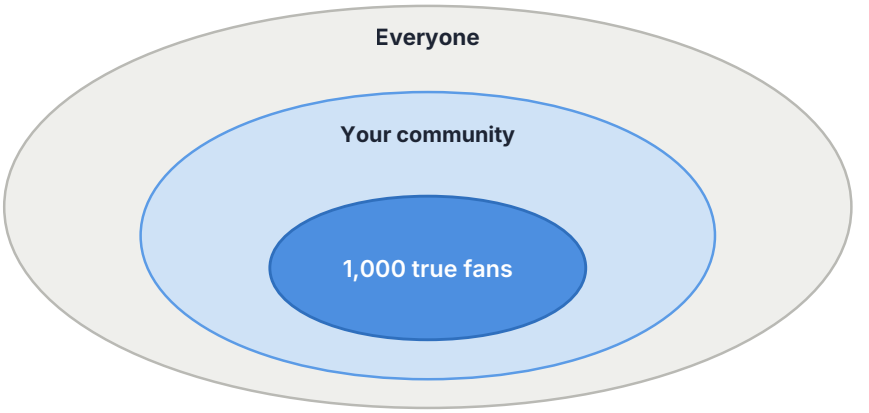
- Explain why taste becomes the scarce ingredient when building is cheap
- Apply the “1,000 true fans” math to a niche you know
- Choose a passion you’re genuinely part of and find its unmet needs
- Build a first version around one core action, with insider taste
- Earn money gently and grow by word of mouth

TYPE 5 OF 8

Taste-led niche products

Everyone has the same tools. Nobody has your obsession.





Build for the center. Let it spread outward.

Why now
When anyone can build, caring more than anyone else is the scarce thing.

Real-world example
A social film-diary app, built by and for film obsessives.

Proof it works
A roller-coaster logging app that started with one fan at a major theme park.

Who builds it
Anyone with a passion and a community to build for.

How it earns
Low-cost subscription, purchases, sponsors, affiliates.

The trap
Building for everyone. Build for the 1,000 who'll tell their friends.

This week
List three things you'd talk about for hours. Pick one you're part of.

In one line
Build for the inner circle.

Figure 5.1 Taste-led niche products at a glance. Don't build for everyone, or even for the whole community. Build for the thousand true fans at the center, and let them carry it outward.

Today, AI lets almost anyone build an app, a website, or a digital product in days. When everyone has the same powerful tools, the tools stop being an advantage. What's still rare is taste: deep knowledge of and love for a specific thing, plus the instinct for what the people who share that love want.

A *taste-led niche product* is built for a passion community you're genuinely part of. It doesn't try to please the whole world. It delights a small group of obsessed people so much that they pay for it and recruit their friends.

KEY TERMS

Taste
An insider's sense of what matters, what feels right, and what's missing in a community's world.

True fan
Someone who will use, pay for, and promote what you make.

Core action
The one thing your product does better than anything else, such as logging a film or a ride.

Affiliate commission
A percentage of a sale paid to you for referring a customer to another seller's product.

Word of mouth
Growth that comes from users recommending you, not from paid advertising.

The Model Explained

The three circles

Circle	What it means
Everyone	The general public. They don't care about your niche, and you shouldn't build for them.
Your community	Everyone who shares the passion: all roller-coaster fans, film lovers, or fly-fishers. Thousands to millions of people.
1,000 true fans	The obsessives who'll use your product daily, pay for it, give feedback, and tell everyone else. **Build for this circle first.**

The phrase *"1,000 true fans"* comes from a widely cited 2008 essay by the writer Kevin Kelly. A creator doesn't need millions of customers to make a good living, only about a thousand people who love the work enough to spend around $100 a year on it. That's $100,000 a year. With AI, one person can now build and run the whole product, so the math works for a solo founder.

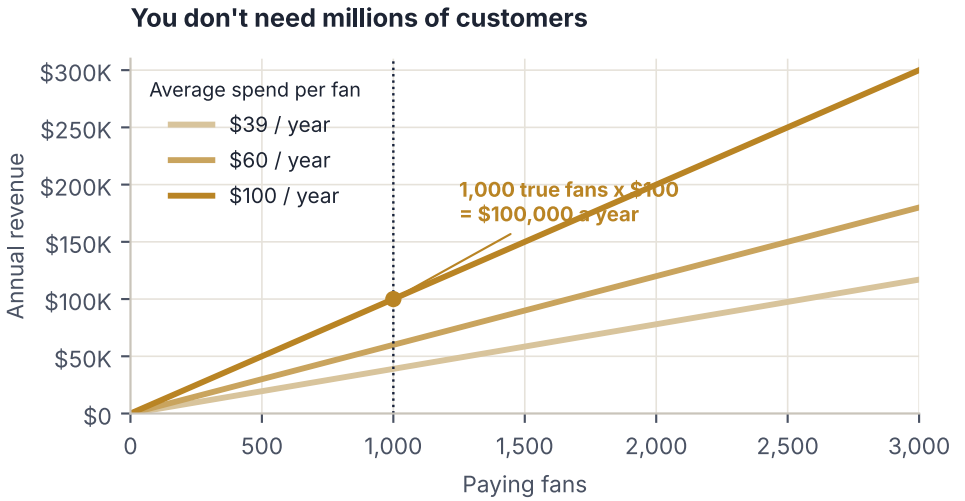


Figure 5.2 Annual revenue by number of paying fans and average yearly spend. A thousand fans spending $100 a year produce $100,000; the same fans at $39 produce $39,000. Raising spend per fan through tiers, merch, and events matters as much as adding fans.

The eight facts, unpacked

Why now. When anyone can build, caring more than anyone else becomes the scarce thing. A big company can copy features, but it can't copy your understanding of the community.

Real-world example. A social app where film lovers log, rate, and review every movie they watch was built by film obsessives for film obsessives. It started tiny and grew to millions of users, even though large movie databases already had more data. They didn't have the taste.

Proof it works. The framework's originator built a roller-coaster app that began at a major theme park: a fan built what coaster fans wanted, such as logging rides, tracking ride counts, and sharing with friends.

Who builds it. Anyone with a passion and a community to build for. You don't need to be a developer or a business expert. You need to be a genuine insider.

How it earns. A low-cost subscription ($3–$10 a month or $30–$60 a year), purchases (one-time unlocks, digital packs, merchandise), sponsors (brands in the niche paying to reach your audience), and affiliates (commissions when users buy gear you recommend).

The trap. Building for everyone. A generic product gets drowned out. A product built for the thousand who'll tell their friends spreads by word of mouth, for free.

How it compares

	Models 1–4	Model 5: Taste-led niche
Customer	Businesses and professionals	Hobbyists and enthusiasts
Your edge	Expertise, efficiency, data	Taste, passion, belonging
Sells on	Return on investment, time saved	Joy, identity, community
Growth	Sales and marketing	Word of mouth

On the map, this model sits in the lower left. It's among the least technical models and usually a lifestyle-scale business, often $50,000 to $500,000 a year rather than a venture-backed giant, though occasionally one becomes a household name.

Why This Is a Real Opportunity

- **Big companies ignore small passionate groups.** No technology giant will build a tool for competitive birdwatchers, but birdwatchers will gladly pay for one.
- **Building is nearly free.** AI app builders let a non-programmer launch a polished app in weeks.
- **Passion communities are loyal.** Hobbyists stay for years and promote what they love.
- **Marketing is built in.** You already know where the community gathers: its forums, chat servers, conventions, and creators.
- **It's sustainable for you.** You'll keep working on it because you care.

TABLE 5.1 • PRODUCT IDEAS BY PASSION

Passion	Possible product
Theme parks and coasters	Ride logger, wait-time predictor, trip planner
Home espresso	Shot log and dial-in assistant, bean tracker, recipe sharing
Fishing	Catch log with weather and tide conditions; spot-sharing among friends
Board games	Play tracker, collection manager, "what should we play tonight" for groups
Sneakers, watches, vinyl	Collection tracker, value tracker, release calendar
Hiking and national parks	Park passport tracker, trail log with badges
Gardening	Planting calendar by zip code, plant diary with photo progress
Running	Race log; a race finder for one type, such as trail or ultra
Cars and car shows	Build log, show calendar, meetup finder
Wine, whiskey, craft beer	Tasting journal, cellar tracker, regional discovery
Live music	Show and setlist logger: a film diary, but for concerts
Birdwatching	Life-list tracker, local sighting alerts

A PATTERN THAT WORKS

"A social diary for ___." Log it, rate it, collect it, share it with friends. People in hobbies love tracking, collecting, ranking, and showing off.

CASE STUDY

"Dialed" for home espresso obsessives

The inner circle: people with $1,000-plus home espresso setups who weigh every shot and swap advice in online espresso forums.

Their pain: dialing in each new bag of beans (grind, dose, time, taste) is trial and error, and their notes are scattered across phone notes and spreadsheets.

The product: log each shot in ten seconds (beans, grind setting, dose, yield, time, taste); an AI dial-in coach ("Sour and fast? Grind two steps finer."); a bean library with roaster details; recipe sharing with friends; and stats ("You've pulled 412 shots this year").

The money: a free basic log; Pro at $4.99 a month or $39 a year (AI coach, unlimited beans, stats); affiliate links to grinders, scales, and roasters; a sponsored "Roaster of the Month."

The math: 1,500 Pro members × $39 = $58,500 a year, before affiliate and sponsor income.

Step-by-Step Plan

WEEK 1	**Phase 0 — Pick your passion**, one you're genuinely part of.
WEEKS 1–2	**Phase 1 — Listen to the inner circle.** Collect complaints, spreadsheets, and wishes.
WEEKS 2–6	**Phase 2 — Build the first version.** One core action, done beautifully.
WEEKS 6–8	**Phase 3 — Launch to the inner circle.** Private beta with 50–100 true fans.
MONTHS 2–4	**Phase 4 — Make money, gently.**
MONTHS 3–12+	**Phase 5 — Grow by word of mouth.**

Phase 0: Pick your passion

1. **List three things you could talk about for hours.**
2. **Check that you're genuinely part of each one:** you do it regularly, you know the insider language, the debates, and the well-known people, and you're active in its online spaces.
3. **Check that the community is reachable and spends money:** active forums or chat servers with thousands of members; events, conventions, video creators, and podcasts; people who already buy gear, tickets, or apps.
4. **Pick one,** the one you'll still care about in three years, not the one that looks most profitable.

Research tools: forum-size statistics sites, video platforms, chat-server directories, social-network groups, and search-trend tools to see whether interest is growing.

Phase 1: Listen to the inner circle

1. **Lurk and collect complaints.** Read a year of top posts. Note every "I wish there was…," "Does anyone know an app that…," and "I track this in a spreadsheet…"
2. **Look at what exists.** Read the one- to three-star reviews of existing apps in the app stores.
3. **Talk to ten to twenty true fans.** "I'm building something for [hobby] people. Can I ask you five questions?" Ask what they track, what frustrates them, and what they'd love that doesn't exist.
4. **Pick one core action** your product will do better than anything else.

> **TRY THIS**
>
> Spreadsheets are a gold mine. When hobbyists track something in a spreadsheet, they're asking for an app. Paste a few hundred forum posts or app reviews into an AI assistant and ask it to group the complaints into themes and rank them by frequency.

Phase 2: Build the first version fast

Keep it small: one core action, done beautifully, with your community's taste built in. Use their language, their inside jokes, and their aesthetic.

TABLE 5.2 • TOOLS FOR THE FIRST VERSION, BY CATEGORY

Product type	Tool categories
Web app (fastest to launch)	AI app builders plus a hosted database and login service
Mobile app (for hobbies done in the field)	AI mobile-app builders, visual mobile-app builders, or a cross-platform framework with an AI coding assistant
Design	Interface-design tools, graphic-design tools, and AI image generators for icons and illustrations
Niche data	Public APIs and open data (always check the license) plus community-contributed data, which builds your moat
AI features (optional)	A model API for recommendations, coaching, summaries, and a "year in review"
Analytics and feedback	Product analytics (free tiers), a feature-request board, a community chat channel
App store accounts	Developer accounts for the two major mobile app stores (about $99 a year for one; a one-time $25 for the other)

THE TASTE CHECKLIST (THIS IS YOUR EDGE)

- ☐ Does it feel *made by one of us*, in its language, references, and design?
- ☐ Does it include the little details only insiders care about?
- ☐ Is the core action fast and satisfying?
- ☐ Is there something to **share**, such as stats, collections, badges, or a year in review? Sharing drives word of mouth.

Phase 3: Launch to the inner circle

1. Run a **private beta with 50–100 true fans** from your interviews. Make them feel like co-creators.
2. Create a **home for the community**; a group chat server is ideal for hobbies.
3. **Ship improvements every week** based on feedback, and announce each one. Visible responsiveness builds loyalty.
4. **Start an email list on day one.** It's your owned audience, so you don't depend on app stores or social algorithms.

Phase 4: Make money, gently

Hobbyists hate being squeezed but gladly support things they love.

TABLE 5.3 • REVENUE STREAMS FOR A NICHE PRODUCT

Stream	How	Tool categories
Low-cost subscription	Free core + Pro at $3–$10/month or $25–$60/year	Mobile subscription management; payment processor
Supporter tier	"Founding member" badge for fans who want to support you	Creator-membership and tipping platforms
One-time purchases	Lifetime unlock, themes, guide packs, printable logs	Payment processor, digital-download stores, in-app purchases
Sponsors	Niche brands sponsor features, newsletters, or events	Sponsorship-management tools or direct outreach
Affiliates	Commission on gear, tickets, and products you recommend	Retailer and brand affiliate programs and networks
Merchandise	Shirts, stickers, and pins with insider jokes	Print-on-demand services plus an online store
Events and trips	Meetups and group outings	Event-ticketing platforms

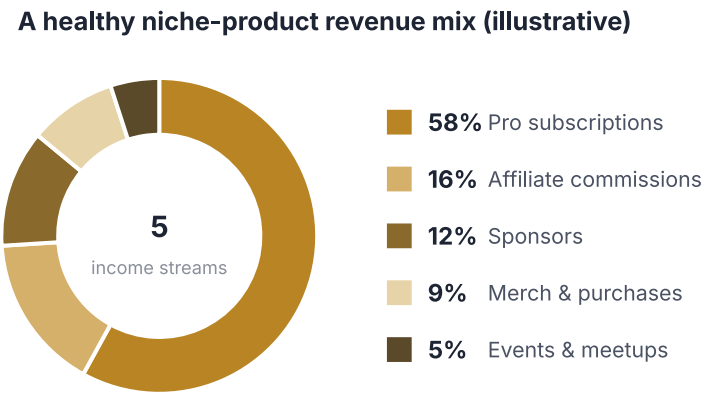


Figure 5.3 A healthy revenue mix for a niche product (illustrative). Several modest streams are more resilient than one large one.

Pricing rule: keep the core experience free so the community grows. Charge for "more": deeper stats, AI features, unlimited items, early access, and status.

Phase 5: Grow by word of mouth

1. **Shareable moments:** year-in-review cards, milestone badges, and collection images designed to look great when posted.
2. **Community creators:** give niche video makers and podcasters free Pro accounts. Their mentions reach the whole community.
3. **Show up in person** at conventions, meetups, tournaments, and parks. Stickers and QR codes at real events work very well.
4. **Authentic content** showing you as a member of the community using the product. Authentic beats polished.
5. **Referrals:** "Invite three friends, get a month of Pro free."
6. **Stay small on purpose.** Expand to the wider community only once the inner circle loves it. Never water it down for everyone.

Risks and How to Handle Them

Risk	Mitigation
Building for everyone (the trap)	Ask of every feature: "Would my thousand true fans love this?" Say no to the rest.
Niche too small to earn	Check the math early (fans × realistic price). Add affiliates, sponsors, and merchandise.
Burnout from community demands	A public roadmap, a predictable release rhythm, and volunteer moderators from among superfans.
Data or intellectual-property issues	Don't use copyrighted data, logos, or trademarks without permission. Mentioning a place by name is generally fine; using its logo isn't. Check API terms. Community-contributed data is safest.
Losing the passion	Pick something you'll love for years.
App store rules and fees	Stores take roughly 15–30%. Offer web sign-up where allowed and use subscription-management tools.

Starter Budget

Item	Approximate monthly cost
AI app builder	$20–$50
AI assistant (planning and help)	$20
Hosted database	$0–$25
Domain and hosting	~$0–$20
App store developer fees	~$8/month equivalent, plus a one-time $25
Community chat server, email list	$0 to start
Total	**About $50–$150 a month**, one of the cheapest models to start

Your Seven-Day Checklist

- ☐ **DAY 1** List three things you could talk about for hours.
- ☐ **DAY 2** Circle the one you're actively part of.
- ☐ **DAY 3** Join or rejoin its three most active online spaces.
- ☐ **DAY 4** Collect thirty or more "I wish there was..." posts and app complaints, and have an AI summarize the themes.
- ☐ **DAY 5** Find who's tracking things in spreadsheets. That's your opening.
- ☐ **DAY 6** Message ten true fans and ask your five questions.
- ☐ **DAY 7** Define your one core action and sketch the main screen.

CHAPTER SUMMARY

- When building is cheap, taste and insider understanding are the scarce ingredients.
- A thousand true fans spending about $100 a year can sustain a solo business.
- Pick a passion you're genuinely part of; mine forums, reviews, and spreadsheets for unmet needs.
- Build one core action beautifully, with something worth sharing.
- Keep the core free, charge for "more," and grow outward from the inner circle by word of mouth.

Review Questions

1. According to the "1,000 true fans" idea, roughly what does a thousand fans spending $100 a year produce?

   A. $10,000
   B. $100,000
   C. $1,000,000
   D. $1,000

2. What is the trap for this model?

   A. Charging too little
   B. Building for everyone instead of the inner circle
   C. Using AI features
   D. Launching on mobile

3. Which signal most strongly suggests an unmet need?

   A. Hobbyists tracking things in spreadsheets
   B. A large existing app
   C. A trending hashtag
   D. High app-store prices

4. Why can a small passionate niche beat a big company with more resources?

5. What is a core action? Give an example from a hobby you know.

6. List four ways a niche product can earn money besides subscriptions.

7. Why should the core experience stay free?
8. Name three ways to grow by word of mouth.

APPLY IT • WORKSHEET 5

Design for your inner circle

Three things I could talk about for hours (circle the one I'm part of):

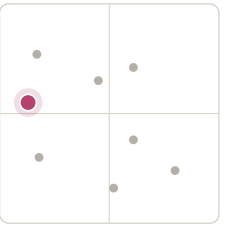
Where the community gathers (online and in person):

Top three complaints or wishes I found:

My one core action, and what users will share:

My fan math: ____ fans × $____ per year = $_____

CHAPTER 6 • MODEL 6 OF 8



6

Ambient Consumer Agents

The best new app might be a member of your group chat.

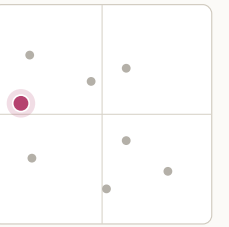
LEARNING OBJECTIVES

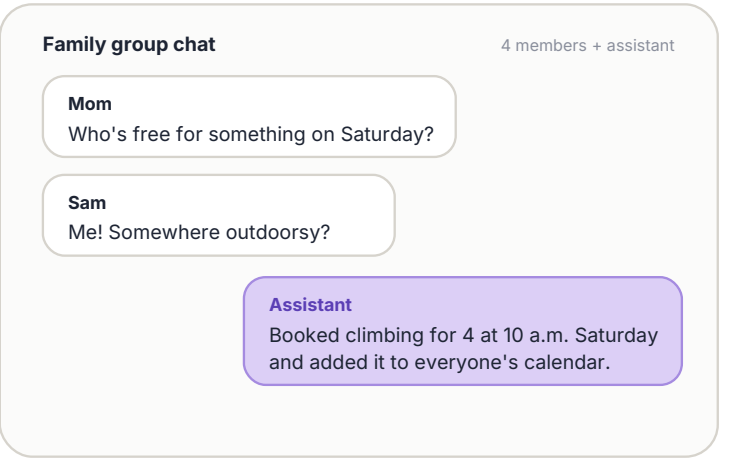
- Explain what an ambient agent is and why chat beats app downloads
- Identify group rituals that create weekly, not novelty, use
- Compare messaging channels and their rules for bots
- Design an agent's behavior, memory, and integrations
- Handle privacy, consent, and pricing for groups

TYPE 6 OF 8

Ambient consumer agents

The best new app might be a member of your group chat.





Why now
Agents can now plan, book, and remember. No app download needed.

Real-world example
AI assistants built into the world's biggest messaging apps.

Proof it works
A family assistant that lives in group chats and answers by voice on a laptop.

Who builds it
Builders who know a group's rituals: families, clubs, teams.

How it earns
Per-group subscription, premium features, booking fees.

The trap
Novelty. Not used weekly after a month? It's a toy.

This week
Spot the job someone always ends up doing in your group chat.

In one line
It lives where people already talk.

Figure 6.1 Ambient consumer agents at a glance. Nobody opened an app: the assistant read the conversation, understood what the group wanted, booked it, and updated everyone's calendar inside the chat.

For fifteen years, building a consumer product meant building an app people download. Getting people to download, open, and keep using a new app is brutally hard. This model flips the problem around: instead of asking people to come to your app, your AI goes to where they already talk.

An *ambient consumer agent* lives in a family group chat, a friends' text thread, a team's chat, or a voice assistant on a computer. "Ambient" means in the background and always around. It stays quiet until someone needs something, and then it plans, books, and remembers for the group.

KEY TERMS

Ambient
Present in the background of everyday life, available without opening a separate app.

Group ritual
A recurring routine a group shares: weekly dinners, carpools, game nights, practice schedules.

Retention
The share of users or groups still active after a period of time.

Bot API
A messaging platform's official interface for automated accounts.

Calendar invite (.ics)
A standard file format that adds an event to almost any calendar app.

The Model Explained

Reading the diagram

A family group chat. Mom asks, "Who's free for something on Saturday?" Sam replies, "Me! Somewhere outdoorsy?" Then the assistant posts: "Booked climbing for 4 at 10 a.m. Saturday and added it to everyone's calendar."

Look at what *didn't* happen. Nobody opened an app, searched for climbing gyms, compared times, collected availability, booked, or sent calendar invites. The work that usually lands on one tired family member just got done.

The eight facts, unpacked

Why now. Agents can now plan (work out options), book (take real actions online), and remember (who's vegetarian, which child has practice on Tuesdays). With no app to download, the biggest barrier in consumer technology disappears.

Real-world example. AI assistants are built into the largest messaging and social apps, so billions of people already have one inside the chat they use every day.

Proof it works. The framework's originator built a family assistant that lives in family group chats and can also be spoken to on a laptop.

Who builds it. Builders who know a group's rituals. Every group has recurring routines, and if you know them intimately, you know what the assistant should do.

How it earns. A per-group subscription ($5–$15 a month per family or team, not per person), premium features, and booking fees or commissions.

The trap. Novelty. People add a fun AI to the chat, play with it for a week, and forget it. If it isn't used weekly after a month, it's a toy, and toys don't make businesses. It must be tied to a recurring job.

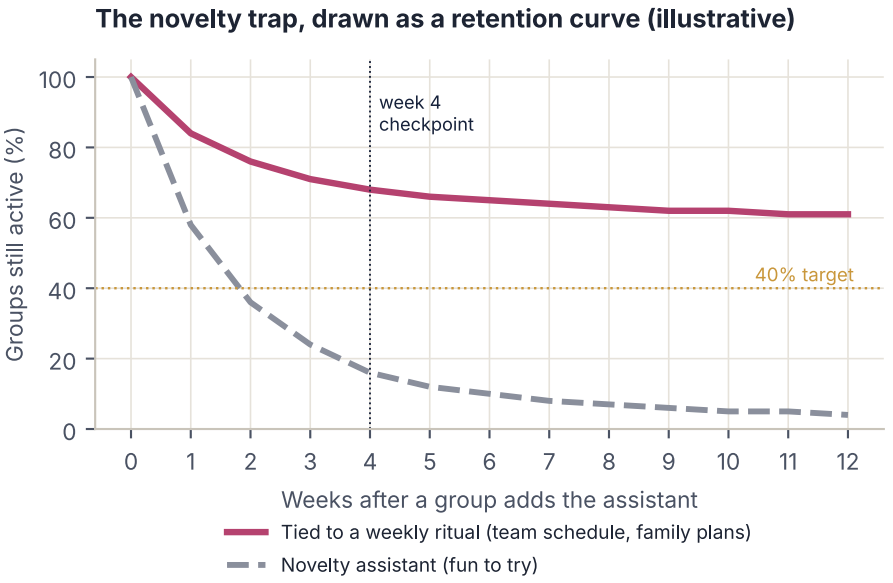


Figure 6.2 The novelty trap as a retention curve (illustrative). Assistants tied to a weekly ritual hold most groups past the four-week checkpoint; novelty assistants fall below the 40 percent target within three weeks.

The hidden superpower: every use is a demo

When the assistant books climbing for the family, four people watch it happen, and each of them is in other group chats with friends, teams, and coworkers. Growth is built into the product. That's why this model sits apart from Model 5:

	Model 5: Taste-led niche	Model 6: Ambient consumer agent
Lives in	Your own app or website	Existing chat apps and voice
Serves	Individuals in a passion community	Groups: families, friends, teams, clubs
Core value	Tracking, collecting, belonging	Coordinating and getting things done
Growth	Word of mouth	Viral by design: every member sees it work

Group Rituals Are Product Opportunities

TABLE 6.1 • THE JOB SOMEONE ALWAYS GETS STUCK WITH

Group	The stuck job	Assistant idea
Families	Weekend plans, school events, pick-ups, birthday gifts	Family logistics assistant

Group	The stuck job	Assistant idea
Youth sports parents	Schedules, snack rotation, carpools, RSVPs, fees, rain-outs	Team manager in the parents' chat
Friend groups	"When are we all free?", picking a restaurant, splitting the bill	Plans-maker: polls, books, splits costs
Trip groups	Itinerary, bookings, shared expenses, packing lists	Group trip planner
Book and supper clubs	Dates, picking the book or host, reminders	Club organizer
Running and hiking clubs	Routes, weather calls, attendance, carpools	Club coordinator
Roommates	Chores, bills, groceries, house rules	House manager
Adult rec leagues	Subs, schedules, standings, fees	League assistant
Volunteer groups	Sign-ups, shifts, reminders	Volunteer scheduler
Caregivers for a parent	Medication reminders, appointments, visit schedule	Care coordination assistant
Wedding parties	Budget, bookings, payments, timeline	Event planner

CASE STUDY

"Coach Kit" for youth sports parents

The ritual. Every youth team has a chaotic parents' group chat: "What time is the game?" "Who's bringing snacks?" "Can someone drive Jake?" "Did everyone pay?" One parent, the team manager, does all the work.

What the assistant does in the chat. Posts the week's schedule every Sunday and a reminder the night before; turns "Who's in for Saturday?" into an automatic RSVP count; runs the snack rotation and matches carpools ("Sarah is driving Jake and Mia"); watches the weather, flags rain-outs, and updates calendars; collects team fees with a payment link and tracks who's paid; and answers "Which field?" instantly.

Weekly use is guaranteed, because games happen every week of the season. That's the cure for the novelty trap.

The money. $29–$49 per team per season (paid by the manager or the club), a league-wide license for organizations with 50+ teams, and a small fee on payments collected.

Step-by-Step Plan

WEEK 1	**Phase 0 — Find the job** that happens weekly in a group you know.
WEEKS 1–2	**Phase 1 — Be the bot yourself** in two or three real groups.
WEEK 2	**Phase 2 — Choose your channel.**
WEEKS 2–6	**Phase 3 — Build the agent:** brain, memory, calendar, booking, payments.
BEFORE LAUNCH	**Phase 4 — Privacy and trust.**
MONTHS 2–3	**Phase 5 — Monetize.**
MONTHS 2–12	**Phase 6 — Launch and grow.**

Phase 0: Find the job

1. **Scroll back through your own group chats.** Look for repeated questions, the person who always organizes or collects money, and the things that fall through the cracks.
2. **Write down five recurring jobs** and how often each happens. Weekly or more often is required to avoid the novelty trap.
3. **Pick a group type you know from the inside** and one job to start with.
4. **Talk to ten organizers**, the people stuck with the job. Ask how they do it now, what's most annoying, and what they'd pay never to do it again.

Phase 1: Be the bot yourself

Before building anything, do the job by hand in two or three real groups. Join as "the assistant" and handle every request, using an AI assistant behind the scenes to draft plans, messages, and schedules. You'll learn quickly what people actually ask for, when the assistant should speak and when it should stay quiet (nobody wants a chatty bot), and what tone feels right in a family or friends' chat.

Phase 2: Choose your channel

This is the most important technical decision. Each chat platform has different rules for bots, **and those rules change often, so check each platform's current policies before building.**

TABLE 6.2 • MESSAGING CHANNELS COMPARED

Channel type	Bot friendliness	Best for	Notes
Open messaging apps with public bot platforms	Excellent: official bot APIs; bots join groups easily	Fastest prototype; international users; clubs	Free bot accounts and mature developer libraries
Community chat servers	Excellent	Gaming groups, clubs, communities	Rich bot features: roles, commands, events
SMS / group text	Good; group messaging has limits	Families and teams (everyone has text messaging)	Use a cloud communications provider; pay per message
Phones' built-in messaging apps	Limited: no official group-bot API; third-party bridges exist	Friend and family groups where people actually talk	Test reliability and read the terms carefully; official business messaging is usually one-to-one
The largest encrypted messaging app	Restricted: official business API with strict rules	Huge reach outside the U.S.	The platform has restricted general-purpose AI assistants on its business API, and group support is limited. Read its current AI policy first.
Workplace chat	Excellent	Work teams	Mature app directories
Voice	Build your own	Hands-free family use; older users	Voice-agent platforms, real-time speech APIs, phone numbers from a cloud communications provider

Recommendation: prototype on an open messaging app with a public bot platform, or on a community chat server (both easy and free), or on SMS through a cloud communications provider (works for everyone). Add the harder channels once you've proven weekly use.

Phase 3: Build the agent

TABLE 6.3 • THE AGENT'S PARTS, BY TOOL CATEGORY

Part	What it does	Tool categories
Brain	Understands the conversation, decides what to do, writes replies	Frontier AI model API with strong tool use; an agent SDK
Memory	Remembers members, preferences, schedules, past plans, allergies	Hosted database plus a long-term memory layer
Calendar	Adds events to everyone's calendar	Calendar APIs or a unified calendar API; standard .ics invite links work for anyone
Finding things	Places, restaurants, activities, events	Maps and places APIs, local-business review APIs, event-listing APIs
Booking	Actually reserving things	Partner and affiliate APIs for restaurants, activities, hotels, and flights; where no API exists, browser-automation platforms (check each site's terms)
Payments	Collect fees, split bills	Payment-processor links and marketplace payments; deep links to peer-to-peer payment apps
Polls and RSVPs	"Who's in?"	Native chat polls or your own tracking
Reminders	"Game tomorrow at 9!"	Background job schedulers or scheduled tasks
Hosting and monitoring	Runs 24/7; catches mistakes	App-hosting platforms; AI call logging, error tracking, product analytics

RULES FOR A GOOD GROUP-CHAT AGENT

- [ ] **Speak only when called or truly useful:** when mentioned, or when there's a clear task. Never chime in on every message.
- [ ] **Confirm before spending money:** "Climbing for 4 at 10 a.m. Saturday, $120 total. Book it?"
- [ ] **Short, friendly, human messages.** It's a family chat, not a corporate email.
- [ ] **Remember things** so people don't repeat themselves.
- [ ] **Be proactive at the right moments,** like the night-before reminder or the weather warning, and never with random chatter.
- [ ] **Easy to pause and easy to remove.**

Phase 4: Privacy and trust (non-negotiable)

- **Consent:** everyone in the group should know the assistant is there. It introduces itself when added.
- **Minimal data:** store only what's needed (plans, preferences), and delete raw messages after processing.
- **Children:** if children are in the chat, children's online-privacy laws (such as COPPA in the U.S.) apply. Many products target adults only.
- **A clear privacy policy:** generate one with a policy-generator service, then have it reviewed.
- **AI providers that don't train on your data** (major model APIs don't by default).
- **Security:** encrypt stored data and lock down API keys.

Phase 5: Monetize

TABLE 6.4 • REVENUE MODELS FOR GROUP AGENTS

Model	Example
Per-group subscription	$5–$15/month per family; $29–$49 per team per season
Free tier + premium	Free reminders and polls; premium booking, calendar sync, memory, payment collection, voice
Booking fees and commissions	A small fee per booking or affiliate commissions from activity and travel partners
Organization licenses	Leagues, clubs, schools, or congregations pay for all their groups
Payment processing	A small percentage when collecting team or trip money

Who pays: almost always the organizer, the person who was stuck doing the job. Aim your marketing at them.

Phase 6: Launch and grow

1. Start with ten to twenty groups you can reach personally.
2. Track one number: **weekly active groups**. If fewer than about 40 percent are still active after four weeks, fix the product before growing.
3. Design for virality: a light sign-off (“Planned by your assistant. Add me to your group: link”) and an easy way to add it to other groups.
4. Go where organizers gather: team-manager groups, parent-teacher groups, parenting forums, league administrators.
5. Partner with organizations. One youth league can roll you out to hundreds of groups.
6. Short videos of a chaotic group chat getting sorted out in seconds are very relatable on short-video platforms.
7. A landing page with one big “Add to your group” button.

Risks and How to Handle Them

Risk	Mitigation
Novelty (the trap)	Tie it to a weekly ritual and measure four-week retention before spending on growth.
Platform rules and policy changes	Start on bot-friendly platforms, support several channels, and keep your own list of organizers' contact details.
Big-tech competition	Go deep on one group type's rituals. Generic assistants won't know the snack rotation or the league rules.
AI mistakes (wrong booking, wrong time)	Confirm before every booking or payment, show details clearly, make undo easy, and log everything.
Annoying the group	Speak only when mentioned or useful; offer a mute command.
Privacy and children's data	Consent, minimal storage, adults-only by default, clear policy.
Booking APIs are hard to get	Start with affiliate APIs and pre-filled booking links; automate real booking later.

Starter Budget

Item	Approximate monthly cost
AI model API usage	$20–$100 early on
AI coding assistant	$20–$200
App hosting	$5–$25
Hosted database	$0–$25
Bot accounts on open messaging apps or chat servers	Free
SMS number and messages	~$1–$2 per number plus ~1¢ per message
Built-in messaging app bridge (if used)	~$50–$200+
Total	**About $50–$400 a month** for a working prototype

Your Seven-Day Checklist

- [ ] **DAY 1** Scroll through your five most active group chats.
- [ ] **DAY 2** List every recurring job someone gets stuck doing, with its frequency.
- [ ] **DAY 3** Pick the weekly job in a group type you know well.
- [ ] **DAY 4** Ask ten organizers about it.
- [ ] **DAY 5** Be the assistant yourself in one or two real chats, with AI help.
- [ ] **DAY 6** Write down exactly when the bot speaks, what it does, and what it remembers.
- [ ] **DAY 7** Choose your starting channel.

CHAPTER SUMMARY

- Ambient agents skip the app download by living where people already talk.
- Every use is seen by the whole group, so growth is built in.
- Anchor the assistant to a weekly group ritual, or it will become a forgotten toy.
- Choose bot-friendly channels first; platform rules vary and change.
- Consent, minimal data, and confirm-before-paying are non-negotiable. The organizer is the buyer.

Review Questions

1. What does “ambient” mean in this model?

   A. Loud and attention-seeking
   B. In the background and available without opening a separate app
   C. Voice-only
   D. Free

2. Which retention signal suggests the product is a toy?

   A. 65% of groups active at week four
   B. Fewer than 40% of groups active at week four
   C. High sign-ups in week one
   D. Groups adding it to new chats

3. Who usually pays for a group assistant?
   A. Every member equally
   B. Advertisers
   C. The organizer who was stuck doing the job
   D. The messaging platform
4. Why does this model have "built-in virality"?
5. List three rules for how a group-chat agent should behave.
6. Why does the chapter recommend prototyping on open messaging apps or SMS?
7. What privacy steps are required before launch?
8. Name three group rituals that happen at least weekly.

APPLY IT • WORKSHEET 6

Design your group assistant

The group type and its weekly ritual:

The stuck job, and who's stuck with it:

When the assistant speaks (and when it stays quiet):

What it remembers about the group:

Starting channel, and why:

Price, and who pays:

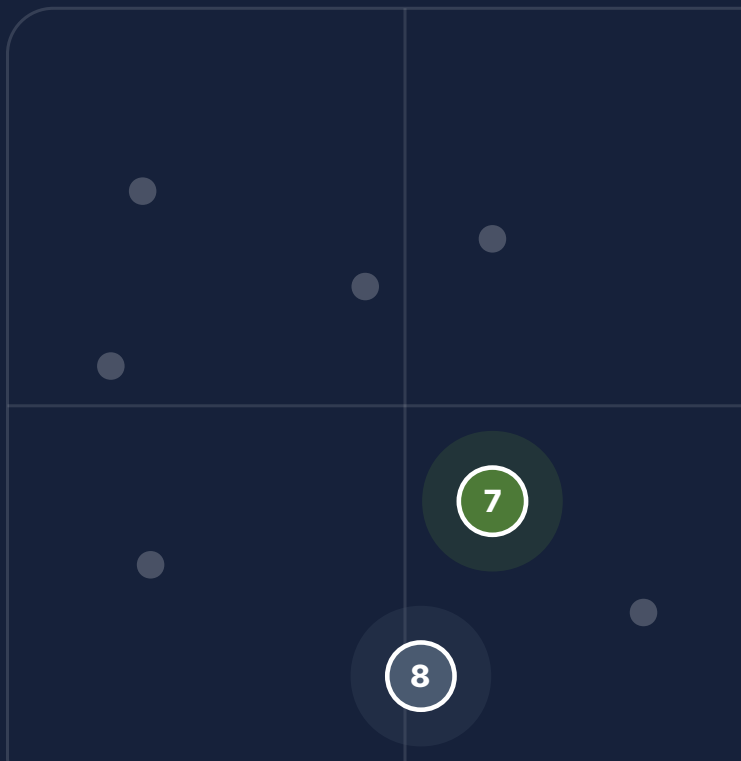
PART FOUR

Building Assets That Compound

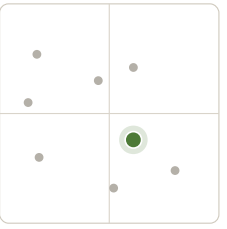
Two models that start slower but become harder to copy every month. The first collects data no one else has and sells the answers it makes possible. The second gives its code away to earn trust and adoption, then sells everything around it.

07 Proprietary Data Products

08 Open-Source-Core Products



CHAPTER 7 • MODEL 7 OF 8



7

Proprietary Data Products

Anyone can copy your app. Nobody can copy your data.

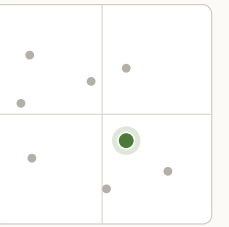
LEARNING OBJECTIVES

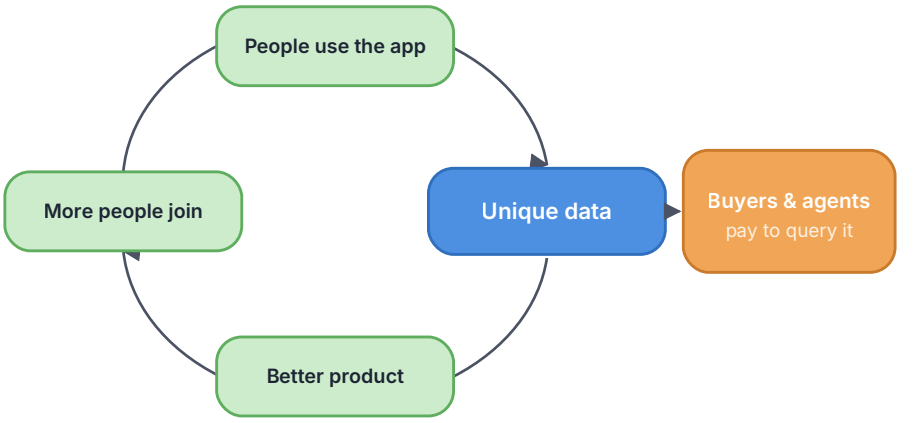
- Explain the data flywheel and why data is a durable moat
- Judge whether a dataset is valuable, using six qualities
- Solve the cold-start problem with proven collection strategies
- Collect data legally and ethically, with consent and anonymization
- Package one dataset into several products for different buyers

TYPE 7 OF 8

Proprietary data products

Anyone can copy your app. Nobody can copy your data.





Why now
Models and code are commodities. Fresh niche data is scarce.

Real-world example
Fitness-app route heatmaps, crowd-sourced traffic from navigation apps, anonymous salary-sharing sites.

Proof it works
Not built yet. The idea: ride logs from the roller-coaster app become a dataset.

Who builds it
Builders in a community whose data nobody captures.

How it earns
Free app. Paid insights, reports, or API access.

The trap
Collecting without consent, or building a dataset nobody needs.

This week
Name a question your niche can't answer. Collect that data.

In one line
The data is the product.

Figure 7.1 Proprietary data products at a glance. Use creates unique data, the data improves the product, a better product attracts more users, and the loop speeds up. Separately, buyers and AI agents pay to query the data.

With AI, anyone can rebuild your app's features in a weekend. What they can't rebuild is the unique information your users have given you over months or years: their activities, prices, experiences, reviews, and results. That body of data is proprietary, meaning only you have it, and it can become your most valuable and most defensible asset.

In a *proprietary data product*, you build a useful app whose real business is the unique data it collects, then sell insights from that data to people and AI agents who need answers nobody else can provide.

KEY TERMS

Proprietary data
Information only you possess, gathered through your own product or relationships.

Data flywheel
A self-reinforcing loop in which usage creates data, data improves the product, and a better product attracts more users.

Cold start
The early problem that no one contributes until there's data, and there's no data until people contribute.

Give-to-get
Users contribute their own data to unlock everyone else's.

Aggregation and anonymization
Combining data into group statistics and removing identifying details, so no individual can be singled out.

Re-identification
Working out who a person is from supposedly anonymous data.

The Model Explained

The data flywheel

A flywheel is a heavy wheel that's hard to start spinning but, once moving, keeps itself going and speeds up. The *data flywheel* works the same way:

1. **People use the app** because it's useful to them: logging runs, rating rides, reporting prices.

2. **Unique data** accumulates as a side effect.
3. **The product gets better:** smarter recommendations, accurate averages, live maps.
4. **More people join,** because the product now beats the alternatives.
5. **Back to step 1**, with even more data, and the cycle accelerates.

The side arrow, **Unique data → "Buyers and agents pay to query it,"** shows a second revenue path. Separately from the app, businesses, researchers, and increasingly AI agents pay to ask questions of your data, usually through an API. This connects directly to Model 3: your data can be the source behind a headless tool.

The eight facts, unpacked

Why now. Models are trained on the public internet, so they know nothing about what happened last week in your niche or about things nobody ever wrote down. Whoever owns that data holds what AI can't generate.

Real-world examples. A fitness-tracking app turned millions of logged runs and rides into heatmaps, and sells aggregated data to city planners designing bike lanes. A crowd-sourced navigation app collected live traffic speeds simply because drivers used it, and that data made it worth about a billion dollars to an acquirer. An anonymous salary-sharing site built a salary database no employer would publish by asking people to share theirs in order to see others'.

Proof it works. The framework's originator admits they haven't built one yet. Their idea is that ride logs from their coaster app could become a dataset of popular rides, real wait times, closures, and ratings by park. Most data products start exactly this way, as a side effect of a useful app.

Who builds it. Builders inside a community whose data nobody captures, where important information lives only in people's heads, group chats, or spreadsheets.

How it earns. A free app maximizes contributors. Revenue comes from paid insights inside the app, reports sold to businesses, and API access for businesses and agents.

The trap. Two traps, in fact. (1) **Collecting without consent** is unethical, often illegal, and destroys trust. (2) **A dataset nobody needs** is worthless, no matter how large it is.

Start from a valuable question, then design the collection. Never the other way around.

How it connects to the other models

Data often sits *underneath* other models. A taste-led app (Model 5) collects it; a headless API (Model 3) sells access to it; it makes a vertical harness (Model 2) smarter than its rivals; and a service business (Model 1) generates it as a by-product, such as real prices from thousands of jobs.

What Makes Data Valuable

TABLE 7.1 • SIX QUALITIES OF VALUABLE DATA

Quality	Meaning	Example
Answers a money question	Someone makes a financial decision based on it	"What should I pay?" "Where should we build?"
Hard to get elsewhere	Not public, not easily scraped	Real salaries; actual contractor quotes
Fresh	Updated constantly; old data loses value	Today's traffic; this week's prices
Granular	Specific enough to act on	By zip code, model, and date
Trustworthy	Verified, enough data points, little fraud	Receipts uploaded, not just claims
Legally clean	Collected with consent and clear rights	Users agreed in plain language

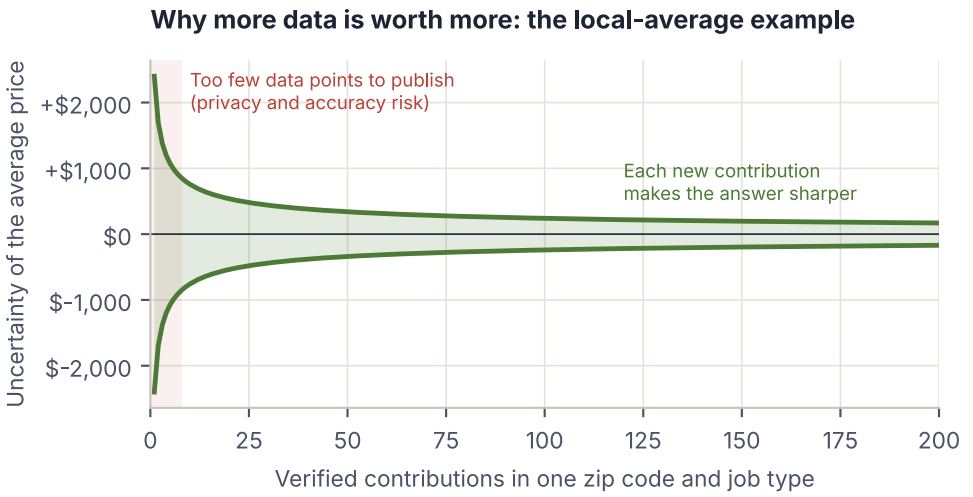


Figure 7.2 Why more data is worth more (illustrative). The uncertainty of a local average shrinks as verified contributions grow. Below a minimum count, results are too unreliable, and too identifying, to publish.

TABLE 7.2 • QUESTIONS NICHES CAN'T ANSWER YET

Niche	Unanswered question	Who'd pay
Home services	What does a roof, HVAC system, or kitchen remodel actually cost in my area?	Homeowners, contractors, insurers, lenders, agents
Restaurants	What are nearby restaurants paying suppliers, or paying in rent?	Owners, suppliers, investors
Small business	What do businesses like mine pay for software, insurance, payment processing?	Owners, vendors, brokers
Freelancers	What do freelancers in my field really charge per project?	Freelancers, agencies, clients
Theme parks	Real wait times, ride downtime, and crowd levels by day	Visitors, travel agents, operators, planning apps
Used goods	What do used cameras, golf clubs, or instruments really sell for?	Buyers, sellers, resellers, insurers
Local services	How long is the wait at this office, clinic, or vet?	Residents, cities
Healthcare (regulated)	What did my procedure actually cost with my insurance?	Patients, employers, benefits brokers
Rentals	What do tenants really pay, and how responsive is this landlord?	Renters, managers, investors

Niche	Unanswered question	Who'd pay
Weddings and events	What do venues, photographers, and caterers actually charge?	Couples, vendors, planners

CASE STUDY

"FairQuote" for home-improvement prices

The question nobody can answer: what does a roof replacement really cost in Tampa? Online estimates are vague ranges, and contractors don't publish prices.

How to collect it (give-to-get): homeowners upload a contractor quote or invoice to see what others paid for the same job nearby. AI reads the document and extracts job type, size, materials, price, zip code, and date. Personal details are stripped out automatically.

The flywheel: more quotes make local prices more accurate, so more homeowners check prices before hiring, so more quotes come in.

The money: free basic ranges for contributors; a $9.99 "Fair Price Report" for one job in one zip code; a contractor subscription at $49–$199 a month ("How do my prices compare?"); API and licensing deals with insurers, lenders, real-estate platforms, and AI home assistants; and an annual "State of Home Prices" report that earns press coverage.

The moat: after two years and 50,000 verified quotes, no competitor can catch up quickly.

Step-by-Step Plan

WEEK 1	**Phase 0 — Find the valuable question** and validate it with five business buyers.
WEEKS 1–3	**Phase 1 — Design the collection engine** and solve the cold start.
BEFORE COLLECTING	**Phase 2 — Consent, privacy, and legal.**
WEEKS 3–6	**Phase 3 — Store, clean, and verify.**
WEEKS 6–10	**Phase 4 — Turn data into products.**
MONTHS 3–12+	**Phase 5 — Grow the flywheel.**

Phase 0: Find the valuable question

1. List ten questions people in your niche ask that have no good answer: "Does anyone know how much…," "Is it normal that…," "What's the best time to…"
2. For each, ask: who makes a **money decision** with this answer? Would **businesses** pay? Can I collect it **legally, with consent**? Can people contribute in **under sixty seconds**? Does it need to be **fresh**? (Fresh is better, because it keeps you needed.)
3. Pick one question with at least three types of buyers.
4. **Validate with buyers early.** Ask five businesses: "If I had verified data on X by zip code, would you pay? How much? In what format?"

Phase 1: Design the collection engine

The hardest problem in data businesses is the *cold start*. These strategies solve it:

TABLE 7.3 • SOLVING THE COLD START

Strategy	How it works	Example
Single-player utility	Useful to one person even with zero other users	Logging your own run is useful alone
Give-to-get	Contribute data to unlock others' data	Share your salary to see salaries
Passive collection	Data is a by-product of normal use (with consent)	Navigation reports traffic speed
Seed with public data	Start with cleaned-up public records, then add user data	Permits, public price filings
Pay or reward contributors	Gift cards, credits, free premium	$5 per verified receipt
Partner for bulk data	Businesses share data in exchange for free insights	Contractors share pricing for benchmarks

Make contributing effortless: a photo upload with AI extraction, one-tap logging, or automatic capture. Every extra field cuts contributions. Tools by category: AI app builders or simple online forms to start; AI vision models or specialized receipt- and document-reading services for extraction; location services (with consent); public open-data portals and, where permitted, web-data extraction services for seeding; digital gift-card platforms for rewards; and survey panels to seed early responses.

Phase 2: Consent, privacy, and legal

This is the first half of the trap, so treat it as non-negotiable.

1. **Plain-language consent:** tell contributors exactly what you collect, how it's used, and that aggregated, anonymized insights may be sold. No hidden fine print.
2. **Anonymize and aggregate:** sell patterns, not people. Strip identifiers, and never show results based on just one or two data points. A common rule is to publish averages only when a group has five to ten or more contributors.
3. **Watch for *re-identification*.** One fitness app's public heatmap once revealed the locations of secret military facilities. Location data needs special care: blur start and end points and suppress sparse areas.
4. **Know the laws.** In the U.S., California's privacy law and a growing list of state laws give people rights, including opting out of the "sale" of their data; health data carries extra rules (HIPAA and state health-data laws). In the EU

and UK, GDPR requires a lawful basis such as explicit consent, and a right to deletion. For children, COPPA applies in the U.S., so avoid collecting from children.

5. **Have a lawyer review** your terms and privacy policy before launch. It's worth a few hundred to a couple of thousand dollars.

Tool categories: privacy-policy and consent-banner generators; privacy-request management platforms as you grow; and open-source differential-privacy libraries for advanced, privacy-preserving statistics.

Phase 3: Store, clean, and verify

Raw data is messy, and clean, trustworthy data is what people pay for. Use a hosted relational database to start and a cloud data warehouse as you grow; data-transformation tools to clean and model; data-pipeline tools to pull from multiple sources; AI to standardize categories ("roof replacement" = "re-roof" = "new roof"), extract fields, and flag oddities; automated data-quality testing; fraud and duplicate detection (duplicate receipts, impossible values, suspicious bursts); and notebook or BI tools to explore. **Trust signals** to build in: "verified by receipt" badges, a sample size beside every number ("based on 47 quotes"), last-updated dates, and outlier removal.

Phase 4: Turn data into products

TABLE 7.4 • ONE DATASET, MANY PRODUCTS

Product	Buyer	Pricing
Free app with basic insights	Contributors, consumers	Free; keeps the flywheel spinning
Premium insights in the app	Power users	$5–$15/month or $5–$30 per report
Reports	Businesses, press, investors	$99–$5,000 per report; an annual "State of..." report as marketing
Dashboards	Businesses wanting ongoing access	$99–$2,000+/month
API access	Software companies, AI agents	Per query ($0.01–$1+) or subscription tiers
MCP server	AI assistants and agents	Usage-based (see Chapter 3)
Bulk data licensing	Large companies, researchers, insurers, funds	$10,000–$500,000+ a year; cloud data marketplaces can list your data

"Agents pay to query it" is the newest and fastest-growing buyer. When someone asks their AI, "Is $14,000 a fair price for a roof in Tampa?", the agent could call your API for the answer. Make your data agent-friendly: clear docs, JSON output, and an MCP server.

Phase 5: Grow the flywheel

1. **Seed the first 500–1,000 data points** yourself with rewards, partners, public data, and your community.
2. **Launch in one area first** (one city, one park, one category) so the data is dense and useful, then expand. Thin data everywhere is useless.
3. **Publish free insights as marketing:** "The real cost of a roof in Florida, 2026" earns press, search traffic, and contributors. Use chart-publishing tools for shareable graphics.
4. **Close the loop:** show contributors their impact ("Your quote helped 212 neighbors").
5. **Approach business buyers** once coverage is real, with sample reports as the hook.

6. **Track flywheel metrics:** contributions per week, returning contributors, coverage, and revenue per data point.

Risks and How to Handle Them

Risk	Mitigation
Collecting without consent (trap)	Plain-language consent, anonymization, aggregation thresholds, legal review; never sell individual-level personal data.
A dataset nobody needs (trap)	Start from a money question and validate with five buyers before building.
Cold start	Single-player utility, give-to-get, seed data, rewards, one launch area.
Bad or fake data	Receipt verification, fraud rules, outlier removal, visible sample sizes.
Re-identification	Minimum group sizes, blurred locations, no rare combinations published.
Scraping or IP issues	Respect terms and robots.txt; prefer contributed and licensed data; get legal advice for gray areas.
Slow to monetize	Sell reports early while the API and licensing grow.

Starter Budget

Item	Approximate monthly cost
App builder or AI coding assistant	$20–$200
Hosted database	$0–$25
AI model API (extraction, cleaning, reports)	$20–$200
Document-reading service (optional)	$0–$100
Contributor rewards to seed data	$0–$500 (optional but powerful)
Privacy tools	$10–$30
Dashboards (free tiers)	$0–$50
Legal review (one time)	~$500–$2,500
Total	**About $50–$1,000 a month**, depending mostly on how hard you seed data

Your Seven-Day Checklist

- ☐ **DAY 1** List ten questions your niche can't answer.
- ☐ **DAY 2** Score each one: money decision? Multiple buyers? Consent possible? Easy to contribute?
- ☐ **DAY 3** Pick one question and name three buyer types.
- ☐ **DAY 4** Talk to three to five potential business buyers.
- ☐ **DAY 5** Design the collection method: single-player value, give-to-get, or rewards.
- ☐ **DAY 6** Draft your plain-language consent statement.
- ☐ **DAY 7** Launch a simple online form or prototype and collect your first fifty data points.

CHAPTER SUMMARY

- Code and models are commodities; fresh, niche, proprietary data is scarce and hard to copy.
- The data flywheel (use → data → better product → more users) accelerates over time.
- Valuable data answers money questions, is hard to get, fresh, granular, trustworthy, and legally clean.
- Solve the cold start with single-player value, give-to-get, seeding, and one dense launch area.
- Consent and anonymization come first; then package the same data as reports, dashboards, APIs, and licenses.

Review Questions

1. What is the "cold start" problem?

   A. Servers that run slowly at first
   B. Nobody contributes until there's data, and there's no data until people contribute
   C. Launching in winter
   D. Investors losing interest

2. Which practice best protects contributors' privacy?
   A. Publishing every individual record
   B. Showing averages only when a group has enough contributors
   C. Selling raw location traces
   D. Hiding the privacy policy
3. Which is one of the two traps in this model?
   A. Charging for reports
   B. Building a dataset nobody needs
   C. Using an API
   D. Launching in one city
4. Draw or describe the data flywheel in four steps.
5. List the six qualities of valuable data.
6. Explain give-to-get with an example of your own.
7. Why should you launch in one area before expanding?
8. Name four products you could build from one dataset.

APPLY IT • WORKSHEET 7

Design your data product

The question my niche can't answer:

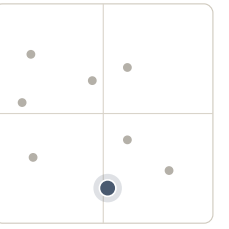
Three buyer types, and the money decision each makes:

How I'll collect it (and solve the cold start):

My plain-language consent statement:

My launch area, and the minimum count before publishing results:

CHAPTER 8 • MODEL 8 OF 8



8 Open-Source-Core Products

When code is cheap to write, give it away and sell everything around it.

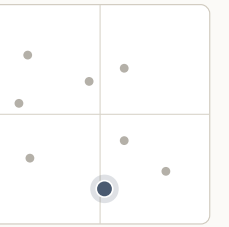
LEARNING OBJECTIVES

- Explain why giving code away can be a winning business strategy
- Describe the revenue "below the waterline": hosting, paid tiers, support, community
- Choose a license that protects your business
- Build a repository, a hosted cloud, and a community
- Launch, monetize, and avoid giving away every reason to pay

TYPE 8 OF 8

Open-source-core products

When code is cheap to write, give it away and sell everything around it.





Give away the tip. Sell everything beneath it.

Why now
Code is cheap. Trust, community, and distribution aren't.

Real-world example
An open-source app backend, open-source product analytics, open-source scheduling: open code, paid cloud.

Proof it works
The framework's originator makes the full case for open source in a separate talk.

Who builds it
Builders happy to work in public and grow a community.

How it earns
Hosted cloud, paid tiers, support, sponsorship.

The trap
Giving it all away with no reason to pay.

This week
Publish a project in a public code repository. Offer a hosted version.

In one line
This is where the money is.

Figure 8.1 Open-source-core products at a glance. The free code is the visible tip of the iceberg. Hosting, paid tiers, support, and community sit below the waterline, and that's where the business makes its money.

Open-source software is software whose code is published for anyone to see, download, run, and modify, free. At first glance that sounds like a terrible business. Why give away your product? This chapter's answer is that AI has made code cheap. Anyone can now generate an app's code quickly, so the code itself is worth less every month.

What's still hard to get is **trust**, **a community**, and **distribution**, meaning people actually finding and using your product. Giving the code away is the fastest way to earn all three. An *open-source-core* business then makes its money from everything around the code.

KEY TERMS

Open source
Software whose code is publicly available under a license that lets others use, study, modify, and share it.

Self-hosting
Running software on your own servers instead of paying a provider to run it.

Hosted cloud
The vendor runs the software for you, handling setup, updates, backups, and uptime.

Open core
A free core product plus paid features for businesses.

License
The legal terms that define what others may do with your code.

Contributor license agreement (CLA)
An agreement that gives a project the rights it needs to code submitted by outside contributors.

The Model Explained

The iceberg

Above the waterline, "what people see": the free public code. **Below it, "where the money is":**

- **Hosting:** most people don't want to install, run, update, secure, and back up software themselves, so they pay you to run it.

- **Paid tiers:** features businesses need, such as team permissions, single sign-on, audit logs, advanced analytics, and higher limits.
- **Support:** guaranteed help, fast response times, setup assistance, custom work.
- **Community:** sponsorships, events, partner marketplaces, certifications, and the goodwill that drives sales.

The free code brings *users*. What's below the waterline turns some of them into *customers*.

The eight facts, unpacked

Why now. If anyone can recreate your code with AI, hiding it protects you less than it used to. Publishing it earns developers' trust (they can inspect it), community contributions, and visibility.

Real-world examples. An open-source alternative to a big-tech app backend that most people pay to use as a hosted cloud, now a multi-billion-dollar company. An open-source product-analytics platform with public code and a paid cloud. An open-source scheduling tool, free to self-host, with paid hosted and enterprise versions. The common pattern is open code and a paid cloud.

Who builds it. Builders happy to work in public: sharing progress, answering questions in public issue trackers and chat, accepting outside contributions, and treating users as collaborators.

How it earns. Hosted cloud (the biggest earner for most), paid tiers, support contracts, and sponsorship.

The trap. Giving it all away with no reason to pay. Many popular open-source projects earn almost nothing because everything valuable is free and easy to self-host. Design the reason to pay from day one.

Why It Works, and Why It's Growing

1. **Trust:** businesses can inspect the code for security and quality, so there's no black box.
2. **No lock-in fear:** customers know they can self-host if your company disappears, which makes them *more* willing to buy your cloud.

3. **Free distribution:** repository stars, developer news sites, self-hosting forums, and word of mouth replace expensive marketing.
4. **Free help and feedback:** the community reports bugs, suggests features, writes integrations, and translates.
5. **An instantly understood pitch:** "Like [expensive product], but open source and cheaper." Every pricey software category is a target.
6. **Agents and developers lean toward open tools:** popular open-source projects are well represented in what AI coding assistants know, so they tend to get recommended and used.

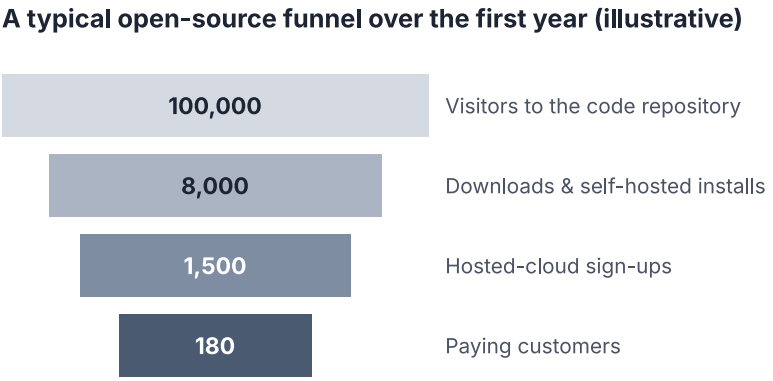


Figure 8.2 An open-source funnel over a first year (illustrative). Only a small share of people who see the code become paying customers. That's normal, because free users are the marketing.

Ways to Make Money Below the Waterline

TABLE 8.1 • OPEN-SOURCE REVENUE MODELS

Model	How it works	Typical example
Hosted cloud	You run it: no servers, updates, or backups for the customer. Often 70–90% of revenue.	A database backend's cloud service
Open core	Free core; paid business features (single sign-on, roles, audit logs, compliance, reporting).	Enterprise editions of developer platforms
Usage tiers	Free up to a limit, then pay as you grow.	Free monthly analytics events
Support and SLAs	Guaranteed response times, dedicated help, priority fixes.	The original open-source business model
Setup and consulting	Install, customize, or migrate for customers.	Common early revenue
Sponsorship	Individuals and companies fund development.	Code-platform sponsorships, open collectives
Marketplace and add-ons	Paid plugins, templates, themes, integrations.	Website-platform ecosystems
Dual licensing	Companies pay for a commercial license to avoid open-source obligations.	Developer frameworks and databases
Training and certification	Courses and certified-partner programs.	Larger projects

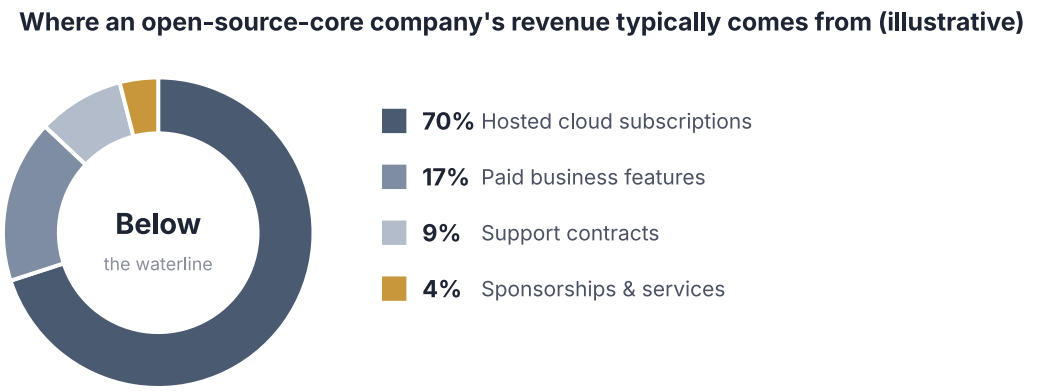


Figure 8.3 Where an open-source-core company's revenue typically comes from (illustrative). Most revenue comes from customers who would rather pay you to run the software than run it themselves.

Best starting combination: free self-hosted version, plus a paid hosted cloud, plus paid business features, with a support add-on.

Choosing a License

The *license* decides what others may legally do with your code. Licenses fall along a spectrum from maximum adoption to maximum protection.

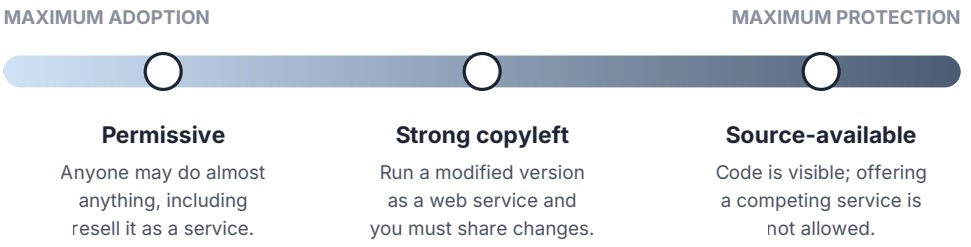


Figure 8.4 The license spectrum. Permissive licenses (such as MIT or Apache 2.0) maximize adoption; strong copyleft (such as AGPLv3) discourages others from reselling your work as a service; source-available or "fair source" licenses (such as FSL or BSL) prohibit competing services outright, often converting to open source after a few years.

TABLE 8.2 • COMMON LICENSE CHOICES

License	What it means	Good for
MIT / Apache 2.0	Anyone can do almost anything, including selling a competing cloud service.	Maximum adoption; libraries and developer tools
AGPLv3	Anyone running a modified version as a web service must publish their changes.	Apps where you want to protect your cloud business
FSL / BSL / similar source-available	Code is visible and usable, but competing services are prohibited; often becomes open source after 2–4 years.	Strongest protection (purists don't call these "open source")
Open-core split	Main code under MIT or AGPL; an enterprise folder under a commercial license.	Clean separation of free and paid features

RULE OF THUMB

For a full application you'll sell as a cloud product, consider AGPL or a fair-source license, plus a separate commercial license for enterprise features. For a small library, use MIT. Have a lawyer look at your choice before launch, and use a contributor license agreement so outside contributions don't complicate your licensing later.

CASE STUDY

"StudioOS," an open-source booking platform for fitness studios

The target. Boutique fitness studios (yoga, Pilates, climbing, martial arts) often pay $150–$500+ a month for booking software and complain about price and lock-in.

Above the waterline (free): class schedules, member booking, waitlists, basic memberships, email reminders. A tech-savvy owner can self-host it with one container command.

Below the waterline (paid): StudioOS Cloud at $49–$149 a month per studio (no servers, automatic updates and backups); Pro features (integrated payments with a small transaction fee, multi-location, a branded mobile app, an AI front-desk assistant, advanced reporting); a $500 migration service; support plans for franchises; equipment-brand sponsorships and a paid integrations marketplace.

Why customers pay: studio owners don't want to run servers. The open code earns trust ("you'll never be locked in"), and the cloud is the convenience they buy.

Stacking: StudioOS could add an ambient agent (Model 6) for the studio's member chat, or collect consented, anonymized pricing and attendance trends across studios (Model 7).

Step-by-Step Plan

WEEK 1	**Phase 0 — Pick the project** and design the reason to pay.
WEEKS 1–6	**Phase 1 — Build the free core** with one-command install.
WEEKS 4–10	**Phase 2 — Build the hosted cloud.**
WEEKS 6–10	**Phase 3 — Docs and community.**
WEEKS 8–12	**Phase 4 — Launch.**
MONTHS 3–12+	**Phase 5 — Monetize and grow.**

Phase 0: Pick the project

1. **Choose a target:** an open-source alternative to an expensive product people complain about; a tool you built for yourself that others in your field need; or a niche tool that doesn't exist yet (combine with Models 2 or 5).
2. **Check demand:** search software-alternatives directories for existing open-source options and gaps; search self-hosting and open-source forums for "is there an open-source alternative to…"; read review-site complaints about the incumbent's price and lock-in.
3. **Design the reason to pay before writing code.** Write down what's free (enough to be genuinely useful), what's paid (convenience, business features, compliance, support), and why a business would pay rather than self-host (setup time, maintenance, security, updates, backups, uptime).

Phase 1: Build the free core

Write it with an AI coding assistant on a modern web framework and a popular open-source database. Package it in **containers** with a single compose file for a one-command install, and later provide charts for large-company container orchestration. Offer **one-click deploy buttons** for popular app-hosting platforms and self-hosting dashboards. Host the code in a **public repository** with issues, discussions, and automated testing. Add **dependency and security scanning** and a `SECURITY.md` file, and automate releases and changelogs.

> **WHAT A GREAT OPEN-SOURCE REPOSITORY NEEDS**
>
> - ☐ A README that explains in ten seconds what it is and who it's for, with a screenshot and a "quick start in two minutes"
> - ☐ One-command install
> - ☐ A clear LICENSE file
> - ☐ CONTRIBUTING guidelines, a code of conduct, and issue templates
> - ☐ "Good first issue" labels to welcome newcomers
> - ☐ A visible link: "Don't want to self-host? Try our hosted cloud free."

Phase 2: Build the hosted cloud

This is the main revenue engine. Use app-hosting platforms for simplicity, or major cloud providers and budget server hosts at scale; managed databases with automated backups; an authentication service (enterprise single sign-on is itself a great paid feature); a payment processor, plus a usage-billing or open-source-focused billing platform if you meter usage; license-key management for paid self-hosted features; and uptime monitoring, error tracking, and product analytics. **Rule:** the cloud must be noticeably easier than self-hosting, with instant sign-up, no setup, automatic updates, backups, and support.

Phase 3: Docs and community

Publish a documentation site (with an `llms.txt` file so AI assistants can learn your tool), a community chat server or forum, a public roadmap with feature voting, an email newsletter you own, sponsorship pages, and star-history tracking. Work in public: share weekly progress, publish a changelog, answer early questions quickly, and credit contributors.

Phase 4: Launch

1. **Developer news sites,** with a "Show" post: "[Product], an open-source alternative to [category leader]." This is often the single biggest launch channel.
2. **Self-hosting and open-source forums**, plus niche forums for your users.
3. **Product-launch sites** for broader tech reach.
4. **Directories:** software-alternatives sites, curated "awesome" lists, and container image registries.

5. **Video:** “Here’s the free code anyone can run, and here’s the cloud version where it just works.” Self-hosting video creators love reviewing new tools.
6. **Comparison pages** (“[Product] vs. [incumbent]”) for search traffic.

Phase 5: Monetize and grow

1. **Track the funnel:** stars → installs (via opt-in telemetry) → cloud sign-ups → paying customers.
2. **Companies self-hosting the free version** are your best leads for support and enterprise features.
3. **Add business features progressively:** single sign-on, roles, audit logs, compliance certification, SLAs.
4. **Price in tiers:** self-hosted $0; cloud starter and pro $19–$149/month; business $200–$1,000/month; enterprise $10,000–$100,000+ a year.
5. **Keep the free version genuinely good.** Taking features away is the fastest way to lose community trust; add paid value on top instead.
6. **Optional funding:** some investors specialize in open-source companies, but many open-source businesses are profitably bootstrapped.

Risks and How to Handle Them

Risk	Mitigation
No reason to pay (the trap)	Design paid value up front: convenience, business features, support, compliance.
A large cloud provider resells your code	AGPL or a fair-source license for apps; enterprise features under a commercial license.
Maintainer burnout	Clear contribution guidelines, issue templates, community moderators, saying no often, prioritizing paying customers.
Popular but not profitable	Watch cloud conversion early; talk to companies using the free version.
Security vulnerabilities	Dependency scanning, a reporting process, prompt patches.
License confusion	Choose on day one, use a CLA, get a short legal review.
Too technical	Partner with a developer co-founder, or start with a smaller tool built with an AI coding assistant.

Starter Budget

Item	Approximate monthly cost
Public code hosting	Free
AI coding assistant	$20–$200
Cloud hosting for the hosted version	$20–$100 early
Managed database	$0–$25
Documentation site, community chat	$0
Domain and website	~$10–$20
Legal review of license (one time)	~$300–$1,500
Total	**About $50–$350 a month**. The code and community cost time more than money.

Your Seven-Day Checklist

- ☐ **DAY 1** List five expensive tools people in your world complain about, or one you built for yourself.
- ☐ **DAY 2** Check alternatives directories for gaps.
- ☐ **DAY 3** Draw your iceberg: free above the line, paid below.
- ☐ **DAY 4** Choose a license (AGPL or fair source for an app, MIT for a library).
- ☐ **DAY 5** Build a minimal version with an AI coding assistant, with a great README and one-command install.
- ☐ **DAY 6** Publish it with a “hosted version coming soon” waitlist.
- ☐ **DAY 7** Post it to one community and collect feedback.

CHAPTER SUMMARY

- When code is cheap, giving it away buys trust, community, and distribution.
- The money is below the waterline: hosted cloud, paid business features, support, and community.
- Choose your license deliberately; AGPL or fair-source licenses protect a cloud business.
- Make the cloud dramatically easier than self-hosting, and keep the free version genuinely good.
- Design the reason to pay before you write the first line of code.

Review Questions

1. For most open-source-core companies, what is the largest revenue source?
   A. Donations
   B. The hosted cloud version
   C. Selling the source code
   D. Advertising
2. Which license most strongly discourages a competitor from reselling your app as a service while remaining open source?
   A. MIT
   B. Apache 2.0
   C. AGPLv3
   D. Public domain
3. What is the trap for this model?
   A. Writing documentation
   B. Giving everything away with no reason to pay
   C. Accepting contributions
   D. Launching on developer news sites
4. Why does open code make customers *more* willing to buy a cloud subscription?
5. List the four things below the waterline.
6. What should a great README include?

7. Why is taking features away from the free version risky?
8. Describe the “open-source alternative to X” pitch and why it works.

APPLY IT • WORKSHEET 8

Draw your iceberg

The project (an alternative to what, or a tool for whom):

Above the waterline (free features):

Below the waterline (paid: cloud, features, support, community):

Why a business would pay instead of self-hosting:

My license choice, and why:

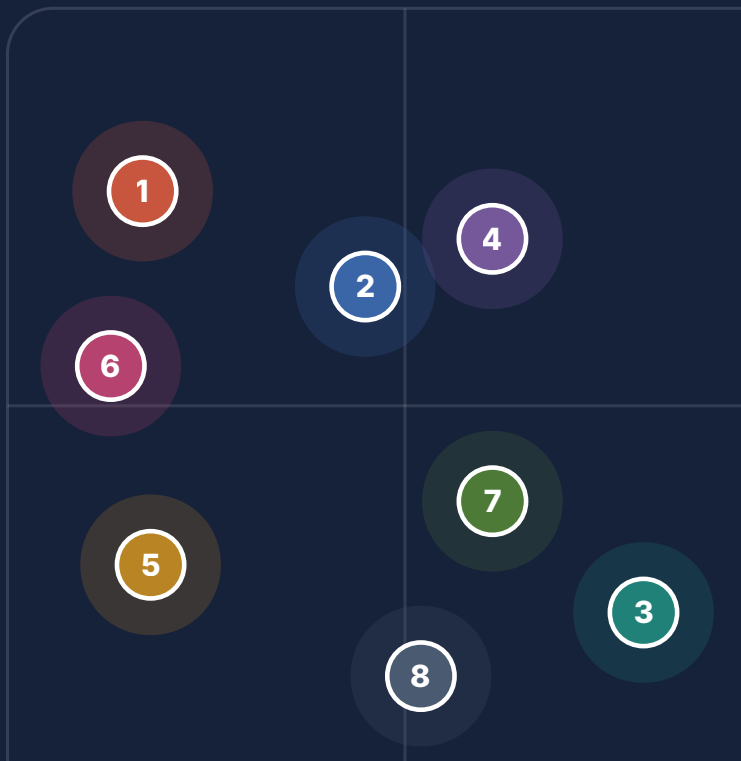
PART FIVE

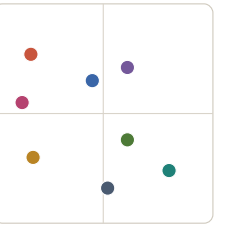
Choosing & Launching Yours

You've seen all eight models. Now match one to what you already have, test it with real people in a week, and follow a ninety-day plan to your first revenue, then stack models to build something that lasts.

09 How to Pick Yours

10 Your First Ninety Days





9

How to Pick Yours

Don’t pick the model that sounds exciting. Pick the one that matches what you already have.

LEARNING OBJECTIVES

- Read the decision table: advantage needed, revenue, technical lift
- Answer the three deciding questions about yourself
- Score all eight models against your own situation
- Validate your top choice with real people in one week

DECISION GUIDE

How to pick yours

Type	Best if you have	How it earns	Technical lift
1. Software-enabled services	Service experience and clients	Retainers, per outcome	Low to medium
2. Vertical agent harnesses	Deep niche expertise	Premium subscription, per job	Medium
3. Headless AI software	A data source or a hard, narrow skill	Usage-based, open-source core	Medium to high
4. Agent-native systems	A repeatable method	Plugin access, membership	Low to medium
5. Taste-led niche products	A passion and a community	Subscription, purchases, sponsors	Low to medium
6. Ambient consumer agents	A group whose rituals you know	Per-group subscription, bookings	Medium
7. Proprietary data products	Access to data nobody captures	Insights, reports, API	Medium
8. Open-source-core products	Comfort building in public	Hosted cloud, support, sponsors	Medium to high

1. What do you know better than almost anyone?
That's your niche.

2. Who already pays for that today, and how?
Service fees: type 1. Pro tools: types 2–4. Fans and groups: types 5 and 6. Data buyers: type 7.

3. Who does the work?
A person watching a screen points left on the map. An agent in a loop points right.

Figure 9.1 The decision guide. The table matches each model to the advantage it requires, how it earns, and how much building it takes; the three questions turn that table into a personal choice.

This is the chapter that turns eight ideas into one decision. It has two parts: a comparison table, and three questions to ask yourself. The key insight is simple: your existing advantage is your head start. You don't choose a model because it sounds exciting; you choose the one that fits what you already have.

Part 1: Reading the Table

Each row of the decision table in Figure 9.1 is one model. The columns ask three things:

Column	What it really asks
Best if you have	What unfair advantage you need to start with
How it earns	Where the money comes from
Technical lift	How much building or coding it requires

TABLE 9.1 • "BEST IF YOU HAVE," IN PLAIN ENGLISH

Model	You're a fit if...
1. Software-enabled services	You've done the work (bookkeeping, marketing, recruiting) and know people who'd pay for it.
2. Vertical agent harnesses	You know an industry's workflow, rules, and pain points better than outsiders do.
3. Headless AI software	You can do one technical thing extremely well, or have access to unique data.
4. Agent-native systems	You have a framework or process you teach or use with clients.
5. Taste-led niche products	You're an obsessive insider in a hobby.
6. Ambient consumer agents	You know how families, teams, or clubs coordinate.
7. Proprietary data products	You're in a world where valuable information isn't being collected.
8. Open-source-core products	You like sharing work openly, building software, and growing a community.

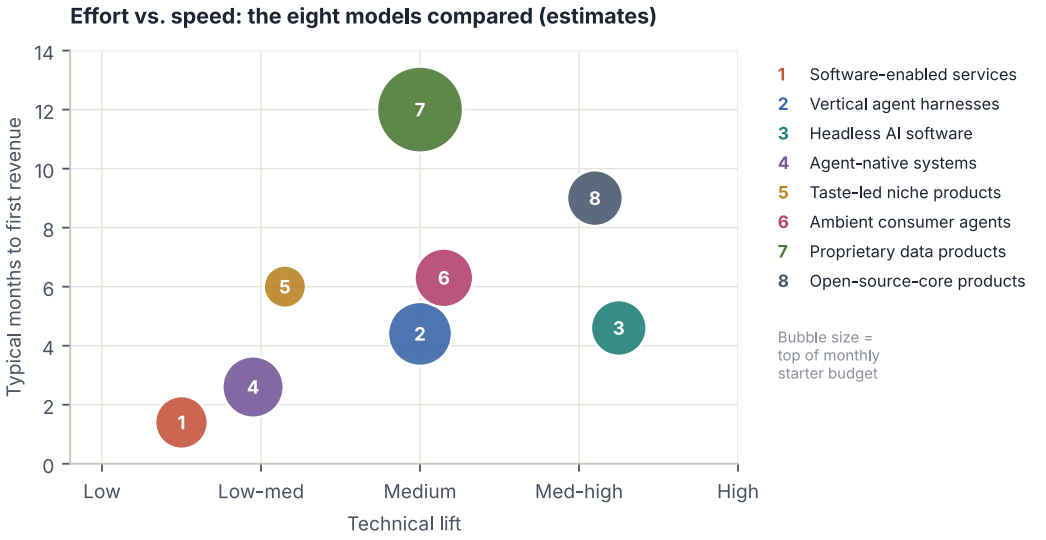


Figure 9.2 Technical lift versus typical months to first revenue for each model (estimates). Models in the lower left (services, agent-native systems) earn soonest with the least building; models in the upper right take longer but tend to be harder to copy.

Part 2: The Three Deciding Questions

Question 1: What do you know better than almost anyone?

That's your niche. You don't research it from scratch; it's what you already know from your job, career, hobbies, family life, or community. "Better than almost anyone" doesn't mean world-class. It means better than the average person, and far better than an engineer who has never done that work.

Question 2: Who already pays for that today, and how?

This points you to the right model by identifying the existing customer:

TABLE 9.2 • FOLLOW THE MONEY THAT'S ALREADY FLOWING

If people currently pay...	Look at...	Why
Service fees (firms, consultants, freelancers)	Model 1	Keep selling the service, delivered ten times more efficiently
For professional tools	Models 2, 3, 4	Sell a better tool: a harness, an agent tool, or a plugin
As fans and groups (hobby and group spending)	Models 5, 6	Build for communities and groups
For data (reports, research, intelligence)	Model 7	Sell the data

This removes a huge risk. **You don't have to convince anyone to start spending.** You redirect money that's already being spent.

Question 3: Who does the work?

"A person watching a screen points left on the map. An agent in a loop points right." This is the horizontal axis of the map from the Introduction. On the left, a human does the work with AI as a helper, and products succeed through experience and relationships. On the right, an agent works on its own while humans supervise, and products succeed through reliability, documentation, and scale. **Lean left** if you're good with people, design, clients, and community. **Lean right** if you're comfortable with systems, automation, and technical building.

> **PLAIN ENGLISH**
>
> Question 1 finds your niche. Question 2 finds your customer and their budget. Question 3 finds the kind of product you'll enjoy building. Where the three answers overlap is your model.

The Decision Process, Step by Step

Step 1: Take inventory of your advantages (days 1–2)

Write honest answers in three groups. **What I know:** jobs I've held and what I did day to day; industries I understand from the inside; problems people ask me for help with; hobbies I could talk about for hours; groups I belong to; methods I've used or taught. **Who I can reach:** past clients, coworkers, and employers; communities where I'm known; any audience I have. **My resources:** hours per week, money I can invest, how soon I need income, and my technical comfort from 1 (email and spreadsheets) to 5 (I write code).

> **TRY THIS**
>
> Ask an AI assistant: "Interview me one question at a time to inventory my skills, knowledge, networks, and resources for starting a business." Answering questions is far easier than filling in a blank page. The Workbook's Advantage Inventory gives you a structured form.

Step 2: Score yourself against all eight models (days 2–3)

Rate each model from 0 to 3 on four criteria: Do I have the required advantage? Can I handle the technical lift? Does it fit my income timeline? Do I want to do this work every day? The two highest totals are your candidates. A blank scorecard is in the Workbook.

TABLE 9.3 • EXAMPLE SCORECARD: A FORMER RESTAURANT MANAGER WHO LOVES HOME ESPRESSO

Model	Have it?	Tech OK?	Timeline fits?	Want it?	Total
1. Services (restaurant bookkeeping)	3	3	3	2	**11**
2. Harness (restaurant ops)	3	2	2	2	9
3. Headless	1	1	2	1	5
4. Agent-native (manager playbook)	2	3	3	2	10
5. Taste-led (espresso log)	3	2	1	3	9
6. Ambient agent	1	2	1	1	5
7. Data (supplier prices)	2	2	0	2	6
8. Open source	0	1	1	1	3

Here the top candidates are Model 1 (fast income) and Model 4 (a scalable version of the same knowledge). A sensible plan is to start with Model 1, package the method as Model 4 within six months, and keep the espresso app as a side project.

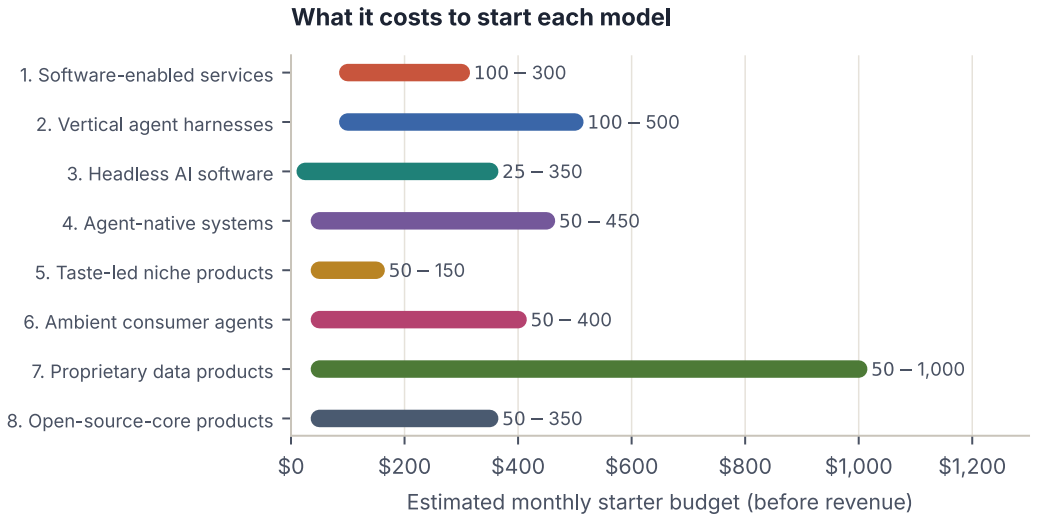


Figure 9.3 Estimated monthly starter budgets before revenue. Every model can be started for less than the cost of one employee; data products vary most, depending on how aggressively you pay to seed data.

Step 3: Apply the three questions to your top two (days 3–4)

For each candidate, complete three sentences: *"I know ___ better than almost anyone because ___."* • *"Right now, ___ pays ___ to ___ for this."* • *"In my product, ___ (a person or an agent) does the work, and the user's role is ___."* If you can't fill in the second sentence, that's a warning sign. Pick a niche where money already flows. To research it, compare prices of existing software on review sites, check what freelancers charge on freelance marketplaces, read complaints about paying for this in forums, and ask an AI assistant to list how businesses in the niche currently pay to solve the problem, with typical prices.

Step 4: Think about stacking (day 4)

The models work best in sequence. If you need income soon, start with Model 1 or Model 4, which make money fastest with the least building, and use that income and learning to fund slower, more defensible models. Chapter 10 shows common paths.

Step 5: Validate before you build (days 5–7)

1. **List twenty potential customers** from your network and communities.

2. **Have ten conversations** of fifteen to twenty minutes. Ask: How do you handle this today? What does it cost in money and time? What's most frustrating? If I could ___, would that be worth $___ to you?
3. **Try a simple pre-sale:** a one-page site with one offer and a "join the waitlist" or "book a call" button, plus a payment link for deposits.
4. **Decide:** three or more people say "yes, I'd pay," or actually pay a deposit, means go. Lots of "interesting" but no commitments means adjust the offer or test your second choice.

The Quick Decision Shortcut

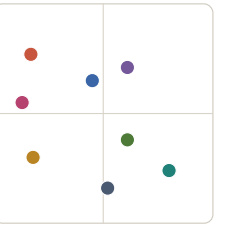
If this sounds like you...	Start with
"I've done a service job for years and know people who pay for it."	Model 1
"I coach, consult, or teach a framework."	Model 4
"I know one industry inside out, and software serves it badly."	Model 2
"I'm obsessed with a hobby and active in its community."	Model 5
"I'm always the organizer in my family, team, or club."	Model 6
"My community has valuable information nobody's collecting."	Model 7
"I can code, or I'm willing to learn with AI help, and want developer customers."	Model 3 or 8

CHAPTER SUMMARY

- Match the model to the advantage you already have; that's your head start.
- Ask three questions: What do I know? Who already pays, and how? Who does the work?
- Score all eight models; your top two are your candidates.
- Validate with ten conversations and a pre-sale before building.
- If you need income soon, start with Model 1 or 4 and stack slower models later.

Review Questions

1. If your potential customers currently pay freelancers for a service, which model does the chapter point you to first?
   A. Model 1
   B. Model 5
   C. Model 7
   D. Model 8
2. On the map, “an agent in a loop” points…
   A. Left
   B. Right
   C. Up
   D. Down
3. What result in validation means “go”?
   A. Ten people say it’s interesting
   B. Three or more people say they’d pay, or pay a deposit
   C. A friend likes the logo
   D. A competitor exists
4. Why is “Who already pays for that today?” such a powerful question?
5. Which two models earn fastest with the least building, and why?
6. List the four scoring criteria used in the scorecard.
7. Write the three sentences from Step 3 for your own top choice.



10

Your First Ninety Days

One model, one niche, one offer, ten conversations, and a weekly habit of getting better.

LEARNING OBJECTIVES

- Follow a ninety-day plan from decision to paying customers
- Set up the business basics every model needs
- Run a weekly review that catches each model's trap early
- Stack models over time into a business that's hard to copy

Choosing a model is the beginning, not the work. This chapter gives you a common ninety-day structure that works for every model, the business setup you'll need regardless of which one you choose, a weekly review habit, and a map of how the eight models stack into something bigger over time.

The Ninety-Day Plan

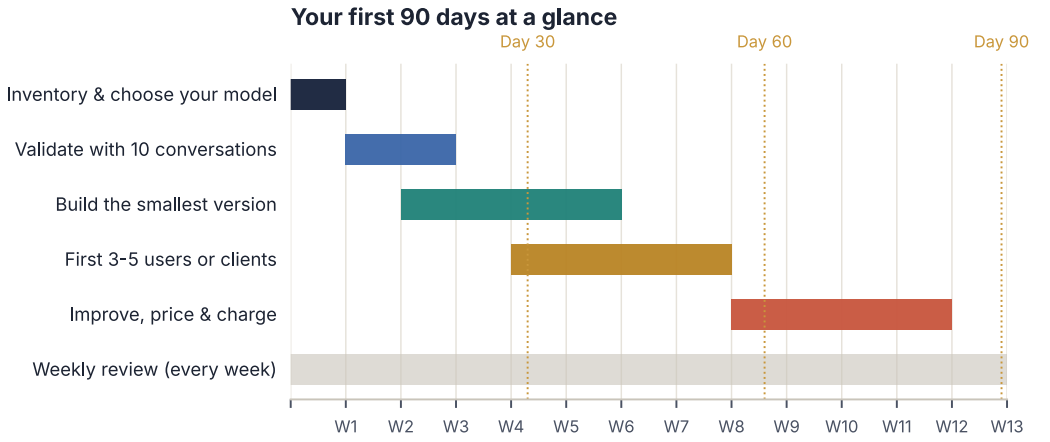


Figure 10.1 The first ninety days at a glance. Decide and validate in the first three weeks, build the smallest useful version by week six, serve your first users by week eight, and spend the last month improving, pricing, and charging. The weekly review runs throughout.

TABLE 10.1 • MILESTONES BY PHASE

Phase	Weeks	Goal	Done when...
Decide	1	Inventory, scorecard, three questions	You can state your model, niche, and customer in one sentence
Validate	2–3	Ten conversations and a pre-sale	Three people have said yes or paid a deposit
Build the smallest version	3–6	One deliverable, skill, tool, or feature that works	A real user has received real value from it
First users	5–8	Three to five users or clients	You have real feedback, and ideally a first payment
Improve and charge	9–13	Fix friction, set prices, sell	Five to ten paying customers

Each model chapter's step-by-step plan slots into this structure: its early phases map to "build the smallest version," its launch phase to "first users," and its monetize phase to "improve and charge."

Business Basics Every Model Needs

TABLE 10.2 • THE UNIVERSAL TOOLKIT, BY CATEGORY

Need	Tool category
AI thinking partner and builder	An AI assistant subscription, plus an AI coding assistant if you'll build software
Notes and planning	An all-in-one workspace or online documents
Leads and customers	A free CRM, spreadsheet-database, or spreadsheet
Landing page	A no-code website builder
Forms and waitlists	An online form builder
Scheduling	An online scheduling link
Payments	A payment processor with payment links and subscriptions
Owned audience	An email newsletter platform
Legal structure	A limited liability company formed through your state's business office or a formation service, with a registered agent if required

Need	Tool category
Banking and bookkeeping	A separate business bank account and accounting software (free options exist)
Protection	Clear contracts or terms of service, a privacy policy, and liability insurance where appropriate

> **A NOTE ON TAXES AND LEGAL SETUP**
>
> Keep business and personal money separate from day one, set aside a portion of revenue for taxes, and save every receipt. Rules on business formation, sales tax, and self-employment tax vary by location, so an hour with a local accountant early on is one of the best investments you can make.

The Weekly Review

Block thirty minutes at the same time every week, and answer five questions in a simple tracker:

- ☐ **Customers:** How many conversations, sign-ups, and paying users this week?
- ☐ **Revenue:** What came in?
- ☐ **Learning:** What surprised me? What did customers ask for?
- ☐ **Trap check:** Am I falling into my model's trap? (See Table 10.3.)
- ☐ **One priority** for next week.

TABLE 10.3 • THE EIGHT TRAPS, AND THE EARLY WARNING SIGN FOR EACH

Model	Trap	Early warning sign
1. Services	Becoming an agency with headcount	Revenue per person flat or falling
2. Harness	A thin wrapper	A new model release causes cancellations
3. Headless	Labs bundle it free	A major assistant announces a similar built-in feature
4. Agent-native	Platform risk	Most customers are unknown to you except through the platform
5. Taste-led	Building for everyone	Features requested by outsiders crowd the roadmap
6. Ambient agents	Novelty	Under 40% of groups active after four weeks
7. Data	No consent, or data nobody needs	No buyer conversations; unclear consent language
8. Open source	No reason to pay	Growing stars, flat cloud sign-ups

Stacking Models Over Time

The strongest businesses rarely stay in one model. They start with one that earns money quickly, then add models that are harder to copy.

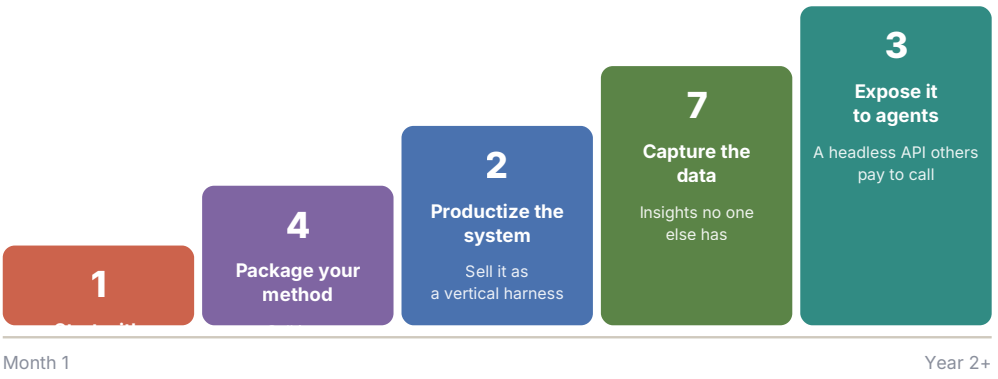


Figure 10.2 One common stacking path. A service business (1) earns cash and refines a method; the method becomes a plugin (4); the system becomes a harness (2); the work generates proprietary data (7); and the data is offered to agents through a headless API (3).

TABLE 10.4 • COMMON STACKING PATHS

Your starting advantage	Start with	Then expand to
Service experience	Model 1 (fast cash)	Model 4 (method as plugin) → Model 2 (system as software) → Model 7 (data from all your jobs)
Coaching or teaching method	Model 4	Model 1 (done-for-you tier) → Model 2
Hobby or passion	Model 5	Model 7 (community data) → Model 3 (API for that data)
Group or family organizing	Model 6	Model 7 (booking and preference data, with consent)
Technical skills	Model 3 or 8	Model 2 (a harness built on your own tools)

All Eight Models, Side by Side

TABLE 10.5 • THE COMPLETE COMPARISON

Model	One-line idea	Tech skill	Monthly start cost	Time to first $	Best for
1. Services	Be the firm that's 10× faster	Low	$100–$300	Fastest (weeks)	Service-business veterans
2. Harness	AI built for one niche's workflow	Medium	$100–$500	Months	Deep niche experts
3. Headless	Tools agents use	High	$25–$350	Months	Developers, data owners
4. Agent-native	Your method as a plugin + dashboard	Low–med	$50–$450	Weeks to months	Coaches, consultants, educators
5. Taste-led	Build for 1,000 true fans	Low–med	$50–$150	Months	Hobbyists with a community
6. Ambient agents	An AI member of the group chat	Med–high	$50–$400	Months	People who know a group's rituals
7. Data	Collect data nobody else has	Med–high	$50–$1,000	Slowest	Insiders in undocumented worlds
8. Open source	Free code, paid cloud and support	High	$50–$350	Months	Builders who like working in public

Start with what you know. Sell to people who already pay. Improve every week. Stack what's hard to copy.

CHAPTER SUMMARY

- Ninety days is enough to decide, validate, build a small version, serve first users, and start charging.
- Every model needs the same business basics: a legal entity, separate banking, simple bookkeeping, payments, and an owned audience.
- A thirty-minute weekly review, including a trap check, keeps you honest.
- Stack models over time, from fast-earning models toward ones that compound and are hard to copy.

Review Questions

1. What should be true by the end of the validation phase?
   A. The product is finished
   B. Three people have said yes or paid a deposit
   C. You have 1,000 followers
   D. You've raised money
2. Which is the early warning sign of Model 6's trap?
   A. Revenue per person falling
   B. Under 40% of groups active after four weeks
   C. Growing stars with flat sign-ups
   D. A new model release
3. List five items from the universal toolkit every model needs.
4. Describe one stacking path that fits your situation.
5. What five questions make up the weekly review?

Afterword

Every period of cheap, powerful new technology produces the same two reactions. Some people wait to see what the experts will do with it. Others notice that the experts don't know their corner of the world nearly as well as they do, and get to work.

The eight models in this book are different routes to the same destination: a business built on something AI can't generate for itself. That might be your judgment as a professional, your method as a coach, your taste as a fan, your understanding of a group's rituals, data you gather with care and consent, or the trust of a community you build in public. The technology is the same for everyone. What you bring to it isn't.

You don't need to master all eight. You need one model, one niche, one offer, and ten honest conversations. Then you improve a little every week. Start with what you know, sell to people who already pay, and keep an eye on your model's trap. The rest of this book will be waiting when you're ready to stack the next one.

The Workbook that follows is where reading turns into doing. Fill it in with a pencil, change your mind, and fill it in again.

Same tools as everyone else. Different knowledge. That's the whole advantage, and it's yours.

The Workbook

Seven worksheets that take you from “which model?” to a running weekly habit. Print them, or copy them into a notebook. Original purchasers may reproduce these pages for personal use.

Sheet	Worksheet	Use it when
W1	Advantage Inventory	Before choosing a model (Chapter 9, Step 1)
W2	Model Scorecard	To compare all eight models (Chapter 9, Step 2)
W3	The Three Questions	To test your top two candidates (Chapter 9, Step 3)
W4	Customer Conversation Tracker	During validation (Chapter 9, Step 5)
W5	One-Page Offer	Before you build or sell
W6	Ninety-Day Planner	Throughout your first quarter (Chapter 10)
W7	Weekly Review	Every week, forever (Chapter 10)

WORKSHEET W1

Advantage Inventory

WHAT I KNOW

Jobs I've held, and what I actually did day to day:

Industries I understand from the inside:

Problems people regularly ask me to help with:

Hobbies or passions I could talk about for hours:

Groups I belong to (family, teams, clubs, congregations, associations):

Methods or frameworks I've used or taught:

WHO I CAN REACH

Past clients, coworkers, and employers:

Communities where I'm known; any audience I have:

MY RESOURCES

Hours per week: _____ Money I can invest per month: $_____ I need income within: _____ months

Technical comfort (circle): 1 2 3 4 5

WORKSHEET W2

Model Scorecard

Score each criterion 0–3 (0 = not at all, 3 = a real strength). Your two highest totals are your candidates.

Model	Have the advantage?	Can handle the tech?	Fits my timeline?	Want to do it daily?	Total
1. Software-enabled services					
2. Vertical agent harnesses					
3. Headless AI software					
4. Agent-native systems					
5. Taste-led niche products					
6. Ambient consumer agents					
7. Proprietary data products					
8. Open-source-core products					

My top two candidates: 1. _____ 2. _____

Why these fit me better than the others:

WORKSHEET W3

The Three Questions

CANDIDATE A: _____

I know _____ better than almost anyone because...

Right now, _____ pays $_____ to _____ for this.

In my product, (a person / an agent) does the work, and the user's role is...

CANDIDATE B: _____

I know _____ better than almost anyone because...

Right now, _____ pays $_____ to _____ for this.

In my product, (a person / an agent) does the work, and the user's role is...

DECISION

I'm starting with _____, because...

The model I'll stack next, and roughly when:

WORKSHEET W4

Customer Conversation Tracker

Goal: ten conversations. Go means three or more say “yes, I’d pay,” or pay a deposit.

#	Name / role	How they handle it today	Cost (money / time)	Biggest frustration	Would pay? $
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					

Pattern I heard most often:

WORKSHEET W5

One-Page Offer

Who it's for (one specific customer type):

The problem, in their words:

What they get (scope, deliverable, or core features):

How it's different from the old way (faster, cheaper, better):

Price and terms (retainer, subscription, per use, per outcome):

Proof (a result, a demo, a testimonial, a before-and-after):

The one call to action:

My model's trap, and the early warning sign I'll watch:

WORKSHEET W6

Ninety-Day Planner

Week	Phase	This week's one priority	Done? / Notes
1	Decide		
2	Validate		
3	Validate		
4	Build		
5	Build		
6	Build / first users		
7	First users		
8	First users		
9	Improve & charge		
10	Improve & charge		
11	Improve & charge		
12	Improve & charge		
13	Review the quarter		

Day-90 goal (customers, revenue, learning):

WORKSHEET W7

Weekly Review

Thirty minutes, same time every week. Copy this page as often as you need.

Week of: _____

Customers: conversations ____ sign-ups ____ paying ____

Revenue this week: $_____ Total to date: $_____

What surprised me:

What customers asked for:

Trap check: am I sliding into my model's trap? What's the evidence?

One thing I'll improve in my system, playbook, or product:

One priority for next week:

Answer Key

Suggested answers to every chapter's review questions. For short-answer questions, these are model answers; yours may be worded differently and still be correct.

Chapter 1 · Software-Enabled Services

1. **B.** Finished professional work, produced with AI systems and a small team.
2. **C.** Labor costs fall because one person can serve far more clients.
3. **B.** Quietly turning back into an agency with headcount.
4. Clients judge value against their alternative: what a traditional firm charges and how long it takes. Your low cost becomes margin and growth budget. Pricing at cost leaves money on the table and can signal low quality.
5. Do I know this work? Is it repetitive? Is it mostly text, documents, or data? Do clients already pay for it? Is it unregulated, or can I partner around the regulation?
6. A point where a qualified person checks AI-produced work before it reaches the client. It catches errors, protects trust, and is the judgment the client is actually paying for.
7. Revenue per person. Aim for roughly $300,000–$1,000,000+, versus about $100,000–$150,000 at traditional firms.
8. Partner with a licensed professional who supervises and signs off, or serve only the unregulated portion of the work.

Chapter 2 · Vertical Agent Harnesses

1. **C.** Advertising.
2. **B.** Users realize they can get the same result elsewhere, and it loses value.
3. **B.** The job with the highest combined score.
4. Vertical products serve one industry deeply (a report tool for home inspectors); horizontal products serve everyone broadly (a general chat assistant).
5. It proves the AI can do the job cheaply and quickly, before you spend money on software. If it doesn't work in a workspace, software won't fix it.
6. Any four: proprietary knowledge, integrations, customer memory, encoded workflow, rules and compliance, evaluation sets.
7. It's the price of general assistants, so it signals "wrapper" and undervalues the hours the harness saves.
8. Model 1 sells finished work, Model 2 sells a tool professionals use, and Model 4 sells a method that installs into the customer's own AI.

Chapter 3 · Headless AI Software

1. **B.** The documentation and the output format.
2. **C.** A continuously updated database of building-permit rules.
3. **B.** Agents parse structured data more reliably.

4. CLI: type one command, get a result (a vending machine). API: send a formatted request, get data back (a takeout window). MCP: a standard plug any compatible agent can use (a universal port).
5. Any four: fresh or hidden data, following standards precisely, exact calculations, reading messy files, real-world actions, verifying output, remembering.
6. Give an agent only your docs and ask it to complete a task. If it struggles, rewrite the docs.
7. Agent usage can spike unexpectedly. Caps prevent surprise bills and build trust with developers.
8. Builders with a data source, and builders with one hard, narrow skill.

Chapter 4 · Agent-Native Systems

1. **B.** The customer's own AI agent, following the expert's method.
2. **C.** To let customers watch progress and approve decisions.
3. **B.** Owning an email list and community, and supporting multiple platforms.
4. Skills (instructions and examples), commands (shortcuts that start workflows), sub-agents (specialized delegates), connectors (links to other apps).
5. Courses must be studied and are often unfinished; a plugin does the work using the method the course would have taught.
6. Diagnose the situation, plan what to do, then do it. Answers will vary by method.
7. What the skill does and when the AI should use it.
8. Give away the diagnose skill to attract users; charge for the plan and do skills, the dashboard, and the community.

Chapter 5 · Taste-Led Niche Products

1. **B.** $100,000.
2. **B.** Building for everyone instead of the inner circle.
3. **A.** Hobbyists tracking things in spreadsheets.
4. Insider taste and understanding can't be copied, big companies ignore small groups, and passionate communities are loyal and spread the word.
5. The one thing a product does better than anything else. Examples vary, such as logging an espresso shot in ten seconds.
6. Any four: one-time purchases, supporter tiers, sponsors, affiliates, merchandise, events.
7. A free core grows the community and word of mouth; you charge for "more."
8. Any three: shareable moments, community creators, in-person events, authentic content, referrals.

Chapter 6 · Ambient Consumer Agents

1. **B.** In the background, available without opening a separate app.
2. **B.** Fewer than 40% of groups active at week four.
3. **C.** The organizer who was stuck doing the job.
4. Every action is visible to all group members, each of whom belongs to other groups. Every use is a demo.
5. Any three: speak only when called or useful; confirm before spending; keep messages short and friendly; remember; be proactive at the right moments; be easy to pause and remove.
6. They have official, free bot support that works in groups (or, for SMS, universal reach), while other channels restrict bots.
7. Consent and introduction, minimal data, children's-privacy compliance, a clear privacy policy, AI providers that don't train on data, and security.
8. Examples: a weekly team schedule, family weekend plans, a weekly dinner, chores, a running-club route.

Chapter 7 · Proprietary Data Products

1. **B.** Nobody contributes until there's data, and there's no data until people contribute.
2. **B.** Showing averages only when a group has enough contributors.
3. **B.** Building a dataset nobody needs.
4. People use the app → unique data accumulates → the product improves → more people join, and the loop repeats.
5. Answers a money question; hard to get elsewhere; fresh; granular; trustworthy; legally clean.
6. Contributors share their own data to unlock everyone else's. Examples vary.
7. Dense data in one area is accurate and useful; thin data spread everywhere is neither, and small counts can identify people.
8. Any four: premium insights, reports, dashboards, API access, an MCP server, bulk licensing.

Chapter 8 · Open-Source-Core Products

1. **B.** The hosted cloud version.
2. **C.** AGPLv3.
3. **B.** Giving everything away with no reason to pay.
4. Customers can inspect the code and can always self-host if the company disappears, so there's no lock-in fear.
5. Hosting, paid tiers, support, community.
6. What it is and who it's for, a screenshot, a two-minute quick start, one-command install, the license, and a link to the hosted version.
7. It destroys community trust. Add paid value on top instead.
8. "Like [expensive product], but open source and cheaper." It's instantly understood and targets known pain around price and lock-in.

Chapter 9 · How to Pick Yours

1. **A.** Model 1.
2. **B.** Right.
3. **B.** Three or more people say they'd pay, or pay a deposit.
4. It redirects money already being spent, so you don't have to create a new habit or budget.
5. Models 1 and 4. They sell expertise you already have and need little building, so you can charge almost immediately.
6. Having the advantage; handling the technical lift; fitting your income timeline; wanting to do the work every day.
7. Answers are personal.

Chapter 10 · Your First Ninety Days

1. **B.** Three people have said yes or paid a deposit.
2. **B.** Under 40% of groups active after four weeks.
3. Any five from Table 10.2.
4. Answers are personal; see Table 10.4.
5. Customers, revenue, learning, trap check, one priority.

Glossary

Affiliate commission
A share of a sale paid to you for referring a customer to another seller's product.

Agent (AI agent)
A program that pursues a goal by deciding on steps and using tools, without a person directing every action.

Agent-native system
A business that packages a method as a plugin for a customer's AI assistant, usually with a dashboard for oversight (Model 4).

Aggregation
Combining individual records into group statistics such as averages.

AGPL (AGPLv3)
A strong copyleft license requiring anyone who runs a modified version as a network service to share their changes.

Ambient agent
An AI that lives in the background of everyday tools, such as group chats, and acts when needed (Model 6).

Anonymization
Removing details that could identify a person from data.

API
Application programming interface: a defined way for software to request data or actions from other software.

Bot API
A messaging platform's official interface for automated accounts.

Churn
The share of customers who cancel in a given period.

CLI
Command-line interface: a program operated by typed commands in a terminal.

Cold start
The early stage when a data or network product has too little data or too few users to be useful.

Connector
A link that lets an AI agent use another application.

Contributor license agreement (CLA)
An agreement giving an open-source project the rights it needs to outside contributions.

Copyleft
A licensing approach that requires derivative works to be shared under the same terms.

Core action
The single thing a product does better than anything else.

Credits
Prepaid units customers spend down as they use a service.

Data flywheel
A loop in which use generates data, data improves the product, and a better product attracts more use.

Deliverable
The specific output a client receives.

Dual licensing
Offering the same code under an open-source license and, for a fee, a commercial license.

Evaluation set (evals)
A collection of real test cases with known-good answers used to measure output quality.

Fair source / source-available
Licenses that make code visible and usable but restrict competing commercial services.

Give-to-get
A collection strategy where users contribute their data to unlock everyone else's.

Gross margin
Revenue minus the direct cost of delivering the product, as a percentage of revenue.

Harness
The tools, knowledge, rules, memory, and workflows built around an AI model for one job (Model 2).

Headless software
Software without a graphical interface, used by other programs and agents (Model 3).

Horizontal product
A product built for many industries or everyone.

Hosted cloud
A vendor-run version of software that customers pay to use without managing servers.

Human-in-the-loop review
A checkpoint where a qualified person reviews AI output before it's used.

JSON
A lightweight, structured text format for data.

Lead magnet
Something valuable given free in exchange for contact details.

llms.txt
A plain-text file summarizing a product's documentation for AI agents.

MCP (Model Context Protocol)
An open standard that lets compatible AI agents discover and use external tools and data.

Moat
A durable advantage that makes a business hard to copy.

Open core
A free open-source core product with paid features for businesses.

Open source
Software whose code is public under a license permitting use, modification, and sharing.

Per-outcome pricing
Charging for a measurable result rather than for time or access.

Platform risk
Dependence on a platform whose rules, fees, or features you don't control.

Playbook
A reusable set of instructions and examples that tells an AI how to perform a step your way.

Plugin
An installable package that adds skills, commands, sub-agents, or connectors to an AI assistant.

Positioning map
This book's two-axis chart placing the eight models by who does the work and what is sold.

Productized service
A service sold with a fixed scope, price, and turnaround, like a product.

Proprietary data
Data only you possess, gathered through your own product or relationships (Model 7).

Re-identification
Working out a person's identity from supposedly anonymous data.

Retainer
A recurring fee for an agreed scope of ongoing work.

Retention
The share of users or groups still active after a period of time.

Retrieval
Searching a knowledge base and giving the most relevant material to an AI model as context.

Revenue per person
Total revenue divided by team size; a key efficiency measure.

Roll-up
Buying several traditional firms and combining or transforming them.

Self-hosting
Running software on your own servers.

Skill
Written instructions, templates, and examples that teach an AI to do a task a particular way.

SLA
Service-level agreement: a contractual promise about uptime or response times.

Software-enabled service
A firm that sells finished work produced with AI systems and a small team (Model 1).

Sub-agent
A specialized AI assistant the main agent can delegate a narrow task to.

Taste
An insider's sense of what matters and what's missing in a community's world (Model 5).

Thin wrapper
A product that adds little beyond a prompt and an interface on top of a general model.

True fan
A customer who will use, pay for, and promote what you make.

Usage-based pricing
Charging per unit consumed, such as per call, page, or search.

Validation
Testing demand with real potential customers before building.

Vertical product
A product built deeply for one industry or user type.

Weekly active groups
The number of groups that used a group product in a given week.

Tool Categories Directory

This guide names tools by category so its advice outlasts any single product. Use this directory to understand what each category does and what to look for when you compare options. Most categories offer free tiers or trials, so test two or three before committing.

AI and building

Category	What it does	What to look for	Typical cost
AI assistant subscription	Chat-based AI for drafting, analysis, research; projects or workspaces hold instructions and files	Projects/workspaces, file uploads, plugin or skill support, business plan that doesn't train on your data	$20–$30/ user; higher tiers to ~$200
AI coding assistant	Writes, edits, and runs code from plain-language requests	Works in your terminal or editor, can run tests, explains its changes	$20–$200
AI app builder	Generates working web or mobile apps from descriptions	Built-in database and login, code export, custom domains	$20–$50
Frontier AI model API	Lets your software call AI models directly	Quality on *your* task, tool use, data-retention terms, price per token	Pay per use
Agent SDK / framework	Handles multi-step agent loops, tools, and retries	Documentation, community, support for MCP	Often free / open source
Vector database / search	Stores knowledge for retrieval by meaning	Hybrid search, filters, hosting options	$0–$70+
Document-parsing service	Reads PDFs, scans, tables, receipts into structured data	Accuracy on your documents, table handling	Pay per page
Evaluation & observability	Logs AI calls and tests output quality against real cases	Test datasets, cost tracking, alerts	$0–$100+
Speech APIs	Speech-to-text and text-to-speech	Accuracy, latency, languages, voice quality	Pay per minute

Operations and automation

Category	What it does	What to look for	Typical cost
No-code automation platform	Connects apps with triggers and actions	Integrations you need, AI steps, error handling; self-hostable options are cheapest at scale	$0–$70
Spreadsheet-database	Spreadsheet-style database for clients, jobs, and queues	Views, forms, automations, API access	$0–$25/user
All-in-one workspace	Docs, wikis, and lightweight databases	Templates, sharing, AI features	$0–$15/user
Online form builder	Intake forms, surveys, waitlists	File upload, logic, automation triggers	$0–$30
AI meeting recorder	Records, transcribes, and summarizes calls	Accuracy, consent notices, CRM sync	$0–$20
Online scheduling link	Lets people book time on your calendar	Calendar sync, reminders, payments	$0–$15
Proposal & e-signature	Sends proposals and collects signatures	Templates, payment collection	$0–$40
Background job scheduler	Runs reminders and scheduled tasks reliably	Retries, monitoring	$0–$20

Infrastructure

Category	What it does	What to look for	Typical cost
Hosted database & login	Database, authentication, file storage in one service	Backups, row-level security, free tier	$0–$25
App-hosting platform	Runs web apps, APIs, and bots	Simple deploys, scaling, logs	$0–$25
Authentication service	User sign-in, including enterprise single sign-on	Social login, SSO, user management	$0–$25+
API-key management & rate limiting	Issues keys and controls usage for headless products	Usage analytics, per-key limits	$0–$25
Error tracking & product analytics	Catches bugs; shows what users do	Free tiers, privacy controls	$0–$50
Containers	Package software for one-command install anywhere	Compose files, registry hosting	Free
Public code repository	Hosts code, issues, discussions, automated tests	Free public and private repositories	Free
Documentation-site generator	Builds searchable docs sites	Markdown, API reference, llms.txt output	$0–$150

Selling and growing

Category	What it does	What to look for	Typical cost
Payment processor	Payment links, subscriptions, invoices, usage billing	Recurring billing, tax handling, payouts	~3% per transaction
Usage-billing platform	Meters and bills per-use products	Credits, caps, invoices	% of revenue or tiered
Digital-sales platform	Sells downloads, licenses, or repository access	License keys, automatic access grants	% of sales
Mobile subscription management	Handles in-app subscriptions across stores	Cross-platform, analytics	Free to start, then % of revenue

Category	What it does	What to look for	Typical cost
No-code website builder	Landing pages and marketing sites	Templates, forms, custom domain	$0–$20
Email newsletter platform	Your owned audience	Automations, segmentation, free tier	$0–$40
Community platform	Members-only forums, courses, events	Payments, mobile app, moderation	$0–$99
Community chat server	Real-time chat for hobby and developer communities	Roles, bots, events	Free
Free CRM	Tracks leads, conversations, deals	Email sync, pipeline views	$0
B2B contact database	Finds companies and decision-makers	Data accuracy, export limits	$0–$100+
Cold-email sending tool	Sends personalized outreach at scale	Deliverability tools, compliance features	$30–$100
Affiliate programs & networks	Pay commissions on products you recommend	Commission rates, cookie length	Free to join
Print-on-demand merchandise	Produces and ships merch without inventory	Product quality, store integrations	Per item

Trust, privacy, and legal

Category	What it does	What to look for	Typical cost
Privacy-policy & consent generator	Drafts policies and consent banners	Coverage of your jurisdictions; still get legal review	$10–$30
Compliance automation	Helps earn security certifications larger customers ask for	Templates, auditor network	Higher; adopt when customers require it
Business formation service	Forms an LLC and provides a registered agent	State coverage, ongoing compliance reminders	~$50–$500 one time
Professional liability insurance	Covers claims of errors in your work	Coverage limits, industry fit	~$40–$100
Accounting software	Bookkeeping, invoices, tax-ready reports	Bank feeds, receipt capture	$0–$40

Index

#

1,000 true fans, 80

A

Agent, 4
Agent-native system, 63
Ambient agent, 93
API (application programming interface), 53

C

CLI (command-line interface), 53
Cold start, 114

D

Data flywheel, 109

E

Evaluation set, 38

G

Give-to-get, 115

H

Headless software, 47
Human-in-the-loop review, 20

L

License (open source), 129

M

MCP (Model Context Protocol), 48

O

Open core, 128
Open-source core, 125

P

Platform risk, 65
Playbook, 19
Positioning map, 6
Proprietary data, 109

R

Re-identification, 115
Revenue per person, 21
Roll-up, 21

S

Skill (plugin), 64
Software-enabled services, 13

T

Taste-led niche product, 79
Thin wrapper, 32

U

Usage-based pricing, 49

V

Vertical agent harness, 30